HOCHSCHULE
CAMPUS WIEN
UNIVERSITY OF
APPLIED SCIENCES

Course manuscript – Version 1.0

Advanced Web Engineering

Concepts, Architectures, and Tools for Modern Web Development

Leon Freudenthaler, Thomas Winter

September 2026

Hochschule Campus Wien - Computer Science and Digital Communications & Software Design and Engineering

Preface

The Web has changed remarkably quickly. What began as a system for publishing and linking documents has developed into a general-purpose platform for applications that compete traditional desktop software. Modern web applications stream media, process large amounts of data, work across a wide range of devices, communicate with remote services, and can connect many users in real time.

Much of this complexity remains invisible to the users. Opening a web application may appear almost effortless. Enter a URL, wait briefly, and start interacting. Behind that simple experience, however, many technologies cooperate. Networks transfer resources, browsers parse and render documents, JavaScript executes application logic, build tools transform source code, frameworks coordinate state and user interfaces, and servers provide data and communication services. These technologies have evolved together with the expectations placed on the Web. Static documents were followed by dynamically generated pages, asynchronous updates, single-page applications, component-based frameworks, and hybrid rendering approaches and so on.

This rapid development can make web engineering appear to be an endless sequence of new frameworks, libraries, and tools. Individual technologies do change quickly, but many of the problems they address are much more persistent. Understanding those problems provides a more durable foundation than learning one particular framework or tool in isolation.

Why does JavaScript use an event loop? Why are modules and build pipelines useful? Why did client-side applications become popular, and why have modern architectures moved parts of rendering back to the server? How do component-based frameworks manage changing interfaces? What makes an application accessible? What determines whether a page feels fast? And what changes when several users interact with shared state at the same time?

This manuscript approaches web engineering through questions such as these. It combines underlying mechanisms, architectural ideas, and practical engineering concerns. The aim is not only to explain *how* particular technologies are used, but also *why* they exist, what problems they address, and which trade-offs they introduce. We kept the manuscript intentionally modular. Chapters are organized into coherent thematic units and can be revisited independently when a particular topic becomes relevant.

The Web will continue to evolve. The individual tools used to build it will change as well. A solid understanding of the mechanisms and engineering principles underneath those tools makes it easier to follow that evolution, evaluate new technologies, and make informed design decisions.

We hope this manuscript encourages you to look beyond what a web technology does and become curious about what makes it work!

Sincerely, Leon and Thomas

How to read this manuscript

This manuscript is used across different courses and learning contexts in web engineering. It is therefore organised as a collection of thematic parts rather than as a single linear course script. Depending on your course, only a subset of these parts may be relevant. Follow the course schedule or instructions from your lecturer to determine which chapters are required.

Within a thematic part, chapters generally build on concepts introduced earlier. Across parts, however, the manuscript can also be used selectively. Cross-references point to background material when a topic depends on concepts explained elsewhere.

Code examples are intentionally small. They normally isolate one concept rather than represent a complete production application. Where the execution result is important, the corresponding console output or an execution trace is shown directly after the listing. Diagrams likewise use simplified models when this makes the underlying mechanism easier to understand. Such diagrams emphasise the relationships relevant to

the discussion and should not necessarily be interpreted as exact representations of a particular browser, runtime, or framework implementation.

Visual conventions

Several recurring boxes highlight different kinds of information throughout the manuscript.

Key concept

Blue boxes summarize concepts that are central to understanding the surrounding topic. They are useful when reviewing a chapter or identifying its essential ideas.

Common pitfall

Red boxes point out misconceptions, surprising behaviour, and implementation patterns that frequently lead to defects.

Engineering perspective

Green boxes connect the underlying concept to software-engineering decisions, trade-offs, and practical application design.

Tooling note

Orange boxes provide practical information about development tools, browser features, commands, documentation, or useful external resources.

Sources and further reading

The manuscript combines textbooks, peer-reviewed research, standards and specifications, and official technical documentation. The JavaScript chapters draw particularly on Simpson's treatment of scope, closure, prototypes, values, and references. Resig, Bibeault, and Maras for execution contexts, functions, browser lifecycle, events, promises, and debugging. And Bevacqua for modular application design, asynchronous flow, and the Build First philosophy.[1–3]

The chapters on web architectures and modern client-side applications are additionally grounded in research on the evolution and engineering of web applications, component-based architectures, reactive programming, and web performance.[4–10] Later chapters complement current platform specifications with empirical and conceptual research on Progressive Web Applications, accessibility, real-time communication, collaborative systems, and dependable and secure computing.[11–16]

For technologies that evolve rapidly, such as TypeScript, React, browser APIs, Progressive Web Applications, performance metrics, and accessibility requirements, official documentation, specifications, and standards remain important sources for current API semantics and platform behaviour. Scholarly literature is used alongside these sources to provide conceptual foundations, empirical evidence, and a broader engineering perspective. Some of the cited research is intentionally foundational. Older publications are retained where they introduced or established concepts that remain relevant to contemporary web engineering. References throughout the chapters provide more specific sources and starting points for further reading.

Contents

Advanced JavaScript and Browser Environments

1 How JavaScript Executes

JavaScript is often called an “interpreted language”, but that label hides the part of the execution model that matters for developers. Before ordinary statements produce effects, an engine reads and parses the program, creates internal representations, registers declarations, and prepares executable code. A syntax error near the end of a file can therefore prevent an earlier `console.log` from running. A useful distinction is the distinction between *preparation* and *execution*.[1, Chapter 1]

1.1 From source text to execution

The exact internals differ between JavaScript engines. ECMAScript specifies the observable behaviour of programs, but it does not require every implementation to use the same parser, bytecode format, optimiser, or machine-code generator. Nevertheless, the following simplified pipeline provides a useful mental model:

1. the host loads a JavaScript source file or module.
2. the engine tokenises the source text into meaningful lexical units.
3. the engine parses those tokens according to the ECMAScript grammar and constructs an internal representation of the program.
4. declarations, lexical environments, and scopes are prepared.
5. the engine produces executable instructions, for example bytecode or machine code.
6. the program executes inside one or more execution contexts.
7. the host environment supplies external capabilities such as the DOM, timers, networking, storage, and rendering.

These stages should not be interpreted as a fixed implementation recipe. Modern engines combine interpretation, compilation, profiling, and optimisation. The pipeline is useful because it separates two developer-visible phases. The program must first be understood as valid JavaScript, and only then can its ordinary statements execute.

Parsing happens before ordinary execution

```
const greeting = "Hello";
console.log(greeting);

greeting = ."Hi"; // SyntaxError: the program cannot be parsed
```

Console output

```
SyntaxError: Unexpected token '.'
```

There is no `Hello` output. The engine parses the complete script before starting its ordinary evaluation, so the syntax error prevents the program from entering that phase. The precise wording of the error message can differ between engines.

1.2 Tokenisation, parsing, and the AST

Source code initially consists only of characters. During lexical analysis, the engine groups those characters into **tokens**. Identifiers, keywords,

literals, punctuation, and operators that have meaning in the language. Consider the following statement:

A small declaration

```
1 const total = price + tax;
```

A simplified token stream looks as follows:

```
const  total  =  price  +  tax  ;
```

The token stream is flat and records the recognised pieces in source order, but not yet their complete grammatical relationships. Parsing adds this structure. For example, the parser recognises that `total` is the declared binding and that `price + tax` is the expression used to initialise it.

An engine commonly represents the parsed program as an **abstract syntax tree** (AST). The exact node names and representation are implementation-dependent, since the ECMAScript specification does not prescribe one particular AST format. Figure 1.1 therefore shows a deliberately simplified, ESTree-like representation intended for explanation rather than an exact dump from a particular engine.

Why syntax trees matter beyond the engine

Development tools (which we cover in Part 6.10) like Linters, formatters, transpilers, static analysers, and bundlers commonly work with ASTs or comparable structural representations. ESLint can identify a particular kind of expression, TypeScript can analyse declarations and types, and a bundler can follow imports because these tools inspect program structure rather than treating source files as undifferentiated text.

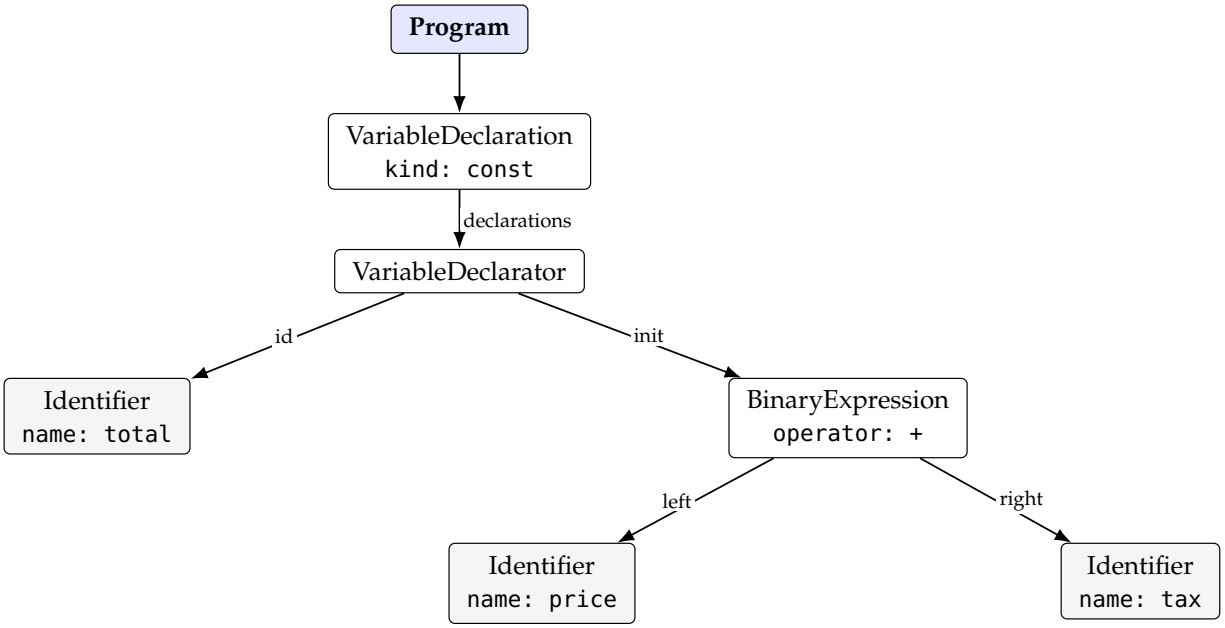


Figure 1.1: A deliberately simplified AST for `const total = price + tax;`.

1.3 The engine and the host environment

JavaScript execution requires cooperation between the **JS engine** and the surrounding **host environment**. The engine implements the ECMAScript language. It parses and executes functions, expressions, objects, modules, and the standard built-in APIs defined by the language. The host connects the program to the outside world.[17]

For example, the following belong to ECMAScript itself:

- language constructs such as functions, classes, and modules.
- standard built-in objects such as `Array`, `Map`, and `Promise`.
- methods such as `Array.prototype.map`.
- operations such as `JSON.parse`.

A web browser supplies the following additional APIs:

- the DOM and methods such as `document.querySelector`.
- timers such as `setTimeout`.
- network access through `fetch`.
- persistent browser storage such as `localStorage`.
- events, navigation, rendering, and user-interface integration.

This distinction becomes visible when the same language runs in another host. Node.js executes JavaScript but does not normally provide a browser document or a `window` object. Instead, it exposes host APIs for files, processes, network sockets, and the operating system.

Language versus host

The JavaScript engine defines how ECMAScript code behaves. The host determines which external capabilities that code can access. Browsers, Node.js, workers, and other environments execute the same core language while exposing different host APIs.

1.4 Execution contexts, stack, heap, and queue

Once a program has been parsed and prepared, JavaScript evaluates it inside **execution contexts**. An execution context records the information needed to continue a particular computation, including the code being evaluated, its lexical environment and bindings, its realm, and the current `this` value where applicable. Global code, module code, and function bodies therefore do not execute in an information-free vacuum.[17]

A function call creates a new context. Active contexts form a last-in, first-out structure called the **call stack**. The newest call occupies the top frame. Returning from that function removes the frame and resumes the caller below it, as illustrated in figure 1.2. [2, Chapter 5]

The stack is only one part of the runtime model. Objects are stored in a region commonly described as the **heap**, while jobs that may execute later wait in a queue. The figure 1.3 illustrates the runtime environment of JavaScript. The diagram also shows the browser host environment because timers, input, networking, storage, and rendering are not implemented by the call stack. A browser API may begin work while JavaScript continues. Once a corresponding callback is eligible, the browser can arrange for later JavaScript execution.

Nested calls create nested stack frames

```
function double(value) {
  return value * 2;
}

function formatScore(score) {
  const result = double(score);
  return `Score: ${result}`;
}

console.log(formatScore(7));
```

Console output

```
Score: 14
```

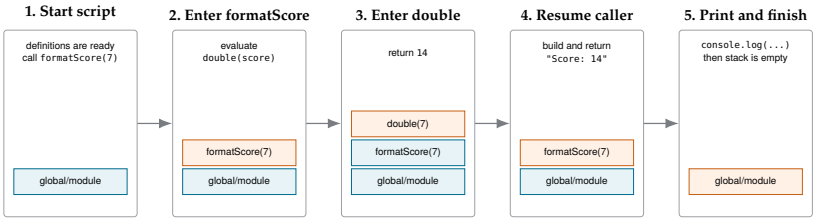


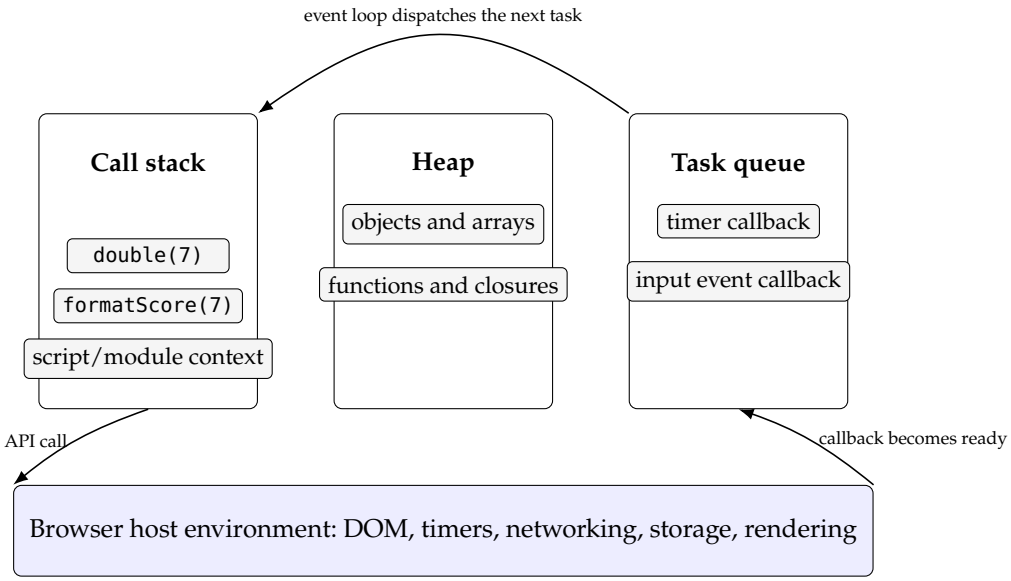
Figure 1.2: Call-stack evolution for the nested function calls. The newest active frame is shown at the top of each stack. Returning removes the top frame and resumes its caller.

At the deepest point, the stack contains the global or module context, `formatScore(7)`, and `double(7)`. When `double` returns, its frame is removed and `formatScore` resumes. When `formatScore` returns, its frame is removed and the caller receives the completed string.

Explore the stack and event loop interactively

Loupe visualises how JavaScript code moves through the call stack, browser APIs, and callback queue. Paste a short example into the tool and step through it. https://latentflip.com/loupe/

The visualisation is especially useful for comparing nested synchronous calls with callbacks scheduled by `setTimeout`.[18]



Conceptually adapted from MDN Web Docs, "JavaScript execution model".

Figure 1.3: A simplified view of the stack, heap, queue, and browser host environment.

1.5 Run-to-completion

JavaScript code on one browser agent normally follows a **run-to-completion** rule, which means once a job begins executing, another ordinary job does not interrupt it halfway through. Each job runs until its stack is empty before the next job begins.[17]

This property removes many low-level interleavings from ordinary JavaScript reasoning. A synchronous function does not normally need to expect a click handler to modify its local state halfway through one statement sequence. The same guarantee creates a responsiveness risk, however. A long-running JavaScript also prevents other work on the same browser thread from progressing.

Synchronous work occupies the main thread

```
console.log("start");

let checksum = 0;
for (let i = 0; i < 500_000_000; i += 1) {
  checksum = (checksum + i) % 97;
}

console.log("done", checksum);
```

The output order is predictable, but the page may appear frozen between the two messages. While the long-running task occupies the main thread, the browser cannot dispatch ordinary input callbacks or paint intermediate changes on that thread.

Run-to-completion does not mean fast completion

The runtime guarantees that the current job is not interrupted by another ordinary JavaScript job. It does not guarantee that the job is short. Expensive loops and synchronous work can still make the interface unresponsive.

1.6 The browser event loop

Interactive applications must react to work that becomes ready later. For example, a timer expires, a network response arrives, or a user clicks a button. The browser manages such operations outside the active JavaScript stack. Once the relevant callback can run, a corresponding task or job is placed into a queue. The **event loop** coordinates when queued work may begin. In the simplified model used here, it can dispatch the next task only after the current JavaScript job has completed and the stack is empty.[2, Chapter 13] [17]

A timer callback runs in a later task

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Console output

```
A
C
B
```

The delay of zero milliseconds means that the callback should become eligible without an intentional waiting period. It does not mean that the callback interrupts the current script. The current script still prints `C` and finishes before the timer callback can begin.

A timer delay is a lower bound, not an appointment

`setTimeout(callback, 0)` schedules a later opportunity to run. A busy call stack, other queued tasks, browser throttling, and scheduling policies can delay that opportunity further.

1.7 Tracing asynchronous execution

The timer example can be read as a sequence of changes to the call stack, the browser's timer machinery, and the task queue. Keeping those parts separate avoids the common misconception that a callback runs immediately when its timer expires.

Table 1.1: Execution trace for the timer example

Moment	Call stack	Browser and queue	Console
1	main script executes the first log	no deferred work	`A`
2	main script calls `setTimeout`	timer is registered	`A`
3	main script executes the final log	timer is pending or already eligible	`A`, `C`
4	main script has finished	callback waits in the task queue	`A`, `C`
5	timer callback executes	callback is removed from the queue	`A`, `C`, `B`

The timer may become eligible while the main script is still running, but it cannot execute until the current job has completed. This distinction between *ready* and *running* is central to reasoning about asynchronous browser code.

1.8 Where Promise scheduling belongs

Promises introduce another scheduling category, commonly described through the **microtask queue**. Promise reactions have different priority rules from timer tasks, but introducing those rules before explaining Promise states and chaining would reverse the conceptual dependency. They are therefore discussed in Chapter 6.

At this point, retain the basic runtime model. Function calls use the stack, objects live in the heap, browser APIs manage deferred work outside the active stack, and queued callbacks can execute only after JavaScript is able to begin another job.

1.9 Chapter summary

What to remember about the JavaScript runtime

- JavaScript source is tokenised and parsed before ordinary execution begins.
- Parsing creates a structural representation of the program. AST-like representations are also central to development tools.
- The JavaScript engine implements ECMAScript, while the host environment provides capabilities such as the DOM, timers, networking, and storage.
- Every active function call occupies an execution context on the call stack.
- Objects and other dynamically allocated data are associated with the heap, while later jobs wait in queues.
- A running job normally continues until its stack becomes empty.
- The event loop dispatches ready callbacks only when JavaScript can begin another job.

Scope, Closures, and Function Forms

2

Large JavaScript programs become difficult when names and effects are reachable from too many places. **Scope** determines where a binding can be resolved. A **closure** determines which outer bindings a function can continue to access. The *form* of a function — declaration, expression, method, or arrow — influences hoisting, `this`, construction, and readability. This chapter builds directly on the runtime model of chapter 1.

2.1 Lexical scope and the scope chain

JavaScript is *lexically scoped*, meaning identifier resolution depends primarily on **where** code is written. If an identifier is not found in the current lexical environment, the engine searches successively outer environments. If no binding exists, evaluation ends with a `ReferenceError`.[2, Chapter 5]

Lexical lookup through nested scopes

```
const applicationName = "Case Atlas";

function createLabel(prefix) {
  const separator = ": ";

  return function label(item) {
    return prefix + separator + item.id + " - " + applicationName;
  };
}

const labelItem = createLabel("Item");
console.log(labelItem({ id: "A-17" }));
```

Console output

```
Item: A-17 - Case Atlas
```

The inner function reads `item` from its own call, `prefix` and `separator` from the surrounding call to `createLabel`, and `applicationName` from the outermost scope.

2.1.1 Shadowing

When an inner scope declares a binding with the same name as an outer one, the inner binding **shadows** the outer binding. Lookup stops at the first match, so the outer binding becomes unreachable by that name inside the inner scope.

An inner binding shadows an outer one

```
const status = "global";

function report() {
  const status = "local";
  console.log(status);
}

report();
console.log(status);
```

Console output

```
local
global
```

Shadowing is legal and sometimes convenient, but a shadowed name forces every reader to track which environment a reference resolves to. Prefer distinct names when both bindings are used near each other.

2.2 Global scope and accidental coupling

A script with many top-level declarations creates a wide shared surface:

A wide global state surface

```
var records = [];
var selectedRecord = null;
var currentView = "overview";
var notes = [];
var filters = {};
```

Any later function can read or replace these values. The dependency is invisible at the call site. A function may appear parameterless while relying on several mutable globals. Modules, presented in chapter 5, reduce this coupling by giving each file its own top-level scope and requiring explicit imports.

2.3 Declarations, hoisting, and the temporal dead zone (TDZ)

Chapter 1 separated *preparation* from *execution*. An engine registers declarations while it creates a scope, before the ordinary statements of that scope run. **Hoisting** is nothing more than the developer-visible consequence of that preparation phase. Source lines do not physically move and bindings simply exist earlier than their declaration lines suggest. Declaration forms differ in *which* scope receives the binding and *when* the binding becomes usable.[1, Chapter 4 and Appendix A]

Table 2.1: Practical differences between declaration forms

Form	Scope	Before declaration line	Reassignment
`var`	function or global	binding exists with `undefined`	allowed
`let`	block	binding exists but is uninitialised (TDZ)	allowed
`const`	block	binding exists but is uninitialised (TDZ)	disallowed
function	surrounding scope	callable after scope creation	allowed*

* Function declarations inside blocks have additional compatibility rules in non-strict code. We use modules, which are always strict, so declarations behave uniformly.

The temporal dead zone exposes an invalid ordering assumption

```
function statusMessage() {
  console.log(status);
  const status = "reviewed";
  return status;
}

statusMessage();
```

Console output

```
ReferenceError: Cannot access 'status' before initialization
```

Prefer `const` when a binding is not reassigned and use `let` when reassignment is part of the algorithm. A `const` binding does not freeze the object it references.

A const binding can still refer to mutable data

```
const item = { id: "A-17", status: "new" };
item.status = "reviewed"; // allowed

console.log(item.status);
```

Console output

```
reviewed
```

2.4 Closures retain environments

Chapter 1 described a call stack whose frames disappear when a function returns, and a heap where objects outlive individual calls. Closures are the point where those two ideas meet. When `createLabel` (lst. 2.1) from the opening example returns, its stack frame is removed — but the lexical environment created for that call is *not* destroyed, because the returned function still references it. The environment survives on the heap for as long as the function does.

Closure is retained lexical access

Closures are not a separate syntax. A function retains access to bindings from the scope in which it was created, even if that outer call has already returned. The retained entities are bindings, not frozen copies of their current values. If the binding changes, the closure observes the change.

Because the retained environment is private to the call that created it, closures provide encapsulation without any dedicated language feature:

Closure-based private state

```
function createIdGenerator(prefix) {
  let counter = 0;

  return function nextId() {
    counter += 1;
    return prefix + "-" + String(counter).padStart(3, "0");
  };
}

const nextCaseId = createIdGenerator("CASE");

console.log(nextCaseId());
```

```
console.log(nextCaseId());
console.log(nextCaseId());
```

Console output

```
CASE-001
CASE-002
CASE-003
```

No other code can read or reset `counter`. The only access path is the returned function. This pattern anticipates the module design of chapter 5, where a module's top-level scope plays a similar encapsulating role.

Closures keep data alive

An environment retained by a closure cannot be garbage collected. A long-lived callback that closes over a large data structure — or over a DOM element — keeps that data reachable for its entire lifetime. When registering long-lived handlers, close over the minimum required state.

2.5 Closures inside loops

Callbacks registered in a loop execute later. With `var`, all callbacks close over the same function-scoped loop binding:

One binding shared by every callback

```
const handlers = [];

for (var i = 0; i < 3; i += 1) {
  handlers.push(() => console.log(i));
}

handlers[0]();
handlers[1]();
handlers[2]();
```

Console output

```
3
3
3
```

With `let`, each iteration has a distinct block-scoped binding:

A distinct binding for every iteration

```
const handlers = [];

for (let i = 0; i < 3; i += 1) {
  handlers.push(() => console.log(i));
}

handlers[0]();
handlers[1]();
handlers[2]();
```

Console output

```
0
1
2
```

The difference is not that the callback “remembers the value”. It closes over a binding, and `let` creates a different binding for each iteration. This is the same principle as in section 2.4 where each iteration of the `let` loop produces its own small environment, and each callback retains its own.

2.6 Function forms

Functions are first-class values in JavaScript. They can be assigned, stored in data structures, passed as arguments, and returned from other functions. Callbacks, higher-order functions, event handlers, and modules follow directly from this property.[2, Chapter 3]

A higher-order transformation

```
const records = [
  { id: "A-17", status: "reviewed" },
  { id: "B-04", status: "new" }
];

const reviewedIds = records
  .filter((record) => record.status === "reviewed")
  .map((record) => record.id);

console.log(reviewedIds);
```

Console output

```
["A-17"]
```

A first-class value must be written down somehow, and JavaScript offers several syntactic forms. They are not interchangeable. The forms differ in hoisting behaviour and, as the following sections show, in how they treat `this`.

A **function declaration** is registered during scope preparation, so it is callable before its line in the source:

A declaration is callable before its source line

```
console.log(double(7));

function double(value) {
  return value * 2;
}
```

Console output

```
14
```

A **function expression** produces a function value during ordinary execution. The surrounding `const` binding follows the temporal dead zone rules from earlier in this chapter, so the equivalent early call fails:

An expression is subject to its binding’s TDZ

```
console.log(double(7));

const double = function (value) {
  return value * 2;
};
```

Console output

```
ReferenceError: Cannot access 'double' before initialization
```

Objects can carry functions as properties using the **method shorthand**, and **arrow functions** provide a compact expression form:

Method shorthand and arrow expressions

```
const stats = {
  count(records) {           // method shorthand
    return records.length;
  }
};

const twice = (value) => value * 2;   // arrow function

console.log(stats.count([1, 2, 3]));
console.log(twice(7));
```

Console output

```
3
14
```

Table 2.2: Function forms at a glance

Form	Example	Available before its line	Own `this`
declaration	`function f() \{\}`	yes (scope preparation)	yes, per call
expression	`const f = function () \{\}`	no (binding in TDZ)	yes, per call
method shorthand	`f() \{\}` inside an object literal	property exists once the literal is evaluated	yes, per call
arrow function	`const f = () => \{\}`	no (binding in TDZ)	no; lexical

The last column of the table is the subject of the next two sections.

2.7 Function invocation and the `this` value

The value of `this` is not determined by where a regular function is defined. It is normally determined by *how the function is called*. For this reason, `this` is sometimes described as an implicit function parameter.[2, Chapter 4]

Consider a function stored as an object property:

A method call supplies the receiving object as this

```
const filterState = {
  term: "camera",

  matches(record) {
    console.log(this.term);
    return record.title
      .toLowerCase()
      .includes(this.term);
  }
};

const result = filterState.matches({
  title: "Depth Camera Log"
});

```

```
console.log(result);
```

Console output

```
camera
true
```

The expression `filterState.matches(...)` is a *method call*. The object before the dot becomes the receiving object, so `this === filterState` while the function executes.

For regular functions, inspect the call site

A regular function does not permanently belong to the object in which it was originally written. To determine `this`, inspect the expression that invokes the function.

The binding can be lost when the function is separated from the object:

Extracting a method changes its call site

```
"use strict";

const filterState = {
  term: "camera",

  matches(record) {
    return record.title
      .toLowerCase()
      .includes(this.term);
  }
};

const detachedMatches = filterState.matches;

detachedMatches({
  title: "Depth Camera Log"
});
```

Console output

```
TypeError: Cannot read properties of undefined (reading 'term')
```

The second invocation is a plain function call. In strict mode, including JavaScript modules, a plain call does not provide a receiving object, so `this` is `undefined`.

JavaScript also provides mechanisms for selecting `this` explicitly:

Selecting this with call and bind

```
function matches(record) {
  return record.title
    .toLowerCase()
    .includes(this.term);
}

const state = { term: "camera" };
const record = { title: "Depth Camera Log" };

console.log(matches.call(state, record));

const matchesCamera = matches.bind(state);
console.log(matchesCamera(record));
```

Console output

```
true
true
```

The `call` method invokes the function immediately with an explicit `this`. The `bind` method creates a new function whose `this` value is fixed for future calls.

A regular function can therefore receive `this` in several ways:

Invocation form	Example	Value of `this`
plain call	`operation()`	`undefined` in strict or module code
method call	`object.operation()`	the object before the dot
explicit invocation	`operation.call(object)`	the explicitly supplied object
bound function	`operation.bind(object)`	the object fixed by `bind`
constructor call	`new Operation()`	the newly created instance

Table 2.3: Common ways a regular function receives `this`

None of these rules apply to arrow functions, which are the subject of the next section. An arrow function cannot be invoked with `new` and ignores the receiver supplied by a method call, `call`, or `bind`.

2.8 Arrow functions and lexical `this`

Arrow functions are concise function expressions, but their main difference from regular functions is semantic rather than syntactic. An arrow function does not receive a `this` value when it is called. A reference to `this` inside an arrow function is resolved exactly like any other outer identifier, namely, through the lexical scope chain described at the start of this chapter. Arrow functions also have no own `arguments` object and cannot be invoked with the `new` statement.[2, Chapters 3–4]

In other words, the previous section's table does not apply to arrows. Whatever the call site looks like, an arrow function's `this` is the `this` of the scope in which the arrow was *written*.

Arrow functions are especially suitable for callbacks that should retain the surrounding context:

A regular method containing an arrow-function callback

```
const filterState = {
  term: "camera",

  matchingTitles(records) {
    return records
      .filter((record) => {
        return record.title
          .toLowerCase()
          .includes(this.term);
      })
      .map((record) => record.title);
  }
};

const records = [
  { title: "Depth Camera Log" },
  { title: "Door Sensor Report" }
];

console.log(filterState.matchingTitles(records));
```

Console output

```
["Depth Camera Log"]
```

The method `matchingTitles` is a regular function. Its method call supplies `filterState` as `this`. The arrow-function callback does not create another `this`. It retains the method's surrounding value. Before arrow functions existed, this situation required workarounds such as `const self = this` or an explicit `bind` — both of which still appear in older codebases.

Using an arrow function as the method itself has different semantics:

An arrow function does not receive the object as this

```
const filterState = {
  term: "camera",

  matches: (record) => {
    return record.title
      .toLowerCase()
      .includes(this.term);
  }
};

filterState.matches({
  title: "Depth Camera Log"
});
```

Console output (in a module)

```
TypeError: Cannot read properties of undefined (reading 'term')
```

The expression still uses dot notation, but an arrow function ignores the call-site binding rule. Its `this` comes from the scope surrounding the object literal. Here, the top level of a module, where `this` is `undefined`, so reading `this.term` throws.

The exact failure depends on the surrounding scope, which makes this bug worse in older code. In a classic non-strict script, top-level `this` is the global object, so `this.term` is merely `undefined`. The call then evaluates `includes(undefined)`, which searches for the literal string `"undefined"` and quietly returns `false`. And nothing throws, the filter simply never matches. A silent wrong answer is harder to diagnose than the module's immediate `TypeError` — one more reason we use modules throughout.

Arrow functions are not shorter methods

Do not choose an arrow function only because it requires fewer characters. Use an arrow function when lexical `this` is intended — typically for callbacks written inside a method or other context whose `this` should be preserved. Use a regular method when the receiving object should determine `this`.

2.9 Chapter summary

What to remember about scope and functions

- Lexical scope follows where code is written, and the scope chain resolves names from inner to outer environments. Inner bindings can shadow outer ones.
- Hoisting is the visible result of the preparation phase from chapter 1. Bindings are registered when a scope is created, and `let` and `const` remain unusable until their declaration line.
- Closures retain access to outer bindings after the creating call has returned. The stack frame disappears, but the environment survives for as long as a function references it.
- Function declarations, expressions, methods, and arrow functions are not interchangeable. They differ in hoisting and in their treatment of `this`.
- For regular functions, `this` is determined by the call site. arrow functions have no own `this`, `arguments`, or `new` behaviour, and resolve `this` lexically.

Predict the output

Each program below is a complete module. Predict the console output, then verify your prediction and explain it using the vocabulary of this chapter.

Exercise 1: bindings, not values

```
let label = "draft";
const show = () => console.log(label);
label = "final";
show();
```

Exercise 2: ordering and the TDZ

```
function describe() {
  console.log(typeof mode);
  console.log(typeof theme);
  var mode = "dark";
  let theme = "solar";
}

describe();
```

Exercise 3: a detached method

```
const counter = {
  value: 41,
  next() {
    return this.value + 1;
  }
};

const step = counter.next;
console.log(counter.next());
console.log(step.call({ value: 99 }));
console.log(step());
```

Values, References, Mutation, and Prototypes

3

Many state-management defects begin with an inaccurate model of assignment. JavaScript always passes values, but an object value identifies an object. Assigning that value to another variable copies the reference, not the object. Both variables can therefore reach the same mutable data. Primitive values, by contrast, are copied directly.[1, Appendix A, pp. 110–112] Primitive values live in bindings, while objects are allocated on the heap, and a binding holds only the reference that identifies the heap object.

3.1 Primitive values and object references

Primitive assignments are independent

```
let currentView = "overview";
let rememberedView = currentView;
currentView = "details";

console.log(rememberedView);
```

Console output

```
overview
```

Object assignments can share one object

```
const source = [{ id: "B" }, { id: "A" }];
const visible = source;

visible.sort((a, b) => a.id.localeCompare(b.id));
console.log(source.map((item) => item.id));
```

Console output

```
["A", "B"]
```

The variables are distinct, but the array is shared. `sort` mutates its receiver, so the change is visible through every alias.

Equality follows the same model. For objects, `===` compares *identity* — whether two references reach the same heap object — not structural content:

Identity is not structural equality

```
const a = { id: "A-17" };
const b = { id: "A-17" };
const alias = a;

console.log(a === b);
console.log(a === alias);
```

Console output

```
false
true
```

Two objects with identical contents are still two objects. Conversely, two variables that compare equal with `===` are two names for one object, and mutation through either name is visible through both.

3.2 Copy before mutation, or prefer a non-mutating result

A shallow copy protects the top-level array structure:

Copy before an in-place operation

```
const source = [{ id: "B" }, { id: "A" }];
const sorted = [...source];

sorted.sort((a, b) => a.id.localeCompare(b.id));

console.log(source.map((item) => item.id));
console.log(sorted.map((item) => item.id));
```

Console output

```
["B", "A"]
["A", "B"]
```

Spread syntax

The notation `[...array]` is called *spread syntax*. It takes the elements of an existing array and places them into a new array.

The copy-then-sort pattern is common enough that the language added `toSorted` (ES2023), which directly communicates a non-mutating transformation:

A non-mutating transformation pipeline

```
const records = [
  { id: "B-04", status: "new" },
  { id: "A-17", status: "reviewed" }
];

const visibleRecords = records
  .filter((record) => record.status !== "archived")
  .toSorted((a, b) => a.id.localeCompare(b.id));

console.log(visibleRecords.map((record) => record.id));
console.log(records.map((record) => record.id));
```

Console output

```
["A-17", "B-04"]
["B-04", "A-17"]
```

`filter` already returns a new array, and `toSorted` sorts a copy, so the pipeline never touches `records`.

Trace identity, not only values

Two arrays may currently contain equal values while still being independent objects, and two different variables may identify the same array. When diagnosing mutation, inspect identity with `===` and follow where references are assigned.

3.3 Shallow and deep copies

Spread syntax creates a shallow copy. Nested objects remain shared:

A shallow copy does not clone nested objects

```
const original = {
  id: "A-17",
  metadata: { status: "new" }
};
const copy = { ...original };

copy.metadata.status = "reviewed";
console.log(original.metadata.status);
```

Console output

```
reviewed
```

To update one nested field without mutating the original object, copy every changed level:

Copy every modified level

```
const updated = {
  ...original,
  metadata: {
    ...original.metadata,
    status: "reviewed"
  }
};

console.log(original.metadata.status);
console.log(updated.metadata.status);
```

Console output

```
new
reviewed
```

Deep cloning is not one universal operation. Functions, prototypes, dates, maps, sets, cyclic graphs, and host objects require deliberate treatment. `structuredClone` handles many built-in data types, but targeted copies often communicate ownership more clearly.

3.4 Mutation as an ownership decision

Mutation is not inherently wrong. Local mutation of a newly created object can be clear and efficient:

Local mutation of a private accumulator

```
function indexById(items) {
  const index = new Map();
  for (const item of items) {
    index.set(item.id, item);
  }
  return index;
}
```

No external code can observe the intermediate state of `index`. Mutation becomes risky when aliases cross an ownership boundary and the set of affected readers is unclear. The most common such boundary is a function call, where an argument is a copied *reference*, so a callee can mutate the caller's object.

Mutation crosses the call boundary

```
function markReviewed(records) {
  for (const record of records) {
```

```
    record.status = "reviewed";
  }
}

const items = [{ id: "A-17", status: "new" }];
markReviewed(items);

console.log(items[0].status);
```

Console output

```
reviewed
```

Nothing in the call expression `markReviewed(items)` reveals that `items` will change. Whether such a function is a defect or a deliberate API is an ownership decision — and it should be a documented one.

3.5 Objects and the prototype chain

Property lookup first checks the object itself. If the property is absent, the engine follows the object's prototype link, then the next prototype, until it finds the property or reaches `null`. This chain implements delegation.[1, Chapter 3, pp. 92–95]

The prototype chain determines *where* a method is found. The call expression determines *which object becomes* `this`. As explained in section 2.7, these are separate mechanisms.

Explicit prototype delegation

```
const recordBehaviour = {
  displayLabel() {
    return `${this.id}: ${this.title}`;
  }
};

const record = Object.create(recordBehaviour);
record.id = "A-17";
record.title = "Depth Camera Log";

console.log(record.displayLabel());
```

Console output

```
A-17: Depth Camera Log
```

`displayLabel` is not an own property of `record`. Lookup delegates to `recordBehaviour`. The receiver of the call expression still determines `this`, so `this === record` while the method executes.

3.6 Classes are syntax over prototypes

JavaScript `class` syntax provides a familiar way to define constructors and methods, but method sharing still relies on prototype links.

Class syntax and shared methods

```
class Record {
  constructor(id, title) {
    this.id = id;
    this.title = title;
  }

```

```
  displayLabel() {
    return `${this.id}: ${this.title}`;
  }
}

const record = new Record("A-17", "Depth Camera Log");
console.log(record.displayLabel());
console.log(Object.hasOwn(record, "displayLabel"));
```

Console output

```
A-17: Depth Camera Log
false
```

The method lives on `Record.prototype` and instances delegate to it. Class methods use the method-shorthand semantics of section 2.6. `this` is supplied by the call site, so a detached class method loses its receiver exactly like the detached object method in section 2.7.

Prototype lookup is dynamic

Prototype delegation happens when a property is read, not when an instance is created. Replacing a method on the prototype affects existing instances unless an instance shadows that name with an own property.

The dynamic nature of lookup is directly observable:

Existing instances see a replaced prototype method

```
Record.prototype.displayLabel = function () {
  return `[${this.id}] ${this.title}`;
};

console.log(record.displayLabel());
```

Console output

```
[A-17] Depth Camera Log
```

The instance was created before the replacement, yet it observes the new method, because each call performs a fresh lookup along the chain.

3.7 Choosing objects, closures, or modules

We now have seen two encapsulation mechanisms. Closures (section 2.4) hide state in a retained lexical environment. Objects and prototypes group data and behaviour behind property access. In (chapter 5) we will see how Modules create file-level boundaries with explicit import and export declarations. These mechanisms overlap, but they are not interchangeable.

Use the mechanism that makes ownership and dependency direction easiest to see. A closure for state that belongs to one creation site, a class when many instances share behaviour, and a module when the boundary should be visible in the import graph.

Mechanism	Privacy boundary	Typical use
closure	the environment of one call; state is invisible outside	per-instance private state, callbacks, factories
object / class	convention and property access; shared behaviour via prototypes	many similar instances with common methods
module	file scope; only exports are reachable	application-wide services, singletons, composition

Table 3.1: Three encapsulation mechanisms

3.8 Chapter summary

What to remember about values and objects

- Primitive assignments copy primitive values, while object assignments copy references to shared heap objects. === compares object identity, not structure.
- Mutating methods affect every alias that reaches the same object or array — including aliases created by passing an argument.
- Spread syntax and common copy patterns are shallow. Nested objects may remain shared.
- Non-mutating transformations such as `toSorted` make ownership and previous state easier to reason about.
- Property lookup follows the prototype chain dynamically, and class syntax still relies on prototype delegation.

Predict the output

Each program below is a complete module. Predict the console output, then verify your prediction and explain it using the vocabulary of this chapter.

Exercise 1: aliases and reassignment

```
let a = { count: 1 };
const b = a;
a.count = 2;
a = { count: 3 };

console.log(b.count);
console.log(a === b);
```

Exercise 2: how shallow is this copy?

```
const original = { tags: ["new"], id: "A-17" };
const copy = { ...original };

copy.id = "B-04";
copy.tags.push("urgent");

console.log(original.id);
console.log(original.tags);
```

Exercise 3: own property shadows the prototype

```
const base = { label() { return "base"; } };
const item = Object.create(base);

```

```
4 console.log(item.label());
5 item.label = () => "own";
6 console.log(item.label());
7 delete item.label;
8 console.log(item.label());
```

The Browser Runtime, DOM Events, and Debugging

4

The browser constructs a DOM tree, applies CSS, executes scripts, dispatches events, performs network requests, and maintains storage. This chapter shows the *host* side of chapter 1. The DOM, timers, events, and storage are host capabilities layered around the engine, and they share one main thread with JavaScript. Effective debugging connects these layers instead of treating JavaScript as an isolated text file.

4.1 Page construction and script execution

As the HTML parser encounters elements, it constructs the DOM. A classic script without `defer` attribute can pause parsing while it is fetched and executed. The "run-to-completion" rule of chapter 1 applies to the parser's thread as well. A module script is deferred by default. Its dependency graph is loaded and it executes after the document has been parsed.[2, Chapter 2]

A module entry point

```
<script type="module" src="./src/main.js"></script>
```

A `DOMContentLoaded` listener is still useful when initialisation must explicitly wait for document construction, but module execution often removes the need for older placement conventions.

4.2 DOM queries and rendering

Imperative rendering typically queries a container, constructs nodes or markup, and then updates the document.

String-based rendering

```
function statCardHTML(value, label) {
  return `
    <div class="stat-card">
      <strong>${value}</strong>
      <span>${label}</span>
    </div>`;
}

const container = document.getElementById("statsPanel");
container.innerHTML = statCardHTML(18, "Records");
```

This is compact, but it treats a string as markup and can recreate an entire descendant tree. For user-controlled text, preserve text semantics:

Text should remain text

```
const preview = document.getElementById("notePreview");
preview.textContent = investigatorNote;
```

innerHTML creates a parsing boundary

Assigning `innerHTML` asks the browser to parse a string as markup. Any unescaped user-controlled fragment can become an element, an attribute, or an executable event handler. Prefer `textContent` for text

and structured DOM APIs when markup must be assembled.[19]

4.3 Event propagation and delegation

An event travels through capture, reaches its target, and usually bubbles through ancestors. Bubbling means that a listener on an ancestor observes events that originate on descendants:

An event bubbles from target to ancestor

```
// <section id="panel"><button id="save">Save</button></section>
const panel = document.getElementById("panel");
const save = document.getElementById("save");

panel.addEventListener("click", () => console.log("panel"));
save.addEventListener("click", () => console.log("save"));

save.click();
```

Console output

```
save
panel
```

Inside a listener, two properties answer different questions:

- `event.target` is the element on which the event originated, while
- `event.currentTarget` is the element whose listener is currently running.

In the panel listener above, `target` is the button and `currentTarget` is the section.

Event delegation exploits bubbling deliberately, because it attaches one listener to a stable ancestor and determines which descendant initiated the event.

Markup with data attributes

```
<ul id="recordList">
  <li><button data-record-id="A-17">Open A-17</button></li>
  <li><button data-record-id="B-04">Open B-04</button></li>
</ul>
```

Delegating clicks from dynamic descendants

```
const list = document.getElementById("recordList");

list.addEventListener("click", (event) => {
  const button = event.target.closest("[data-record-id]");
  if (!button || !list.contains(button)) return;

  console.log("open", button.dataset.recordId);
});
```

If the second button is clicked, the handler produces:

Console output

```
open B-04
```

`closest` climbs from the click target upward and is not limited to the listener's subtree, so a matching ancestor *above* the list would otherwise be accepted. The `contains` check restricts the match to descendants of the list.

Delegation is useful for lists whose children are replaced or inserted dynamically. The listener remains attached to the stable ancestor.

4.4 Browser storage and JSON boundaries

`localStorage` is a host API that stores strings. Structured values must be serialised and parsed, making storage a runtime validation boundary.

Serialising and reading a value safely

```
const key = "workspace-notes";
localStorage.setItem(key, JSON.stringify([{ id: "N1", text: "Check
     log" }]));

function readNotes() {
  const raw = localStorage.getItem(key);
  if (raw === null) return [];

  try {
    const value = JSON.parse(raw);
    return Array.isArray(value) ? value : [];
  } catch (error) {
    console.error("Invalid stored notes", error);
    return [];
  }
}
```

Corrupted storage is not a type-system problem. It is invalid input received at runtime. The JSON round trip is also lossy in a way that connects to chapter 3, because dates become strings, and maps, sets, functions, and prototype links do not survive serialisation. A storage module therefore owns not only the key names but also the reconstruction of proper values from parsed JSON.

4.5 The debugger as a model-inspection tool

Logging records selected values at selected moments. A debugger pauses execution and exposes the complete local state, lexical scopes, call stack, and next control-flow decision. Useful operations include the following:

Step over execute the current line without entering called functions.
Step into enter a called function and inspect its frame.
Step out finish the current function and pause in its caller.
Conditional breakpoint pause only when an expression is true.
Logpoint emit diagnostic data without changing the source file.
Watch expression repeatedly evaluate an expression while stepping.

Breakpoints are usually set in the Sources panel, but a `debugger;` statement in source code also pauses execution when developer tools are open. It is convenient during exploration and should be removed before code is shared.

A small program for stack inspection

```
function normaliseId(rawId) {
  return rawId.trim().toUpperCase();
}

function openRecord(rawId) {
  const id = normaliseId(rawId);
```

```
  console.log("opening", id);
}

openRecord(" a-17 ");
```

A breakpoint inside `normaliseId` shows `rawId` in the local scope and `openRecord` below it in the call stack. The paused state is a direct view of the execution contexts described in chapter 1. Step Out returns to the exact line that requested normalisation.

4.6 A practical DevTools map

Tool	Primary questions
Console	Which errors, warnings, and values were reported?
Sources / Debugger	Which code is active, which bindings exist, and who called this function?
Network	Which request ran, what status and payload returned, and how long did each phase take?
Application / Storage	Which persisted values, caches, and service-worker data exist?
Elements	Which nodes and styles exist in the rendered document?
Performance	Which tasks, layout work, and rendering operations consume time?

Table 4.1: Browser tools and the questions they answer

The Performance panel is where the "run-to-completion" rule of chapter 1 becomes visible. Each JavaScript job appears as one task on the main-thread track, and a long task is exactly the "frozen page" scenario from that chapter, now measurable.

Debug from symptom to state transition

A visible symptom is only the observation point. Reproduce it, identify the state transition immediately before it, pause where that transition occurs, and inspect both the current frame and its callers. This is more reliable than adding unrelated logs until the symptom disappears.

4.7 Chapter summary

What to remember about the browser and debugging

- The browser combines HTML parsing, DOM construction, CSS, JavaScript, networking, storage, and event dispatch on a shared main thread.
- DOM updates should preserve the distinction between text and trusted markup.
- Events propagate through capture and bubble phases. `event.target` identifies the origin, `event.currentTarget` the listening element, and delegation exploits bubbling deliberately.

- Storage boundaries require serialisation, runtime validation, and reconstruction of values that JSON cannot represent.
- Breakpoints, stepping, scopes, watches, and call stacks reveal execution state more precisely than scattered logging.
- Console, Network, Application, Elements, and Performance panels answer different diagnostic questions.

Predict the output

Assume the markup `<section id="panel"><button id="save">Save</button></section>` and predict the console output of each program.

Exercise 1: capture, target, bubble

```
const panel = document.getElementById("panel");
const save = document.getElementById("save");

panel.addEventListener("click", () => console.log("bubble"));
panel.addEventListener("click", () => console.log("capture"),
  { capture: true });
save.addEventListener("click", () => console.log("target"));

save.click();
```

Exercise 2: which element is which?

```
const panel = document.getElementById("panel");

panel.addEventListener("click", (event) => {
  console.log(event.target.id);
  console.log(event.currentTarget.id);
});

document.getElementById("save").click();
```

Exercise 3: a JSON round trip loses types

```
const stored = JSON.stringify({
  when: new Date(0),
  tags: new Set(["new"])
});

const parsed = JSON.parse(stored);
console.log(typeof parsed.when);
console.log(parsed.tags);
```

ES Modules and Maintainable Boundaries

5

A module is a unit of encapsulation and dependency. It defines which names belong together, which values form the public interface, and which implementation details remain private. Splitting a file by line count alone does not create a useful architecture. Boundaries must follow responsibilities and dependency direction.[3, Chapter 5]

5.1 Classic scripts and module scripts

A classic script shares the page's global environment with other classic scripts — the wide shared surface we criticised in chapter 2. A module has its own top-level scope, is always strict, supports static imports and exports, and is deferred by default.[20, 21]

Classic and module scripts

```html
<!-- Classic script: top-level var declarations can collide -->
<script src="legacy.js"></script>

<!-- Module entry point: dependencies are explicit -->
<script type="module" src="./src/main.js"></script>
```

Modules are fetched under browser security and MIME-type rules. Opening a project through `file://` commonly fails for modules and `fetch` requests. We need a local HTTP server as part of the execution environment (e.g. Live Server in VS Code).

5.2 Named and default exports

Named exports make the imported identifier explicit and allow several public values per module:

Named exports

```js
// src/utils/dates.js
export function formatDate(timestamp) {
  return new Date(timestamp).toLocaleDateString("en-GB");
}

export function compareByTimestamp(a, b) {
  return new Date(a.timestamp) - new Date(b.timestamp);
}
```

Importing named exports

```js
import { formatDate, compareByTimestamp } from "./utils/dates.js";
```

A namespace import, `import * as dates from "./utils/dates.js"`, collects all named exports as properties of one object and can improve readability when many related helpers are used together.

A default export represents one primary value and can be locally renamed:*

* The example uses `async` and `await`, which are introduced in chapter 6. Read them here as "perform the request and wait for its result".

A default-exported repository

```
const recordRepository = {
  async loadAll() {
    const response = await fetch("data/records.json");
    return response.json();
  }
};

export default recordRepository;
```

Neither form is universally superior. Named exports often make refactoring more consistent because the exported name travels with the symbol. A default export is reasonable when a module clearly exposes one primary object or component.

5.3 Module scope and live bindings

ES module imports are live, read-only views of exported bindings. An importer cannot assign to the imported name, but it observes reassignment performed by the exporting module. This parallels closures from section 2.4, where in both cases what is shared is a *binding*, not a copied value.

An import observes the exporter's reassignment

```
// src/state/counter.js
export let count = 0;

export function increment() {
  count += 1;
}
```

main.js

```
import { count, increment } from "./state/counter.js";

console.log(count);
increment();
console.log(count);
```

Console output

```
0
1
```

The importer did not re-read anything explicitly. The name `count` is a view of the exporting module's binding. The view is read-only — an assignment in the importer fails at runtime:

Imported bindings cannot be assigned

```
import { count } from "./state/counter.js";

count += 1;
```

Console output

```
TypeError: Assignment to constant variable.
```

Exporting a mutable binding directly is therefore possible, but encapsulating the state behind functions is usually the better interface:

Encapsulating mutable state behind functions

```
// src/state/records.js
let records = [];
```

```
3
4  export function getRecords() {
5    return records;
6  }
7
8  export function replaceRecords(nextRecords) {
9    records = [...nextRecords];
10 }
```

This is safer than exporting the binding directly because the module controls replacement and can later validate input or notify subscribers. It does not automatically make the returned array immutable. Recalling chapter 3, `getRecords` returns a reference, so the API may need to return a copy or expose narrower query functions.

5.4 Designing module boundaries

A browser application often contains the following responsibilities:

- data access and loading.
- application state.
- persistence.
- formatting and lookup helpers.
- navigation.
- view rendering.
- forms and event wiring.
- startup composition.

One defensible first decomposition has the following structure:

A responsibility-oriented source tree

```
src/
  main.js
  data/
    api.js
  state/
    records.js
    workspace.js
  navigation/
    router.js
  views/
    overview.js
    records.js
    timeline.js
  storage/
    storage.js
  utils/
    dates.js
    lookups.js
```

The exact names are not a universal architecture. The value lies in the reasons you need them for. The data module knows URLs, the storage module knows serialisation, views know DOM structure, and the entry point composes the pieces.

Export policy is architecture

An export grants other modules permission to depend on a value. Export only what a consumer genuinely needs. A private helper

can change freely and a public export becomes part of the module's contract.

5.5 Dependency direction

Lower-level modules should not import higher-level orchestration merely to trigger behaviour. For example, a state module should not import a view and render it directly. Instead, the entry point can compose both:

Composition at the application boundary

```
import { loadRecords } from "./data/api.js";
import { replaceRecords } from "./state/records.js";
import { renderOverview } from "./views/overview.js";

async function start() {
  const records = await loadRecords();
  replaceRecords(records);
  renderOverview();
}

start();
```

This keeps policy in the composition root and reduces circular dependencies. When a lower-level module must trigger higher-level behaviour — for example, state change should update a view — pass a callback or subscription function downward instead of importing upward. The dependency arrow then still points in one direction.

5.6 Circular dependencies

A circular dependency exists when module A imports B while B directly or indirectly imports A. ES modules define behaviour for cycles, but top-level initialisation can become difficult to reason about. The failure mode is the temporal dead zone already introduced in chapter 2.

a.js

```
import { valueB } from "./b.js";
export const valueA = "A sees " + valueB;
```

b.js

```
import { valueA } from "./a.js";
export const valueB = "B sees " + valueA;
```

main.js

```
import { valueA } from "./a.js";
console.log(valueA);
```

Console output

```
ReferenceError: Cannot access 'valueA' before initialization
```

Loading `a.js` first triggers `b.js`, whose import cycles back to the not-yet-initialised `a.js`. When `b.js` evaluates `valueA`, that binding exists but is still in its temporal dead zone. Common causes of cycles are state modules importing views, utilities importing feature modules, or feature modules importing the entry point. Break the cycle by moving composition outward or extracting a lower-level abstraction used by both sides.

5.7 Strict mode as a safety boundary

Modules always use strict mode. Assigning to an undeclared identifier throws instead of silently creating an accidental global:

A misspelling fails near its cause

```
function updateStatus() {
  currentStaus = "reviewed"; // misspelled, no declaration
}

updateStatus();
```

Console output

```
ReferenceError: currentStaus is not defined
```

The immediate failure is preferable to creating an unrelated global property whose existence is discovered much later.

5.8 Behaviour-preserving refactoring

When introducing module boundaries, separate structural change from feature change. Preserve externally observable behaviour, move a small responsibility at a time, and run the application after each step. If behaviour changes during a structural refactor, the cause should be narrow enough to locate through a small diff.

A module split is a dependency redesign

The goal is not the maximum number of files. A good split reduces the number of places that can directly change state, makes required dependencies visible, and gives each public interface a clear reason to exist.

5.9 Chapter summary

What to remember about modules and boundaries

- ES modules provide file-level scope, strict mode, and explicit static dependencies.
- Imports are live, read-only views of exported bindings — bindings are shared, not values copied.
- Export policy defines a module's public interface. Implementation details should remain private by default.
- Good boundaries follow responsibilities and dependency direction rather than arbitrary file size.
- A composition root wires state, views, and services without forcing lower-level modules to import application entry points. Cycles surface as temporal-dead-zone errors during initialisation.
- Behaviour-preserving refactoring separates structural movement from feature changes and keeps each diff reviewable.

Predict the output

Predict the console output of each program, then verify and explain.

Exercise 1: live bindings

```
// flag.js
export let enabled = false;
export function toggle() { enabled = !enabled; }

// main.js
import { enabled, toggle } from "./flag.js";
const copy = enabled;
toggle();
console.log(enabled, copy);
```

Exercise 2: a cycle that works

```
// a.js
import { fromB } from "./b.js";
export function fromA() { return "A"; }
console.log(fromB());

// b.js
import { fromA } from "./a.js";
export function fromB() { return "B and " + fromA(); }

// main.js
import "./a.js";
```

Explain why Exercise 2 succeeds although the cycle example in this chapter failed. The answer combines two ideas we showed earlier. Function declarations are registered during scope preparation (chapter 2), and imports are live views of bindings.

Promises, async/await, and Error Flow

6

Asynchronous code separates initiation from completion. A function starts an operation now, while success or failure becomes known later. The main design questions are therefore ordering, error propagation, ownership of intermediate state, and whether a late result is still relevant.

6.1 From callbacks to explicit future results

A callback can represent later work:

A timer callback represents later completion

```
console.log("before");
setTimeout(() => console.log("callback"), 0);
console.log("after");
```

Console output

```
before
after
callback
```

For one operation this is manageable. For several dependent operations, callbacks can distribute success and failure paths across nested functions. A Promise gives the future result an object with a defined state and a composable API.[2, Chapter 6]

6.2 Promise states and settlement

A Promise begins **pending** and settles exactly once as either **fulfilled** or **rejected**. Calling `then` registers reactions and returns a new Promise, which is why chains can transform results step by step. Throwing inside a reaction rejects the Promise returned by that `then`.

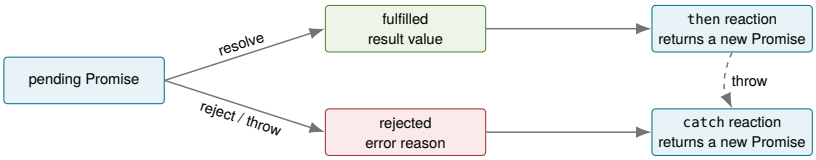


Figure 6.1: A Promise settles once. Reactions transform fulfilment or rejection and themselves produce new Promises, which enables chaining.

A Promise chain with explicit response validation

```
fetch("data/records.json")
  .then((response) => {
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }
    return response.json();
  })
  .then((records) => {
    console.log(`loaded ${records.length} records`);
  })
  .catch((error) => {
    console.error("loading failed", error.message);
  });
```

On success, a two-record response produces:

Console output

```
loaded 2 records
```

The first reaction returns the Promise produced by `response.json()`. The next reaction therefore waits for JSON parsing. If a nested Promise is not returned, the outer chain cannot wait for it.

An unhandled rejection is still a failure

Omitting the final `.catch()` does not make errors disappear. A rejected Promise with no attached rejection handler is reported by the runtime as an *unhandled rejection* that is visible in the browser console and, in Node.js, sometimes fatal to the process.

6.3 Promise reactions and the microtask queue

Promise reactions do not run synchronously in the same call stack that settles the Promise. They are queued as **microtasks**. After an ordinary task completes, the browser drains the microtask queue before moving to another ordinary task and before the next rendering opportunity.[2, Chapter 13]

This is the missing piece from chapter 1's runtime model. Function calls use the call stack, objects live on the heap, and two separate queues wait for later work. Timer and event callbacks wait in the **task queue**. Promise reactions wait in the **microtask queue**, which the event loop always drains completely before it looks at the task queue again.

Microtasks run before the next timer task

```
console.log("A");

setTimeout(() => console.log("timer"), 0);
Promise.resolve().then(() => console.log("promise"));

console.log("B");
```

Console output

```
A
B
promise
timer
```

The Promise is already fulfilled, but its reaction is still deferred. It runs after the current script and before the timer task — not because it was scheduled earlier, but because the microtask queue always empties before the next task is taken.

6.4 An async function always returns a Promise

A plain return value becomes a fulfilled Promise

```
async function loadLabel() {
  return "ready";
}

const result = loadLabel();
console.log(result instanceof Promise);
```

```
result.then((value) => console.log(value));
```

Console output

```
true
ready
```

Treating `result` as the string itself is a category error. The function call returns a Promise representing the future string.

6.5 async/await as Promise syntax

`await` pauses only the surrounding async function until the awaited Promise settles. It does not block the JavaScript thread or stop unrelated events. The continuation resumes through Promise scheduling.[22, 23]

Linear notation for a Promise-based operation

```
async function loadRecords() {
  const response = await fetch("data/records.json");
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  const records = await response.json();
  return records;
}

const records = await loadRecords();
console.log(records.length);
```

Assuming the same two-record response as the earlier chain, this prints 2. The listing uses a *top-level* `await` which is legal at the top level of an ES module, which every file we use is, but not inside a classic script or an ordinary function body outside a module.

The notation is sequential, but the browser remains free to handle other tasks while network I/O is pending.

async/await changes notation, not concurrency by itself

Replacing a correct Promise chain with `await` normally preserves its scheduling. Consecutive awaits start operations one after another. Concurrency changes only when independent operations are started before waiting for their combined result.

6.6 Sequential and concurrent operations

Sequential requests

```
const people = await fetchPeople();
const locations = await fetchLocations();
```

The second request begins only after the first completes. If the resources are independent, start both and await the pair:

Concurrent independent requests

```
const peoplePromise = fetchPeople();
const locationsPromise = fetchLocations();

const [people, locations] = await Promise.all([
```

```
  peoplePromise,
  locationsPromise
]);
```

`Promise.all` fulfils when all inputs fulfil and rejects when one rejects. It does not automatically cancel other operations that are already running. That way we can avoid unnecessary serial latency in independent asynchronous flows.[3, Chapter 6]

`Promise.all` is the right combinator only when every input must succeed for the result to be useful. Other combinators answer different questions:

Table 6.1: Promise combinators answer different questions

Combinator	Settles when
`Promise.all`	every input fulfils; rejects as soon as one input rejects
`Promise.allSettled`	every input has settled, regardless of outcome; never rejects
`Promise.race`	the first input settles, fulfilled or rejected
`Promise.any`	the first input fulfils; rejects only if every input rejects

`Promise.allSettled` is the common choice when partial success is useful, for example, loading several independent panels of a dashboard where one failing request should not blank the rest:

Reporting partial success

```
const results = await Promise.allSettled([
  fetchPeople(),
  fetchLocations()
]);

for (const result of results) {
  if (result.status === "rejected") {
    console.error("one panel failed to load", result.reason);
  }
}
```

6.7 Error handling with try/catch/finally

Structured success, error, and cleanup paths

```
async function refreshRecords() {
  setLoading(true);
  clearError();

  try {
    const records = await loadRecords();
    replaceRecords(records);
  } catch (error) {
    showError(error);
  } finally {
    setLoading(false);
  }
}
```

`finally` expresses cleanup that must happen for both fulfilment and rejection. Success-only updates belong in `try`. Failure representation

belongs in `catch`. If the cleanup itself throws, it can replace the original error, so cleanup should be small and reliable.

6.8 Do not disguise failure as empty data

A low-level loader can add technical context and rethrow:

Add context without swallowing failure

```
export async function fetchJson(url) {
  try {
    const response = await fetch(url);
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return await response.json();
  } catch (cause) {
    throw new Error(`Could not load ${url}`, { cause });
  }
}
```

Returning an empty array for every error makes network failure indistinguishable from a valid empty result. Loading, error, and empty states represent different facts and should remain distinct. Exporting only `fetchJson` — and keeping any retry or caching detail private to the module — is the same export-policy discipline from chapter 5, where callers depend on a narrow, honest contract, not on the loader's internals.

6.9 Cleanup, cancellation, and relevance

A Promise cannot generally be "uncalled", but many underlying APIs support cancellation. `fetch` accepts an `AbortSignal`:

Abort a request that is no longer relevant

```
const controller = new AbortController();

fetch(url, { signal: controller.signal })
  .then(handleResponse)
  .catch((error) => {
    if (error.name !== "AbortError") reportError(error);
  });

controller.abort();
```

The listing calls `controller.abort()` immediately only to show the API. In a real component it would run later, for example when the user navigates away, or when a newer request supersedes this one. Because `controller` is a normal object, whatever code holds onto it decides when to call `abort`. The closures from section 2.4 are the usual way that decision is kept close to the request that created it, rather than exposed globally.

Cancellation matters when a newer request supersedes an older one or when the owner of an operation is removed. Every asynchronous operation needs an owner that defines how success, failure, cleanup, and obsolescence are handled.

6.10 Chapter summary

What to remember about asynchronous control flow

- A Promise represents a future fulfilment or rejection and settles only once. An unhandled rejection is still a reported failure.
- Promise chains transform values and propagate failures through the Promise returned by each reaction.
- Promise reactions run as microtasks, which the event loop drains completely before the next timer or event task.
- `async/await` changes control-flow notation but does not make sequential operations parallel automatically. `Promise.all` and `Promise.allSettled` express different concurrency and failure policies.
- Error handling must preserve meaningful failures rather than silently converting them into empty data.
- Parallel work, cancellation, cleanup, and stale-result policies must be chosen explicitly according to ownership.

Predict the output

Each program below is a complete module. Predict the console output, then verify your prediction and explain it using the vocabulary of this chapter.

Exercise 1: ordering three kinds of work

```
console.log("1");

setTimeout(() => console.log("2"), 0);

Promise.resolve()
  .then(() => console.log("3"))
  .then(() => console.log("4"));

console.log("5");
```

Exercise 2: where does the throw land?

```
async function risky() {
  throw new Error("boom");
}

async function run() {
  try {
    await risky();
  } catch (error) {
    console.log("caught:", error.message);
  }
}

run();
console.log("after run()");
```

Exercise 3: all versus allSettled

```
function immediate(value) {
  return Promise.resolve(value);
}

```

```
function failing() {
  return Promise.reject(new Error("failed"));
}

Promise.allSettled([immediate("ok"), failing()]).then((results) => {
  console.log(results.map((r) => r.status));
});
```

Build, Dependency Management, TypeScript, and Delivery

7 The Build First Mindset

A web application can begin as a few files opened directly in a browser. As it grows, developers need reliable ways to start it, check it, prepare it for release, and publish it. A **build process** turns those recurring steps into documented commands that every developer and an automated system can repeat. Build First treats this automation as part of application design rather than packaging added shortly before release.[3, Chapters 1–4]

7.1 From files to a project

A directory becomes a maintainable project when it answers practical questions such as these:

- Which software and versions are required?
- Which libraries and development tools must be installed?
- Which command starts, checks, and builds the application?
- Which generated files should be deployed?

In the JavaScript ecosystem, Node.js runs development tools outside the browser. A **package manager** such as npm or pnpm records and installs external libraries and tools called **dependencies**. A **build tool** such as Vite serves source during development and creates production output. **Continuous Integration**, or CI, later runs the same project commands on a server. Chapter 8 introduces these concrete tools step by step.

7.2 Source, build, distribution, and deployment

These terms describe different stages:

Source Code, configuration, assets, and metadata maintained by developers.
Build process Checks and transformations applied to the source.
Distribution Generated output intended for a target environment, commonly placed in `dist/`.
Deployment Publishing that distribution so users can access it.
Environment configuration Values that vary by target, such as base URLs or feature flags.

Development favours fast feedback and readable diagnostics. Production favours compatibility, caching, and reduced transfer and processing cost. A successful build creates a distribution, but it does not necessarily deploy it.

A build should be repeatable

Given the same source, lockfile, tool versions, and configuration, a build should produce functionally equivalent output without undocumented manual steps.

7.3 A typical workflow

The development-to-deployment flow can be read as a sequence:

1. A developer edits source files.
2. A development server runs the application locally and reports errors.
3. Quality commands check formatting, code rules, types, and tests.
4. A production build creates the distribution.
5. A preview serves that distribution for inspection.
6. A deployment process publishes the verified distribution.

Keeping these stages distinct makes failures easier to locate. The development server is a tool, not the deployed application. Previewing development mode is also not the same as testing the production distribution.

7.4 Tasks and quality gates

A **task** is one repeatable unit of work, such as installing dependencies, checking types, running tests, or creating a production build. A **quality gate** is a required check that stops the workflow when it fails. A type error can therefore prevent a build, and a failed build can prevent deployment. Later chapters cover linting and formatting (chapter 9), TypeScript (chapter 10), and CI (chapter 11).

7.5 Project scripts provide one interface

JavaScript projects normally record commands in the `scripts` object of `package.json`. For now, read this file as the project's command menu. The next chapter explains where it comes from and how a package manager runs it.

A project-level command interface

```
{
  "scripts": {
    "dev": "vite",
    "typecheck": "tsc --noEmit",
    "lint": "eslint .",
    "build": "npm run typecheck && vite build",
    "preview": "vite preview"
  }
}
```

Contributors can remember `npm run build` instead of every tool flag. CI invokes the same script. The underlying implementation may change while the project-level command remains stable.[24]

7.6 Why automate early?

Manual procedures are difficult to review and easy to perform inconsistently. Automation moves knowledge from a person's memory into version-controlled files. Its setup cost occurs early, but its value grows with every build and contributor.[3, Chapter 1]

A missing automated step can be expensive

Bevacqua describes a trading firm whose deployment relied on manually copying code to several servers. One server was missed and retained obsolete behaviour, later contributing to a catastrophic loss.[3, Chapter 1] A student project has lower stakes, but the same failure mechanism applies. A required step that exists only in someone's memory will eventually be skipped.

7.7 Safe repetition and brownfield adoption

A task is **idempotent** when repeating it produces the same intended state. Builds should not depend on leftovers from an earlier run, and CI should install the dependency versions recorded in the lockfile.

An existing, or **brownfield**, project can adopt tooling incrementally:

1. Record how the current application runs and which behaviour must remain.
2. Add project metadata and choose a package manager.
3. Introduce a development server.
4. Add check-only quality commands.
5. Migrate to TypeScript in small groups.
6. Add CI and deployment after local commands are reliable.

7.8 Chapter summary

What to remember about Build First

- Node.js runs tools, a package manager installs dependencies and runs scripts, and a build tool creates development and production experiences.
- Source, build, distribution, and deployment name different stages.
- Quality gates turn expectations into enforced checks.
- Project scripts give developers and CI a stable command interface.
- Existing projects can adopt automation incrementally.

8 Dependencies, Package Managers, and Vite

A browser executes the application's frontend code, but development tools also need somewhere to run. In this course, that environment is Node.js. Installing Node.js normally also provides npm, a **package manager**. npm downloads the libraries and tools required by a project, records them in project files, and runs named project commands. pnpm is an alternative package manager with a mostly equivalent role.[24]

This chapter follows the setup in the order a new contributor encounters it. It begins with the package manager and `package.json`, continues through installation and the lockfile, and then introduces Vite's development and production modes.

8.1 Creating project metadata

Run package-manager commands from the project's root directory. With npm, an existing directory can be initialised as follows:

Create package metadata

```
npm init -y
```

This creates `package.json`. The file is a JSON document that describes the project. It is not application code and the browser does not load it. Developers and tools read it to discover project metadata, commands, and direct dependencies.[24]

A minimal package.json file

```
{
  "name": "case-atlas",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  },
  "devDependencies": {
    "typescript": "^5.9.0",
    "vite": "^7.0.0"
  }
}
```

The fields have distinct purposes. `name` and `version` identify the project. `private` prevents accidental publication to a package registry. `type: "module"` tells Node.js to interpret JavaScript files as ES modules. `scripts` names commands, and the dependency objects record packages required by the project. The versions shown here are illustrative.

8.2 Installing packages

The following command adds Vite as a development dependency:

Install a project-local development tool

```
npm install --save-dev vite
```

npm updates `package.json`, creates or updates `package-lock.json`, and places installed package files under `node_modules/`. Source code may import installed libraries, while scripts may run installed command-line tools. The `node_modules/` directory is generated and is normally excluded from Git. The manifest and lockfile are committed so another contributor can recreate it with `npm install`.[25]

Commit declarations, regenerate installed files

Commit `package.json` and the package-manager lockfile. Do not normally commit `node_modules/`. The package manager regenerates installed files from the committed declarations.

8.3 Running project scripts

Each property in `scripts` maps a short project command to a tool command. For example, `npm run dev` looks up `dev` and runs `vite`. Package managers temporarily make executables from local dependencies available to scripts, so contributors do not need a separate global Vite installation.

Run the commands declared by the project

```
npm run dev
npm run build
npm run preview
```

The package script is the stable interface. The command behind it remains visible in `package.json` and can change without requiring contributors or CI to learn a different workflow.

8.4 The dependency graph

A package named directly in `package.json` is a **direct dependency**. That package can depend on other packages, called **transitive dependencies**. Together they form a dependency graph. The package manager resolves and installs this graph, which may be much larger than the short list written by the application team.

8.5 dependencies and devDependencies

`dependencies` are required by the application at runtime or by consumers of a published package. `devDependencies` support development, checking, and building. For a statically deployed Vite application, Vite, TypeScript, ESLint, and Prettier are development dependencies because their code does not need to ship as runtime libraries. A frontend framework imported by the browser belongs in `dependencies`, even though it is bundled during the build.[24]

The distinction describes purpose, not importance. A dev dependency can be essential to release correctness.

8.6 The lockfile

The manifest usually permits version ranges. The lockfile records the exact resolved versions and integrity information for the full dependency tree. Committing it helps local machines and CI install the same graph.[25]

Use the lockfile command in CI

For npm, `npm ci` requires a consistent lockfile and installs from it without rewriting dependency choices. It is normally preferable to `npm install` in CI because unexpected manifest-lock drift becomes a failure rather than an implicit update.

8.7 Semantic version ranges

Semantic Versioning expresses versions as `major.minor.patch`. Conventionally, a major change may break compatibility, a minor change adds compatible functionality, and a patch fixes compatible defects. Range operators such as ^ or ~ define which future versions a package manager may select when the lockfile is updated.

A version range is not a guarantee of compatibility. Package authors can make mistakes, transitive dependencies can change behaviour, and tooling may introduce new checks. The lockfile controls when the project adopts a newly resolved graph. Review and CI decide whether that graph is acceptable.

8.8 Choosing npm or pnpm

npm and pnpm both manage manifests, scripts, dependency installation, and lockfiles. npm is normally installed together with Node.js and therefore has the lowest setup friction. pnpm uses a content-addressable store and links package files into projects, which reduces duplicated storage across projects.[26] Choose one package manager for a project, commit its lockfile, document its commands, and use the same choice in CI. Do not alternate between managers because their lockfiles and installation layouts differ.

8.9 What Vite does during development

Vite provides a development server with module resolution, transformations, and Hot Module Replacement. It serves application source through the browser's module system and transforms files on demand, while dependencies are prepared for efficient development.[27, 28]

A plain static server maps URLs to files. Vite additionally understands the module graph, resolves bare imports from installed packages, transforms TypeScript syntax, injects HMR support, and reports build-time diagnostics.

8.10 Hot Module Replacement (HMR)

HMR replaces an affected module or reloads an appropriate boundary without necessarily performing a full page reload. This shortens feedback loops and may preserve some runtime state. Preservation is not always desirable. Stale state can hide startup defects, so a full reload remains an important verification step.

HMR success is not production verification

Development mode and production mode use different performance and optimisation paths. A page that works through the dev server can still fail after bundling because of base paths, asset URLs, environment variables, or assumptions about module boundaries. Always inspect and preview the production build.

8.11 Production builds

Vite uses `index.html` as an entry point by default and produces optimised static assets suitable for static hosting. The build resolves the dependency graph, transforms modules, bundles code, minifies output, rewrites asset references, and commonly emits content-hashed filenames.[29]

A typical output tree

```
dist/
  index.html
  assets/
    index-C8x2mL9a.js
    index-Dq91nR4p.css
```

A content hash changes when file content changes. Servers can cache hashed assets for a long time because a changed asset receives a new URL. The HTML entry point typically receives shorter caching because it points to the current asset names.

8.12 Source maps and diagnostics

Minified stack traces refer to generated files. Source maps relate generated positions to original source. They improve debugging but may expose source structure or content, so teams decide deliberately whether and how production maps are published.

8.13 Base paths and static hosting

A project site on GitHub Pages is often hosted below a repository path rather than the domain root. Absolute asset paths such as `/assets/index.js` then point to the wrong location. Vite's `base` configuration controls generated URLs:

A repository-relative Vite base path

```
import { defineConfig } from "vite";

export default defineConfig({
  base: "/case-atlas/"
```

```
});
```

Local preview catches many but not all hosting differences. The deployed URL is part of release verification — chapter 11 shows a deployment workflow that builds and previews this exact project before publishing it.

8.14 Chapter summary

What to remember about dependencies and Vite

- The package manifest declares project metadata, scripts, and direct dependencies. The lockfile records the resolved graph.
- Runtime dependencies and development dependencies differ by purpose, not by importance.
- Deterministic installation requires a committed lockfile and project-local tools.
- Vite serves source modules efficiently during development and creates optimised, hashed production assets during builds.
- Production preview, source-map policy, and deployment base paths are part of validating the generated distribution.

9 Linting, Formatting, and Asset Processing

A build tool transforms code. Quality tools, by contrast, check properties of code. Their responsibilities overlap at the edges, so a project should define a clear pipeline rather than relying on editor defaults. This chapter supplies two of the quality gates named in chapter 7. Linting and formatting.

9.1 Linter versus formatter

A formatter decides how code is laid out. Whitespace, line wrapping, quote style, trailing commas, and indentation. A linter analyses code patterns that may be erroneous, inconsistent, or difficult to maintain. For example, unused variables, unreachable code, unsafe equality, floating Promises, or framework-specific rules.

Formatting and linting address different issues

```
// Formatter: spacing and line layout
const label={id:"E04",title:"Camera"};

// Linter: a likely defect
function findEvidence(id) {
  allEvidence.find((item) => item.id === id); // missing return
}
```

Prettier, a formatter library, would format both snippets but would not infer the missing return.[30] ESLint, a linting library, can detect many semantic patterns but should not be configured to fight a dedicated formatter over every whitespace decision.[31]

9.2 Check mode and fix mode

Local fix commands optimise developer flow and CI check commands preserve auditability.

Separate check and repair scripts

```
{
  "scripts": {
    "lint": "eslint .",
    "lint:fix": "eslint . --fix",
    "format": "prettier --write .",
    "format:check": "prettier --check ."
  }
}
```

CI should not silently rewrite source in a temporary runner and then report success. It should fail and require the corrected change to be committed — this is exactly the quality-gate behaviour chapter 7 described in the abstract, now given a concrete pair of commands.

9.3 Rules as executable team decisions

A linter configuration is a version-controlled coding policy. Avoid enabling a huge ruleset without understanding it. Rules should have a rationale, an acceptable false-positive rate, and a migration strategy for existing code.

Typical examples include the following:

- disallowing `var`.
- reporting unused variables and unreachable branches.
- requiring explicit handling of Promise-returning calls.
- preventing accidental reassignment.
- restricting direct use of `innerHTML`.
- enforcing consistent module imports.

The `innerHTML` rule enforces the parsing-boundary caution from chapter 4 automatically, on every pull request, instead of relying on every contributor remembering it.

Static rules cannot prove that a loading flag is reset or that two arrays are unintentionally aliased — the aliasing hazard from chapter 3 is invisible to a linter, since both the shared and the copied version are syntactically valid. Linting complements, rather than replaces, runtime testing and a correct mental model.

9.4 Preprocessing and postprocessing

Preprocessing converts source-oriented syntax into another form before final output. Examples include TypeScript to JavaScript, Sass to CSS, JSX transformation, or compile-time environment substitution. **Postprocessing** transforms generated or standard syntax, for example adding vendor prefixes, minifying CSS, or optimising assets.

The boundary is contextual and a tool may combine several phases. What matters is knowing which artefact each task consumes and produces.

9.5 Sass and CSS processing

Sass adds variables, nesting, mixins, and modular composition that compile to CSS. Modern CSS itself now supports many capabilities that historically required preprocessors, so Sass should be selected for actual project needs rather than habit.

A small Sass-style source fragment

```
$accent: #006c8e;

.evidence-card {
  border-left: 4px solid $accent;

  &__title {
    font-weight: 700;
  }
}
```

The deployed browser receives CSS, not Sass. Source maps can preserve a connection to the original source during debugging.

9.6 Minification

Minification removes unnecessary characters and may rename local bindings or simplify expressions. Its purpose is smaller transfer and parse cost, not secrecy.

Readable source and minified form

```
function formatLabel(evidence) {
  return `${evidence.id}: ${evidence.title}`;
}

// Conceptual minified form:
function formatLabel(e){return`${e.id}: ${e.title}`}
```

9.7 Obfuscation is not a security boundary

Obfuscation deliberately makes code harder to read, but the browser must still receive executable logic and required secrets cannot be safely embedded in frontend code. Obfuscation may slow casual inspection or reverse engineering. It does not enforce authorisation, protect API keys, or validate user actions. Security decisions belong on trusted servers and in protocol design. chapter 11 returns to this point for build-time secrets specifically.

Frontend code is observable

Anything delivered to a user's browser can be inspected, modified, and replayed. Build transformations may reduce size or readability, but they do not turn client code into a trusted execution boundary.

9.8 A coherent local pipeline

Composed project scripts

```
{
  "scripts": {
    "dev": "vite",
    "typecheck": "tsc --noEmit",
    "lint": "eslint .",
    "format:check": "prettier --check .",
    "check": "npm run typecheck && npm run lint && npm run format:
    check",
    "build": "npm run check && vite build",
    "preview": "vite preview"
  }
}
```

The order is intentional. Fast, diagnostic checks can fail before an unnecessary production build. A single `check` script makes the local and CI expectations identical — chapter 11 invokes this exact script from a workflow, so a failure locally and a failure in CI mean the same thing.

9.9 Chapter summary

What to remember about the quality pipeline

- Formatters standardise layout, while linters analyse code patterns and project rules.
- Check commands and fix commands serve different purposes and should be explicit project scripts.
- Preprocessing and minification transform source for compatibility, maintainability, or transfer efficiency.
- Obfuscation does not create a security boundary because browser-delivered code remains observable and modifiable.
- A composed `check` command keeps local development and CI expectations aligned.

TypeScript: Static Types for JavaScript

10

TypeScript is a programming language and toolchain built on JavaScript. Valid JavaScript is close to valid TypeScript, while TypeScript adds a static type system, editor information, and compile-time analysis. The resulting program still runs as JavaScript in a browser, Node.js, or another JavaScript runtime. TypeScript therefore does not replace JavaScript's runtime model. It adds a layer that can describe and check relationships before execution.[32]

The central goal is not to attach a type annotation to every variable. It is to make important assumptions explicit enough that tools can detect contradictions, support navigation and refactoring, and explain APIs while code is being written.

10.1 How TypeScript fits into a JavaScript toolchain

A TypeScript source file normally passes through two logically separate activities:

1. **type checking** analyses the program and reports inconsistencies.
2. **transformation** removes type-only syntax and produces JavaScript that a runtime can execute.

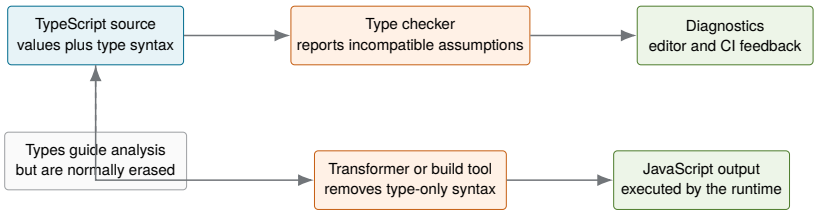


Figure 10.1: Type checking and JavaScript transformation are related but distinct. A project may use `tsc` for both, or combine a dedicated type-checking step with a separate build tool.

The TypeScript compiler, `tsc`, can perform both activities. In many frontend projects, a build tool transforms TypeScript quickly while `tsc -noEmit` runs as a dedicated type-checking step. This separation matters because successful transformation does not necessarily mean that the program is type-correct.

TypeScript does not execute in the browser

Browsers execute JavaScript. Type annotations, interfaces, and most other TypeScript-only constructs are removed before execution. Runtime behaviour still follows JavaScript semantics.

10.2 Type erasure and emitted JavaScript

Consider a small typed function:

TypeScript source with annotations

```ts
type TemperatureUnit = "celsius" | "fahrenheit";

function formatTemperature(
  value: number,
  unit: TemperatureUnit
): string {
  return `${value} deg ${unit}`;
}

console.log(formatTemperature(21, "celsius"));
```

A typical JavaScript output contains the executable statements but not the type declarations:

Conceptual emitted JavaScript

```ts
function formatTemperature(value, unit) {
  return `${value} deg ${unit}`;
}

console.log(formatTemperature(21, "celsius"));
```

Console output

```
21 deg celsius
```

The compiler can reject `formatTemperature("warm", "kelvin")` before the program runs, but the runtime does not know that `TemperatureUnit` ever existed.

10.3 Inference and explicit annotations

TypeScript often infers a useful type from an initial value or expression:

Inference avoids redundant annotations

```ts
const courseTitle = "Advanced Web Engineering"; // string
const sessionCount = 12;                        // number
const published = false;                        // boolean

const labels = ["runtime", "modules", "build"]; // string[]
```

Annotations are most valuable where they communicate an intended contract. For example, in function parameters, return values, public objects, reusable generic code, and boundaries to external data.

A function contract stated explicitly

```ts
function percentage(part: number, total: number): number {
  if (total === 0) return 0;
  return (part / total) * 100;
}
```

Local annotations that merely repeat the initializer often add noise:

An annotation that adds little information

```ts
const count: number = 3;
```

The preferred balance is to let inference describe obvious local values and annotate boundaries where an incorrect assumption would propagate.

10.4 Everyday value types

TypeScript includes types for JavaScript primitives and common compound values:

- Primitive types. `string`, `number`, `boolean`, `bigint`, `symbol`, `null`, and `undefined`.
- Arrays. `string[]` or `Array<string>` describe a sequence of strings.
- Tuples. `[number, number]` describes a fixed-position pair.
- Object types. Describe required, optional, and readonly properties.
- Function types. Describe parameters and return values.

Arrays, tuples, objects, and functions

```ts
const tags: string[] = ["browser", "state"];
const point: [number, number] = [12, 8];

type Article = {
  id: string;
  title: string;
  summary?: string;
  readonly publishedAt: string;
};

type Formatter = (article: Article) => string;
```

An optional property such as `summary?` may be absent, so reading it yields `string | undefined`. A `readonly` property cannot be reassigned through that type, but `readonly` is a compile-time restriction rather than a runtime freeze.

10.5 Literal types and unions

A literal type represents a particular value rather than every value of a primitive category. Unions combine alternatives:

A closed set of supported states

```ts
type RequestState = "idle" | "loading" | "success" | "error";

let state: RequestState = "idle";
state = "loading";

// state = "finished"; // error: not part of RequestState
```

Literal unions are especially useful for modes, status values, variants, and finite protocol states. They are more precise than a general `string` and usually integrate better with ordinary JavaScript values than a numeric enum.

Unions can also describe values with different shapes. A **discriminated union** gives each variant a common property whose literal value identifies that variant:

A discriminated union models mutually exclusive states

```ts
type LoadState<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; message: string };

function renderMessage(state: LoadState<string[]>): string {
```

```
  switch (state.status) {
    case "idle":
      return "Not requested";
    case "loading":
      return "Loading...";
    case "success":
      return `${state.data.length} items loaded`;
    case "error":
      return state.message;
  }
}
```

After checking `state.status`, the compiler knows which properties exist in that branch.

10.6 Control-flow analysis and narrowing

A union is useful only if code can establish which alternative it currently has. TypeScript narrows types through ordinary JavaScript checks such as `typeof`, equality, property tests, `instanceof`, and user-defined predicates.[33]

Narrowing a union with typeof

```
function normaliseId(id: string | number): string {
  if (typeof id === "number") {
    return id.toString();
  }

  return id.trim();
}
```

The branch does not cast the value. It proves information that the compiler can follow through the control flow.

10.6.1 User-defined type predicates

A reusable check can communicate its result through a predicate return type:

A predicate narrows unknown input

```
type UserProfile = {
  id: string;
  displayName: string;
};

function isUserProfile(value: unknown): value is UserProfile {
  if (typeof value !== "object" || value === null) {
    return false;
  }

  const candidate = value as Record<string, unknown>;
  return (
    typeof candidate.id === "string" &&
    typeof candidate.displayName === "string"
  );
}
```

The assertion inside the validator is local and followed by concrete runtime checks. Callers receive a safely narrowed value only when those checks succeed.

10.7 any, unknown, never, and void

Several special types express different levels of knowledge:

any Opts out of type checking for a value. Operations on it are accepted and unsoundness can spread.

unknown Represents a value whose type is not yet known. It must be narrowed before use.

never Represents a value that cannot occur, such as the result type of a function that always throws or an impossible union branch.

void Commonly describes a function whose return value is intentionally ignored.

unknown requires evidence before use

```ts
function printLength(value: unknown): void {
  if (typeof value === "string") {
    console.log(value.length);
  }
}
```

any is contagious

Once a value becomes `any`, later property access, calls, and assignments are largely unchecked. Prefer `unknown` at uncertain boundaries and narrow it deliberately. Use `any` only as a local, temporary escape hatch with a clear reason.[34]

10.8 Nullability and absence

With `strictNullChecks`, `null` and `undefined` are not silently accepted where another type is expected. This exposes many common failure paths:

A lookup may not find a result

```ts
type Course = {
  id: string;
  title: string;
};

function findCourse(
  courses: Course[],
  id: string
): Course | undefined {
  return courses.find((course) => course.id === id);
}

const selected = findCourse([], "awe");

if (selected) {
  console.log(selected.title);
}
```

Optional chaining and nullish coalescing express common fallback logic:

Handle optional values without hiding valid falsy values

```
const heading = selected?.title ?? "No course selected";
```

The non-null assertion `value!` tells the compiler to ignore possible absence. It does not add a runtime check. Use it only when an invariant has already been established elsewhere and is clearer than repeating the proof locally.

10.9 Object types, interfaces, and type aliases

Interfaces and type aliases can both describe object shapes:

Equivalent object contracts

```
interface User {
  id: string;
  name: string;
}

type UserRecord = {
  id: string;
  name: string;
};
```

Interfaces support declaration merging and an extension syntax. Type aliases can name unions, intersections, tuples, primitives, and computed type expressions. Neither is universally superior. A consistent local convention is usually more valuable than treating the choice as a design doctrine.[35]

10.9.1 Structural typing

TypeScript compares most object types by their structure rather than by an explicit nominal declaration:

Compatibility follows the available properties

```
type Named = {
  name: string;
};

const lecturer = {
  name: "Ada",
  department: "Computer Science"
};

function printName(value: Named): void {
  console.log(value.name);
}

printName(lecturer);
```

The object is accepted because it contains the required `name` property. It does not need to declare that it implements `Named`. Structural typing works naturally with JavaScript's object-oriented and compositional styles, but it also means that two conceptually different values may be compatible if their structures happen to match.

10.10 Function types and callbacks

Function types describe required arguments, optional parameters, rest parameters, and return values:

A callback contract

```ts
type Comparator<T> = (left: T, right: T) => number;

function sortCopy<T>(
  values: readonly T[],
  compare: Comparator<T>
): T[] {
  return [...values].sort(compare);
}
```

The `readonly T[]` parameter communicates that `sortCopy` will not mutate the caller's array through that reference. The returned `T[]` is a new mutable array.

Return-type annotations are useful on exported functions and complex branches because they make accidental changes visible. For short local callbacks, inference is often clearer.

10.11 Generics preserve relationships between types

A generic uses one or more type parameters to express a relationship that should remain consistent across different concrete types.[36]

A generic result preserves the item type

```ts
type Page<T> = {
  items: T[];
  page: number;
  total: number;
};

function firstItem<T>(page: Page<T>): T | undefined {
  return page.items[0];
}
```

Calling `firstItem` with `Page<User>` produces `User | undefined`. With `Page<string>`, it produces `string | undefined`. A version using `unknown` would lose that relationship, and a version using `any` would stop checking it.

10.11.1 Generic constraints

A constraint states what a generic value must support:

A generic constraint requires an id property

```ts
function indexById<T extends { id: string }>(
  values: readonly T[]
): Map<string, T> {
  return new Map(values.map((value) => [value.id, value]));
}
```

The function remains reusable, but callers must provide objects with a string `id`.

10.12 Creating types from other types

TypeScript can derive new types from existing declarations. This reduces duplication and keeps contracts aligned.

10.12.1 `keyof`, indexed access, and `typeof`

Derive keys and property types

```ts
type Settings = {
  theme: "light" | "dark";
  pageSize: number;
  compact: boolean;
};

type SettingName = keyof Settings;
type Theme = Settings["theme"];

const defaultSettings = {
  theme: "light",
  pageSize: 20,
  compact: false
} as const;

type DefaultSettings = typeof defaultSettings;
```

`keyof Settings` produces the union `"theme" | "pageSize" | "compact"`. Indexed access obtains the type of a selected property. In a type position, `typeof` derives a type from a value declaration.

10.12.2 Utility types

Built-in utility types perform common transformations.[37]

Common utility types

```ts
type User = {
  id: string;
  name: string;
  email: string;
  active: boolean;
};

type UserUpdate = Partial<Omit<User, "id">>;
type PublicUser = Pick<User, "id" | "name">;
type UserDirectory = Record<string, PublicUser>;
type ImmutableUser = Readonly<User>;
```

These types do not modify runtime objects. They describe alternative compile-time views of the same kinds of values.

10.13 Exhaustiveness checking

When every union member should be handled, `never` can make missing cases visible:

An exhaustive switch exposes new variants

```ts
type Theme = "light" | "dark" | "system";

function themeLabel(theme: Theme): string {
  switch (theme) {
    case "light":
      return "Light";
    case "dark":
      return "Dark";
    case "system":
      return "Use system setting";
    default: {
      const impossible: never = theme;
      return impossible;
    }
  }
}
```

If another `Theme` variant is introduced and not handled, the assignment to `never` fails during type checking.

10.14 Classes and access modifiers

TypeScript supports JavaScript classes and can check constructor parameters, implemented interfaces, inheritance, and member visibility:

A typed class with a private field

```ts
interface Clock {
  now(): Date;
}

class SystemClock implements Clock {
  now(): Date {
    return new Date();
  }
}

class SessionTimer {
  constructor(
    private readonly clock: Clock,
    public readonly startedAt = clock.now()
  ) {}

  elapsedMilliseconds(): number {
    return this.clock.now().getTime() - this.startedAt.getTime();
  }
}
```

The keywords `public`, `protected`, and TypeScript's `private` primarily affect static access checks. JavaScript's `##privateField` syntax provides runtime-enforced private fields. Composition through small interfaces often produces looser coupling than deep inheritance.

10.15 Modules and declaration files

TypeScript follows JavaScript's module syntax:

Types and values can be imported separately

```ts
import { createClient } from "./client";
import type { ClientOptions } from "./client";
```

A type-only import can be erased completely from emitted JavaScript. This makes it explicit that the import is used only by the type checker.

Type information for browser APIs, JavaScript runtimes, and libraries comes from **declaration files** ending in `.d.ts`. Declaration files describe available values and types without providing executable implementations. TypeScript ships declarations for standard language and DOM APIs. Libraries may bundle their own declarations, or consumers may install community-maintained packages under the `@types` scope.[38]

A declaration is a contract, not an implementation

A `.d.ts` file tells the type checker what a JavaScript API looks like. It does not create that API at runtime. The corresponding JavaScript library or platform feature must still be present.

10.16 Runtime boundaries still require validation

TypeScript checks code that participates in the typed program. Values arriving from HTTP, storage, form data, URL parameters, third-party scripts, or plain JavaScript remain runtime values whose shape can differ from expectations. chapter 4 treated `localStorage` as a runtime validation boundary for exactly this reason. The type checker does not change that, since it has no visibility into what was actually stored.

A type assertion does not validate JSON

```ts
type Preferences = {
  theme: "light" | "dark";
  pageSize: number;
};

const raw = JSON.parse(localStorage.getItem("preferences") ?? "
    null");
const preferences = raw as Preferences;
```

The assertion changes only the compiler's belief. It does not check the stored value. A safer boundary parses and validates before returning a trusted domain type:

Validate before exposing a trusted value

```ts
function parsePreferences(value: unknown): Preferences | null {
  if (typeof value !== "object" || value === null) return null;

  const candidate = value as Record<string, unknown>;
  const validTheme =
    candidate.theme === "light" || candidate.theme === "dark";

  if (!validTheme || typeof candidate.pageSize !== "number") {
    return null;
  }

  return {
    theme: candidate.theme,
```

```
    pageSize: candidate.pageSize
  };
}
```

For large schemas, runtime-validation libraries can generate detailed diagnostics and infer corresponding TypeScript types. The architectural rule is stable. Uncertain data is `unknown` at the boundary and becomes a domain value only after validation.

10.17 Compiler configuration

A `tsconfig.json` defines which files belong to the program and how they are analysed and emitted.[39]

A strict browser-oriented configuration

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noEmit": true,
    "isolatedModules": true
  },
  "include": ["src"]
}
```

Important options have different responsibilities:

`target` Controls the JavaScript syntax level emitted by `tsc` and influences default library declarations.

`lib` Selects standard API declarations available to the program, such as ECMAScript and DOM APIs.

`module` Controls the module syntax emitted by the compiler.

`moduleResolution` Selects how imports are matched to files and packages.

`strict` Enables a family of stronger checks, including nullability and implicit-`any` analysis.[40]

`noEmit` Uses the compiler only for analysis when another tool handles transformation and bundling.

`isolatedModules` Ensures that each file can be transformed independently by tools that do not perform whole-program analysis.

Configuration must match the actual runtime and build pipeline. Declaring an API in `lib` does not provide a polyfill, and choosing a modern `target` does not guarantee that every deployment environment supports that syntax.

10.18 What TypeScript can and cannot guarantee

TypeScript is effective at detecting mismatched contracts, missing properties, invalid union members, unsafe null access, incorrect callback signatures, and many inconsistent refactorings. It also improves editor

completion, navigation, and automated renaming because the tool has a model of relationships between declarations.

It does not prove that:

- an external payload is valid without runtime validation.
- a sorting algorithm is logically correct.
- an event listener is registered the intended number of times.
- a user interface is accessible or usable.
- a network request succeeds or arrives in the expected order.
- a value described as a date string contains a real date.
- a security-sensitive operation is authorised.

Types complement tests, runtime checks, code review, static analysis, and architectural constraints. They are strongest when they encode meaningful invariants rather than mirror every incidental implementation detail.

10.19 Gradual adoption

TypeScript can be introduced incrementally. JavaScript and TypeScript files may coexist, JavaScript can receive checking through `checkJs` and JSDoc, and strictness can increase over time. A practical sequence is to establish the toolchain, type stable module boundaries, model central domain values, and then reduce escape hatches as understanding improves.

Renaming files is not a type migration

A codebase full of `any`, broad assertions, and non-null assertions may compile as TypeScript while preserving most JavaScript uncertainty. Migration is complete only when important assumptions are expressed and checked rather than suppressed.

10.20 Chapter summary

What to remember about TypeScript

- TypeScript is a static type checker and transformation tool for JavaScript. Browsers still execute JavaScript.
- Inference handles many local values, while annotations are most useful at functions, public APIs, and external boundaries.
- Literal unions, narrowing, discriminated unions, generics, and derived types express relationships that plain JavaScript syntax cannot state explicitly.
- `unknown` preserves safety at uncertain boundaries, whereas `any` disables checking and can spread unsoundness.
- Interfaces and type aliases describe structural contracts. Declaration files describe the types of existing JavaScript APIs.
- Type information is normally erased, so network responses, storage, and other external values still require runtime validation.
- Strict compiler settings improve feedback, but they must be aligned with the actual runtime and build pipeline.

- TypeScript reduces classes of mistakes and supports refactoring. It does not replace testing, validation, security checks, or sound architecture.

11 Continuous Integration and Automated Deployment

Continuous Integration (CI) executes the project's verification process in a clean, shared environment when repository events occur. Continuous delivery or deployment extends the pipeline to produce and possibly publish a release artefact. The central idea is evidence for every accepted change should have a reproducible record showing which checks ran and whether they passed.

11.1 Workflows, jobs, and steps

A GitHub Actions **workflow** is a YAML-defined automation triggered by repository events. It contains one or more **jobs**. Each job runs on a runner and contains ordered **steps**. A step either executes shell commands or invokes a reusable action. Jobs can run in parallel unless dependencies are declared.[41]

A development verification workflow

```yaml
name: Verify

on:
  push:
  pull_request:

permissions:
  contents: read

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository
        uses: actions/checkout@v6

      - name: Set up Node.js
        uses: actions/setup-node@v6
        with:
          node-version: 22
          cache: npm

      - name: Install locked dependencies
        run: npm ci

      - name: Run project checks
        run: npm run check
```

Action versions in examples are illustrative. A real repository should verify supported versions and pin trusted actions deliberately.

11.2 Triggers

push Runs after commits are pushed to matching branches or paths.
pull_request Verifies proposed changes in the context of a pull request.
workflow_dispatch Enables a manual run from the UI or API.

A verification workflow commonly runs for pushes and pull requests. A deployment workflow commonly runs only for the protected main branch and may also allow manual dispatch for controlled recovery.

11.3 Dependency caching

Caching stores reusable package-manager download data between runs. It improves speed but must not be required for correctness. A cache miss should simply download dependencies again. A cache is not the same as the lockfile and should not replace `npm ci`.[42]

Lockfiles ensure correctness and caches improve speed

The lockfile specifies the dependency graph. The cache avoids downloading unchanged artefacts. Deleting the cache should make a run slower, not change which versions are installed or whether the build succeeds.

11.4 Fail fast and preserve the last good deployment

A deployment workflow should build a new artefact before touching the live site. If linting, type checking, formatting, or building fails, the previously published version remains available. This is preferable to partially updating a site or taking it offline because a new commit is invalid.

Build and deploy GitHub Pages as separate jobs

```yaml
name: Deploy Pages

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run build
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v4
        with:
          path: dist
```

```yaml

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    permissions:
      pages: write
      id-token: write
    steps:
      - name: Deploy
        id: deployment
        uses: actions/deploy-pages@v4
```

GitHub Pages custom workflows upload a built artefact and a later deployment job publishes it. The deployment job requires narrowly scoped Pages and identity-token permissions.[43]

11.5 Permissions and least privilege

The automatically provided `GITHUB_TOKEN` receives permissions configured at workflow or job level. Grant only what a job needs. Verification normally requires `contents: read`. Pages deployment requires `pages: write` and `id-token: write` in the deploy job. Over-granting increases the impact of a compromised third-party action or malicious workflow change.[44]

Third-party actions are dependencies

An action referenced with `owner/repository@tag` executes code in the runner. Review its publisher and permissions, prefer official or trusted actions, and consider pinning immutable commit hashes in higher-assurance projects.

11.6 Secrets and frontend builds

Repository secrets can be injected into workflow steps, but values embedded into a frontend bundle are no longer secret once deployed. Use secrets for deployment credentials or server-side operations, not for API keys that the browser must receive. Vite variables exposed to client code are public configuration.

11.7 Workflow logs as diagnostic artefacts

A failed run should identify the failing job, step, command, and error output. Good scripts preserve useful exit codes and avoid swallowing failures. When a type-check command fails, later build and deploy steps should not run.

```yaml
npm run typecheck   # non-zero exit status blocks the job on
    errors
```

Commands communicate success through exit status

```
npm run lint        # non-zero exit status blocks the job on violations
npm run build       # runs only if previous commands succeeded
```

11.8 Development and deployment workflows

Separating verification from deployment clarifies purpose:

- the development workflow gives rapid feedback for branches and pull requests.
- the deployment workflow proves that the exact main-branch source can be installed, checked, built, and published.
- both call shared `package.json` scripts, avoiding duplicated command logic.
- deployment permissions exist only in the deployment job.

The deploy workflow re-runs checks rather than trusting a developer's machine. It can reuse a verified artefact from another workflow, but that requires explicit artefact provenance and coordination. For a small course project, rebuilding in one self-contained deployment workflow is easier to audit.

A pipeline turns local convention into shared evidence

After a tooling migration, an application is no longer merely "a set of files that works on one machine". It has explicit module boundaries, a locked dependency graph, repeatable commands, static type checks, automated quality gates, a production distribution, and a deployment record. A CI/CD pipeline is what it looks like when that step no longer depends on anyone remembering it.

11.9 Chapter summary

What to remember about CI/CD

- Workflows contain jobs and ordered steps that run project commands in clean shared environments.
- Locked installation supports correctness, while dependency caches primarily improve speed.
- Verification should fail before build or deployment when types, linting, formatting, or compilation are invalid.
- Deployment jobs require narrowly scoped permissions, trusted actions, and explicit artefact publication.
- A failed release should leave the last known-good deployment intact rather than partially replacing it.
- CI/CD turns local conventions into auditable evidence about which source was checked, built, and released.

Evolution of Web Architectures

From Static Documents to Rich Web Applications

12

Web architectures evolve by moving responsibilities between the browser, the server, and the build process. Rendering, navigation, data access, and state management have each moved across that boundary several times. The result is not a sequence in which one architecture permanently replaces another. It is a growing set of strategies whose trade-offs suit different products, networks, devices, and development teams.

This chapter follows three early historical stages, namely static documents, dynamically generated server pages, and AJAX-enhanced interfaces. It asks *what problem did each architectural step solve, and which new responsibilities did it introduce?*[45]

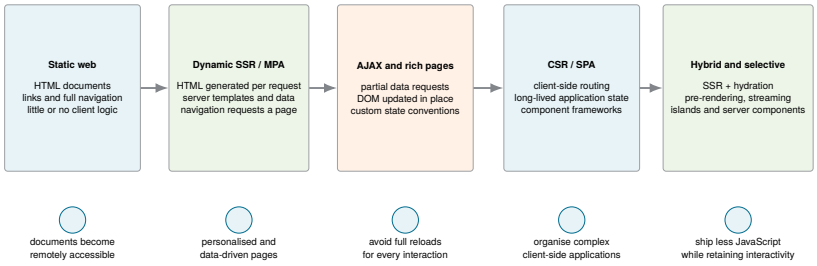


Figure 12.1: A simplified evolution of web-application architectures. Each stage addresses limitations of earlier approaches, but none makes the previous stages universally obsolete.

12.1 The static web: linked documents

The early web was primarily a collection of documents. An author prepared HTML files, placed them on a server, and connected them with hyperlinks. A request identified a resource, and the server returned its stored representation. The browser parsed the HTML, loaded referenced assets, and displayed the document. The original World-Wide Web system description is a useful primary account of this document-and-link model and of the web's early client–server architecture.[4]

A static document links to another document

```html
<nav>
  <a href="/index.html">Home</a>
  <a href="/articles.html">Articles</a>
</nav>

<main>
  <h1>Articles</h1>
  <p>This document is already complete on the server.</p>
</main>
```

When the user activates the second link, the browser requests another HTML file and replaces the current document. This model remains useful for documentation, small informational sites, downloadable resources, and content whose HTML changes only when a new version is published.

Static does not mean visually plain. CSS, images, media, and small scripts can all be used. The defining property is that the server does not generate a different HTML response for each request.

Static describes generation, not appearance

A static page can be visually rich and may contain client-side behaviour. It is static because its HTML representation was prepared before the request rather than generated from current request data.

12.2 The need for dynamic content

Static files become limiting when content depends on a database, an authenticated user, submitted form values, permissions, inventory, or current time. Early systems used technologies such as CGI to run a process for a request and return its output. Later server-side languages and frameworks made it practical to combine templates with application data on every request.

The server now performs application work before sending the response. The browser still receives ordinary HTML, but that HTML can vary between users and requests.

12.3 Dynamic server rendering

A dynamically rendered page is produced from a template and data:

Conceptual server template

```
<h1>Article {{ article.id }}</h1>
<p>{{ article.title }}</p>

{% if article.published %}
  <span class="status">Published</span>
{% endif %}
```

The syntax differs between PHP, JSP, ASP.NET, Rails, template engines, and other systems, but the request flow is similar:

1. the browser requests a URL.
2. the server executes application logic and reads required data.
3. a template combines structure and data into HTML.
4. the browser receives a complete document.
5. navigating to another page begins another request-response cycle.

This is **server-side rendering**. The server produces the HTML representation for the current request.

12.4 Multi-page applications

A traditional **multi-page application** associates navigation with document requests. Different URLs commonly correspond to different server routes and complete HTML responses. Activating a link or submitting a form usually causes the browser to leave the current document and load another one.

The server naturally participates in routing, authorisation, data access, and rendering. Temporary browser memory is usually discarded during navigation, while state that must survive is encoded in URLs, form submissions, cookies, sessions, or persistent storage.

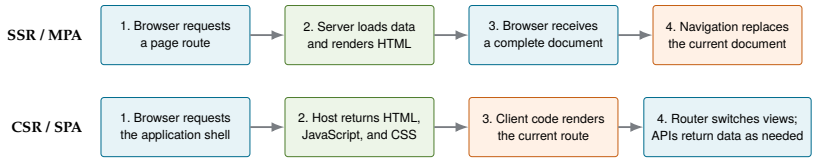


Figure 12.2: Typical navigation flows for a server-rendered MPA and a client-rendered SPA. In the first row, later navigation normally begins another document request. In the second, the document remains active while the client changes URL state and rendered components.

12.4.1 Strengths of the traditional model

A server-rendered page can contain meaningful text and links in its first HTML response. Browser navigation, forms, history, refresh, and deep links follow the platform's default behaviour. Application secrets and database access remain on the server. The client may require little JavaScript for its main content to function.

12.4.2 Costs of full-document navigation

Every navigation may require a server round trip and complete document replacement. The server performs rendering work for each request. Preserving temporary interface state across pages requires deliberate design. Highly interactive interfaces can feel cumbersome if every small change submits a form or reloads a document.

These limitations became more visible as web applications tried to resemble desktop applications. Maps should move continuously, search suggestions should appear while typing, messages should update in place, and forms should save without interrupting the current task.

12.5 AJAX and progressively richer pages

The next major shift did not immediately replace server rendering. Instead, JavaScript began requesting data in the background and updating part of an existing page. Garrett popularised the term **AJAX** for an interaction model based on asynchronous JavaScript, data exchange, and incremental page updates rather than a complete navigation for every action.[46] Later software-architecture research analysed AJAX applications explicitly in terms of client components, asynchronous communication, and server-push concerns, illustrating that richer pages had become an architectural problem rather than merely a scripting technique.[5]

A partial update after a data request

```
async function refreshStatus(articleId) {
  const response = await fetch(`/api/articles/${articleId}`);
  const article = await response.json();

  const status = document.querySelector("[data-article-status]");
  status.textContent = article.status;
}
```

The page may still have been generated on the server. JavaScript enhanced selected interactions, like autocomplete, map movement, inline editing, validation, background saves, and partial refreshes. Libraries such as jQuery reduced browser incompatibilities and simplified DOM selection, event handling, and network requests.

12.5.1 What AJAX improved

A small request could return only the data needed for one interface region. The browser no longer had to discard the entire document for every interaction. This reduced interruption and made richer, more immediate experiences possible.

12.5.2 What AJAX made harder

The browser now held increasingly important application logic and state. When updates were scattered across event handlers and selectors, it became difficult to answer:

- which data was authoritative.
- which DOM regions depended on that data.
- which request result was still relevant.
- which handler had to run after another.
- how navigation and browser history should represent the current view.

Rich pages created application-level client state

A page that performs one isolated request can remain simple. A page that coordinates many requests, editable forms, navigation states, cached data, and shared widgets has become an application. At that point, ad-hoc DOM updates need an explicit architecture.

12.6 Why client-side frameworks emerged

As frontend codebases and teams grew, developers adopted patterns such as MVC and MVVM to separate models, views, and update logic. Frameworks attempted to make a crucial relationship systematic. Application state changes, and the visible interface should follow that state consistently.

The shift toward single-page applications was therefore not caused by one browser API. It was an organisational response to increasingly long-lived, stateful client software. The following chapter examines how client-side rendering and browser-managed navigation changed the architecture more fundamentally.

12.7 Chapter summary

What to remember about the early evolution of web applications

- The static web served prepared, linked documents and relied on full-document navigation.
- Dynamic server rendering generated request-specific HTML from application logic, templates, and data.
- Traditional multi-page applications let server routes and browser navigation coordinate complete page transitions.
- AJAX enabled background data requests and partial DOM updates without replacing the whole document.

- Richer client interaction also created long-lived browser state, ordering problems, and growing coordination complexity.
- Client-side frameworks emerged to provide systematic rendering, state ownership, and application structure rather than scattered DOM manipulation.

Rendering and Navigation Architectures

13

Modern web architecture discussions often mix several independent decisions. A system may render HTML on the server or in the browser, replace complete documents or preserve one long-lived document, and store application state in the server, client memory, URL, or several coordinated locations. Understanding these axes separately makes architectural comparisons more precise.

13.1 Three decisions that are often confused

Rendering location Is HTML produced primarily on the server, in the browser, or at build time?

Navigation model Does navigation replace the complete document, or does one long-lived document switch views?

Application state Is state reconstructed on each request, held by a long-lived client application, encoded in the URL, or shared between client and server?

Server-side rendering is commonly associated with multi-page applications, and client-side rendering is commonly associated with single-page applications. However, these pairs are not definitions of one another. A multi-page site can contain client-rendered widgets, and an SPA can receive server-rendered HTML for its first request and then continue with client-side navigation.

SSR/CSR and MPA/SPA describe different axes

SSR and **CSR** describe where a view is rendered. **MPA** and **SPA** describe how navigation is organised. A system can combine them. For example, server-render the first view, activate it in the browser, and use SPA navigation afterwards.

13.2 Client-side rendering

With **client-side rendering**, JavaScript running in the browser creates or updates the page's DOM from application data. A fully client-rendered application commonly receives a small HTML shell and one or more JavaScript modules:

A minimal application shell

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Article Portal</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Before the first client-rendered view is available, the browser may need to:

1. download and parse the HTML shell.
2. discover and download JavaScript and CSS.
3. parse, compile, and execute the JavaScript.
4. initialise routing and application state.
5. obtain data that was not embedded in the response.
6. create and commit the initial DOM.

Once initialisation is complete, later updates can be fast because the application keeps running. Shared layouts remain mounted, cached data can be reused, and the server can exchange data rather than complete documents.

13.3 Single-page applications

A **single-page application** uses one long-lived document to host many views. The term does not mean that the application has only one screen or one URL. It means that navigation usually changes the client-side view without requesting a replacement HTML document. Research on migrating multi-page systems to single-page AJAX interfaces already treated this shift as an architectural transformation involving navigation, state, and client-side interface structure.[47]

A typical SPA combines:

- a client-side component or rendering system.
- a browser router.
- long-lived application state.
- asynchronous data requests.
- APIs that return data such as JSON.
- build tooling that produces the browser assets.

The server may still provide authentication, application data, business rules, persistence, and APIs. The architectural change is that the browser now coordinates much more of the visible application lifecycle.

13.4 SPA communication flow

A simplified client-rendered SPA proceeds as follows:

1. the browser requests the application entry URL.
2. the server returns an HTML shell and references to client assets.
3. the browser loads and executes the application.
4. the current URL selects a view.
5. the application requests required data.
6. state changes cause the relevant UI description to be recalculated.
7. later navigation changes the URL and view without replacing the document.

The server often returns data instead of complete HTML:

A data response used by a client-rendered view

```
{
  "id": "article-17",
  "title": "Rendering on the Web",
  "status": "published"
}
```

This frontend/backend separation can support several clients against one backend, but it also makes loading, error, stale-data, and compatibility states part of the client design.

13.5 Client-side routing

A router maps browser locations to views and keeps the address bar, history stack, and rendered application aligned. The browser location contains several distinct forms of information:

Path, query, and fragment carry different route information

```
const url = new URL("https://example.org/articles/17?tab=notes#
    latest");

console.log(url.pathname);                // /articles/17
console.log(url.searchParams.get("tab")); // notes
console.log(url.hash);                    // #latest
```

A client router typically handles:

- static and dynamic path matching.
- nested layouts and child routes.
- links that avoid full document navigation.
- Back and Forward integration.
- route parameters and query parameters.
- redirects and missing routes.
- coordination between URL state and rendered state.

13.6 Hash routing and the History API

Two common techniques allow client navigation without a full page replacement:

Hash routing Route state follows the \# fragment. The fragment is not sent to the server, so a static host can return the same document while client code interprets the hash.

History API routing The application uses normal-looking paths and calls browser history APIs to add or replace entries without an immediate document request.[48]

History-based routing produces cleaner URLs, but a direct request such as `/articles/17` still reaches the server first. A static host or web server must therefore return the application entry document for client-side routes that do not correspond to physical files.

A route is not complete until refresh and deep linking work

A route should be bookmarkable, shareable, navigable with Back and Forward, and valid when opened in a new tab. Updating only an internal variable creates view switching, not a complete routing model.

13.7 State in long-lived applications

A server-rendered document is often short-lived. Navigating away discards its DOM and most in-memory JavaScript state. An SPA may remain active for hours, so it must manage several categories of state deliberately:

Local UI state Whether a menu is open, which tab is selected, or the current text in a draft field.
Shared application state Data needed by several distant views, such as the signed-in user, permissions, or a selection.
Server state Remote data cached in the client, which can be loading, stale, invalidated, or updated by another actor.
URL state The current route, route parameters, query filters, sorting, and pagination information that should survive sharing and navigation.
Persisted client state Values stored in mechanisms such as `localStorage` or IndexedDB and restored after a reload.

The important design question is ownership. If the same fact is stored independently in several places, those copies can disagree. If every value is made global, unrelated changes can affect large parts of the interface.

Each state value needs a clear owner

State does not become simpler merely by moving it into the browser. A durable SPA design distinguishes local, shared, server, URL, and persisted state, and defines how each value is updated, reset, shared, and restored.

13.8 Unidirectional data flow

Modern client frameworks often organise updates as a one-way cycle:

1. state describes the current application facts.
2. rendering derives the visible UI from that state.
3. user or network events request state transitions.
4. updated state causes a new render.

This replaces a web of imperative instructions such as “change this class, then update that count, unless another request completed”. It does not eliminate complexity, but it provides one direction in which to reason about change. More broadly, model-driven work on rich internet applications treated presentation, navigation, data, and client–server distribution as explicit architectural concerns rather than isolated implementation details.[6]

13.9 MPA and SPA compared

Neither model is universally superior. Their trade-offs appear in different places:

Table 13.1: Typical differences between traditional MPAs and client-rendered SPAs

Concern	Traditional MPA	Client-rendered SPA
Initial HTML	commonly complete for the requested route	may be an application shell before JavaScript runs
Navigation	requests and replaces documents	changes client route and component tree
Temporary UI state	commonly reset on navigation	can remain in browser memory
Routing responsibility	primarily server and browser defaults	client router plus server fallback
Data exchange	often HTML responses and form submissions	commonly API requests and structured data
Failure handling	server errors commonly replace the page	loading, retry, stale, and partial error states live in the UI
Client complexity	can remain small	often substantial and long-lived

13.10 Benefits and costs of SPAs

A single-page architecture can support responsive transitions, reusable client-side state, offline capabilities, and a clean separation between a data API and multiple frontends. It also introduces significant responsibilities.

The simplified claim that SSR is always better for search engines and CSR is always better for interaction is misleading. Server-rendered pages can contain highly interactive client components, while client-rendered applications can prefetch, cache, and produce meaningful content through other rendering strategies. The correct choice depends on content, latency budgets, devices, personalisation, deployment constraints, and how much interaction occurs after the first view.

Potential benefit	Corresponding cost or risk
Fast transitions after start-up	More code and data may be required before first interaction
Long-lived client state	Ownership, synchronisation, and reset rules become complex
API-oriented backend	More network failure and loading states must be represented in the UI
Rich interaction without document reloads	Routing, focus, history, and browser conventions require explicit handling
Reusable client cache	Cached data can become stale or race with newer requests
Independent frontend deployment	Client and server contracts must evolve compatibly

Table 13.2: Typical SPA trade-offs

13.11 Chapter summary

What to remember about rendering and navigation architectures

- SSR and CSR describe where HTML is rendered. MPA and SPA describe how navigation is organised.
- Client-side rendering moves view construction into the browser and usually requires JavaScript before the initial UI is complete.
- An SPA keeps one document alive while client-side routing maps URLs to changing views.
- History-based routes require a server fallback so deep links and refreshes still deliver the application entry document.
- Long-lived clients must distinguish local, shared, server, URL, and persisted state and give each value a clear owner.
- SPAs can provide rich, fast interactions after start-up, but they increase client complexity, loading states, routing responsibility, and synchronisation concerns.
- Architectural choices should be evaluated per product rather than treated as a simple progression from MPA to SPA.

Hybrid Rendering and Modern Web Architectures

14

Pure server rendering and pure client rendering expose different weaknesses. A server-rendered document can become visible quickly but may replace the whole page on navigation. A client-rendered application can offer rich continuity after start-up but may require substantial JavaScript before useful content and interaction are available. Modern architectures combine build-time, server-time, and client-time work rather than committing every route to one extreme.

14.1 Why hybrid rendering emerged

On slow devices or networks, a blank application shell followed by a large JavaScript download produces a poor first experience. Public content also benefits from useful HTML being available before client execution. At the same time, products still require forms, navigation, personalisation, and responsive interactions. Empirical page-load research has shown why this trade-off is not reducible to transfer size alone. Dependencies between networking, parsing, script execution, and rendering determine the critical path to usable content.[9]

Hybrid frameworks therefore ask several more precise questions:

- Can the first route be rendered before it reaches the browser?
- Which parts actually require JavaScript?
- Can HTML be sent before all server work is complete?
- Can routes be generated ahead of time?
- Can interactivity be activated selectively or on demand?

14.2 Server rendering followed by hydration

A common hybrid strategy renders HTML on the server for the initial request and then activates a client application over that existing markup. In React, `hydrateRoot` attaches React behaviour to HTML previously produced by a compatible server renderer.[49]

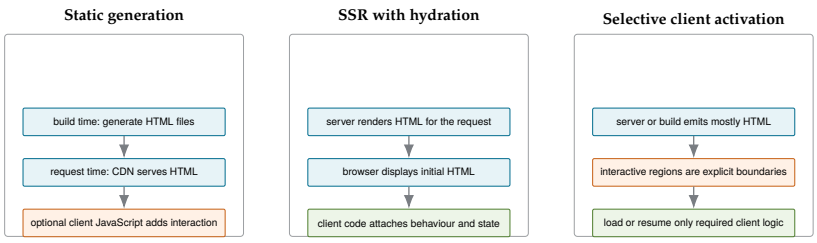


Figure 14.1: Three hybrid strategies. Static generation moves rendering to build time, hydration attaches client behaviour to server-rendered HTML, and selective approaches activate only designated regions.

A simplified sequence proceeds as follows:

1. the server renders HTML for the requested route.
2. the browser can parse and display that HTML.
3. JavaScript downloads and executes.

4. the framework reconstructs the component model for the existing markup.
5. event handlers and client state become active.
6. later navigation may continue through client-side rendering.

The visible HTML may therefore arrive before the page is fully interactive. This interval must be considered in the design. Controls that look active but cannot yet respond can confuse users.

Hydration activates existing HTML

Hydration does not mean rendering an empty page from scratch. It connects client-side component behaviour to compatible HTML that was produced before the client application started.

14.3 Hydration costs and mismatches

Hydration improves the first visible result but does not make client work disappear. The browser may still download, parse, and execute much of the application code. The framework may reconstruct a large component tree even when only a small region is interactive.

The server and client must also agree on the initial output. Differences caused by time, random values, browser-only APIs, locale, or inconsistent data can create hydration mismatches. Correct hybrid rendering therefore requires deterministic initial rendering or carefully isolated client-only regions.

14.4 Pre-rendering and static site generation

Static site generation produces route HTML during the build rather than for each request. The generated files can be served from a static host or CDN without executing application rendering logic on every request.

This is attractive for documentation, course material, public catalogues, marketing pages, and other content that changes less frequently than it is requested. Client JavaScript can still enhance selected regions, so static generation is not equivalent to "no JavaScript". Related systems research has explored separating reusable static page structure from dynamic content as a way to reduce mobile page-load work, reinforcing the broader architectural value of moving stable rendering work out of the critical request path.[50]

Strategy	HTML generation time	First response	Typical update mechanism
CSR	in the browser	shell plus client assets	deploy client code or change API data
Dynamic SSR	on each request	request-specific HTML	next request reflects current server data
SSG / pre-rendering	during build	stored HTML file	rebuild or selectively regenerate
SSR + hydration	on request, then activated on client	HTML plus client assets	server response and later client updates

Table 14.1: When HTML is generated in common rendering strategies

React exposes static rendering APIs, while application frameworks commonly integrate pre-rendering with routing, data loading, caching, and deployment.[51]

14.5 Regeneration and caching

Build-time rendering becomes challenging when content changes frequently or the number of possible routes is very large. Frameworks therefore combine static generation with cache invalidation or selective regeneration. A route may be pre-built, regenerated after a time interval, or recomputed when its data changes.

This creates another architectural responsibility, since cached HTML needs a freshness policy. The system must define when content can be stale, what invalidates it, and whether users can tolerate different versions at different edge locations.

14.6 Streaming server rendering

Traditional SSR may wait until the complete page is ready before sending a response. **Streaming SSR** sends HTML in chunks as portions become available. A shared shell or fast content can arrive first, while slower regions follow later.

Streaming can improve perceived progress and reduce the time before meaningful content appears. It also requires explicit boundaries for loading, errors, and content ordering. A slow child should not unnecessarily block unrelated page regions.

14.7 Selective and incremental hydration

Hydrating an entire application eagerly can perform work for components that the user never interacts with. Selective approaches activate only specific regions, or delay activation until a region becomes visible, idle time is available, or the user interacts.

The central performance question changes from "how quickly can the whole application hydrate?" to "which capabilities must become interactive now?" This can reduce initial JavaScript work but introduces coordination between server-rendered content and partially active client regions.

14.8 Islands architecture

An **islands architecture** treats most of a page as static or server-rendered HTML and embeds isolated interactive components where needed. Astro describes these interactive components as client islands and only ships client JavaScript for components explicitly activated for the browser.[52]

A content page might contain server-rendered navigation and article text, while a search widget, media player, or configurator becomes an interactive island. This avoids turning the complete document into one long-lived client application when only a few areas need state.

14.9 Server Components

Server Components allow some component logic and data access to remain on the server and avoid contributing corresponding implementation code to the client JavaScript bundle. Client Components are used where browser state, event handling, or browser APIs are required. Current Next.js documentation combines both kinds within a route and can stream server-rendered results.[53]

The boundary is not simply "backend versus frontend file". It determines where code executes, which data can be accessed directly, what must be serialised, and which components can contain client interactivity.

14.10 Resumability

Hydration commonly reconstructs client execution state from server-rendered HTML and application code. A resumable architecture instead serialises enough information for the browser to continue from the server-produced state and loads behaviour when required. Qwik describes this approach as **resumability**.[54]

Resumability aims to avoid eagerly re-executing the whole component tree on start-up. It changes the framework and serialization model substantially, so it should not be treated as a small hydration optimisation.

14.11 Edge-aware rendering

Content delivery networks increasingly execute caching and application logic closer to users. Edge rendering can reduce network distance for dynamic or personalised responses, while regional caches can serve pre-rendered content quickly.

Moving work to the edge introduces constraints as well. Execution environments may differ from a full server runtime, globally consistent data may still be distant, and cache invalidation becomes part of application correctness.

14.12 A shared direction: less unnecessary client work

Streaming, islands, selective hydration, Server Components, resumability, and edge rendering are different techniques. Their shared direction is more important than any individual framework name:

- send useful HTML early.
- ship less JavaScript when a route needs less interaction.
- avoid blocking unrelated page regions.
- perform work in the cheapest appropriate environment.
- preserve browser navigation and document semantics where possible.

14.13 Architecture as a product decision

The historical timeline should not be read as a maturity scale. A content site may be best served as static HTML. A transaction-heavy administrative application may benefit from an SPA. A public product page may use server rendering and selective interaction. A complex platform may use several strategies on different routes.

A useful architecture decision records:

- how quickly meaningful content and interaction must appear.
- how much interface state persists across navigation.
- whether content is public, personalised, or frequently changing.
- whether offline operation or long-lived sessions matter.
- which devices and network conditions must be supported.
- how routing, caching, deployment, and failure handling will work.
- what operational and conceptual complexity the team can maintain.

Prefer explicit trade-offs over architecture labels

"SPA", "SSR", "static", and "hybrid" are starting points, not complete designs. A useful decision explains which work occurs at build time, request time, and interaction time, and why that distribution fits the product.

14.14 Chapter summary

What to remember about hybrid web architectures

- Hybrid rendering distributes work across build time, request time, and browser interaction time.
- Hydration activates compatible server-rendered HTML, but still incurs client download, execution, and reconstruction costs.
- Static site generation creates HTML during the build and trades request-time work for rebuild and freshness policies.
- Streaming sends ready page regions earlier, while selective hydration activates only the client regions that need behaviour.
- Islands, Server Components, and resumability use different mechanisms to reduce unnecessary client-side JavaScript and start-up work.
- Edge execution and caching can reduce network distance but introduce runtime and consistency constraints.
- Architecture remains a product decision. The best design matches content, interaction, performance, deployment, and team constraints.

React

15 React Foundations

React is a JavaScript library for describing user interfaces as compositions of components. It does not, by itself, define a complete application architecture. Routing, data access, persistence, deployment, and many forms of state management are separate decisions. Its core contribution is a rendering model in which the UI is calculated from data, and changes are requested by updating state rather than by issuing a sequence of DOM commands.[55]

This chapter introduces the foundation of that model, the declarative UI, JSX, function components, props, composition, the component tree, and the render–commit cycle. Interactivity and shared state are developed in later chapters so that the static rendering model is clear first.

15.1 From imperative DOM updates to declarative UI

An imperative interface describes the operations required to move the current DOM into another state:

Imperative DOM update

```
const status = document.querySelector("[data-status]");
const button = document.querySelector("[data-review-button]");

status.textContent = "Reviewed";
status.classList.add("status--complete");
button.disabled = true;
```

The code must know what the DOM currently contains and which operations are required. As the number of states grows, branches can leave the DOM only partially updated.

A declarative component describes the desired result for the current data:

Declarative React component

```
type ReviewStatusProps = {
  reviewed: boolean;
};

function ReviewStatus({ reviewed }: ReviewStatusProps) {
  return (
    <>
      <span className={reviewed ? "status status--complete" : "
    status"}>
        {reviewed ? "Reviewed" : "Pending"}
      </span>
      <button disabled={reviewed}>Mark reviewed</button>
    </>
  );
}
```

The component does not issue commands such as “add this class” or “disable that element”. It calculates one description for `reviewed === true` and another for `reviewed === false`. React is responsible for applying the necessary host-DOM changes.

State determines UI

React code should answer “what should the interface look like for these inputs?” Event handlers request state changes and rendering translates the resulting props and state into JSX.

15.2 Adding React to a Vite and TypeScript project

A browser application normally uses two runtime packages:

- `react` provides components, Hooks, Context, and the core rendering model.
- `react-dom` connects React to the browser DOM.

A Vite project also uses its React plugin and TypeScript declarations during development and building. The exact versions should be selected and locked by the project rather than copied from a manuscript.

Typical React dependencies in a Vite project

```
npm install react react-dom
npm install --save-dev @vitejs/plugin-react @types/react @types/
    react-dom
```

Vite configuration with the React plugin

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()]
});
```

The browser entry document contains a mount point:

React mount point

```
<div id="root"></div>
<script type="module" src="/src/main.tsx"></script>
```

The TypeScript entry module creates a React root and renders the top-level component. The explicit null check prevents a missing mount point from being hidden by a non-null assertion.

A typed React entry point

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { App } from "./App";

const container = document.getElementById("root");

if (!container) {
  throw new Error("Missing #root mount point");
}

createRoot(container).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

`createRoot` establishes the browser DOM node that React owns. Calling `render` begins the component tree. On the initial client render, React replaces existing descendants of that root. Server-rendered HTML uses hydration instead.[56]

15.3 JSX is JavaScript syntax for UI descriptions

JSX resembles HTML but is not an HTML string and is not interpreted directly by the browser. A build transform converts JSX into JavaScript expressions that create React elements, which are immutable descriptions of the desired UI.[55]

JSX combines markup-like structure and expressions

```
const record = {
  id: "A-17",
  title: "Depth Camera Log",
  reviewed: true
};

const heading = (
  <h2 className={record.reviewed ? "reviewed" : "pending"}>
    {record.id}: {record.title}
  </h2>
);
```

Braces switch from JSX structure to a JavaScript expression. An expression can read a variable, access a property, call a pure helper, construct an array, or calculate a value. Statements such as `if`, `for`, and `switch` cannot appear directly inside braces because they do not produce a value. They belong before the returned JSX or in a helper.

Some attributes differ from HTML because JSX maps to JavaScript properties. For example, CSS classes use `className`. Event handlers receive functions rather than strings. Style objects use JavaScript property names.

JSX attributes are JavaScript values

```
<button
  className="action-button"
  disabled={!record.reviewed}
  aria-label={`Open ${record.title}`}
>
  Open record
</button>
```

JSX is not string concatenation

A JSX element is a value in the React rendering model, not a prebuilt DOM node and not an HTML string. React decides how that description is committed to the target platform.

15.4 Function components and purity

A function component is a JavaScript function whose name begins with a capital letter and whose return value describes UI. React calls components while rendering. Given the same props, state, and context, a component

should calculate the same JSX and should not mutate existing data or perform unrelated external work.[57]

A small pure component

```tsx
type StatusBadgeProps = {
  label: string;
  tone: "neutral" | "positive" | "warning";
};

export function StatusBadge({ label, tone }: StatusBadgeProps) {
  return <span className={`badge badge--${tone}`}>{label}</span>;
}
```

The component reads its inputs and returns a result. It does not change the prop object, increment a global counter, write to storage, or query the DOM during render.

A render-time side effect creates unstable behaviour

```tsx
let renderCount = 0;

function RecordTitle({ title }: { title: string }) {
  renderCount += 1; // changes external state during rendering
  return <h2>{title}</h2>;
}
```

React may render more than once, restart work, or perform development checks. Code whose correctness depends on an exact number or order of render calls is therefore unsafe. Event handlers are the ordinary place for changes caused by user actions.

15.5 Props, typed contracts, and children

Props are read-only inputs supplied by a parent. TypeScript can describe the component's public contract and catch missing, misspelled, or incompatible values at the call site.[58, 59]

Typed named props

```tsx
type RecordCardProps = {
  id: string;
  title: string;
  reviewed: boolean;
};

function RecordCard({ id, title, reviewed }: RecordCardProps) {
  return (
    <article className="record-card">
      <h3>{id}: {title}</h3>
      <StatusBadge
        label={reviewed ? "Reviewed" : "Pending"}
        tone={reviewed ? "positive" : "neutral"}
      />
    </article>
  );
}
```

A component can also accept nested JSX through the conventional `children` prop. This is composition. The wrapper controls structure while the caller supplies content.

Composition through children

```tsx
import type { ReactNode } from "react";
```

```

type PanelProps = {
  title: string;
  children: ReactNode;
};

function Panel({ title, children }: PanelProps) {
  return (
    <section className="panel">
      <h2>{title}</h2>
      <div className="panel__body">{children}</div>
    </section>
  );
}

function Overview() {
  return (
    <Panel title="Case overview">
      <p>Three records still require review.</p>
      <StatusBadge label="In progress" tone="warning" />
    </Panel>
  );
}
```

Use named props when the component needs a specific data contract. Use `children` when the component provides a layout or semantic wrapper around arbitrary nested content.

15.6 The component tree

Components compose into a render tree. Parent-child relationships describe ownership and data flow, but they do not necessarily match file imports one-to-one. A parent may receive a child as `children`, or several components may be implemented in one module.[60] Component- and message-based decomposition has a substantially older software-architecture lineage. For example, the C2 architectural style formalised GUI systems as compositions of components communicating through explicit messages.[7]

A component boundary is useful when it creates at least one of the following:

- a reusable visual or behavioural concept.
- a meaningful typed interface.
- an isolated responsibility that can change independently.
- a place where data ownership or error handling changes.
- a smaller unit that makes the parent easier to understand.

Very small fragments do not automatically deserve components. Excessive extraction creates indirection without improving ownership or reuse.

A composed page hierarchy

```
function DashboardPage({ records }: { records: RecordSummary[] })
    {
  const reviewedCount = records.filter((record) => record.reviewed
    ).length;

  return (
    <PageLayout title="Dashboard">
      <SummaryCards
```

```
        total={records.length}
        reviewed={reviewedCount}
      />
      <RecentRecordList records={records.slice(0, 5)} />
    </PageLayout>
  );
}
```

15.7 Render, reconciliation, and commit

A UI update has three conceptual stages:

1. a trigger requests rendering, such as the initial mount or a state update.
2. React calls components to calculate the next UI description.
3. React commits the necessary changes to the host DOM.

React documentation describes this as render and commit. Reconciliation compares the previous and next element trees, using component types, positions, and keys to determine which identities can be preserved.[61, 62]

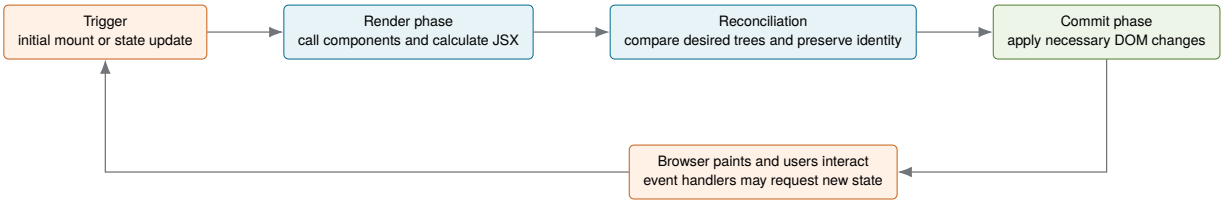


Figure 15.1: React separates calculating the desired UI from committing changes to the host DOM. Rendering must therefore remain pure and repeatable.

The informal term **virtual DOM** often refers to the in-memory descriptions used during this process. It should not be understood as a complete browser DOM clone. React elements do not contain layout, computed style, event dispatch, or the full browser API. They describe what the UI should contain. Research on compiler-augmented virtual DOM implementations such as Million.js also demonstrates that tree comparison and DOM update strategies have measurable implementation trade-offs. This should not be read as a claim that React uses the same algorithm.[10]

The virtual DOM is an accounting model, not a speed guarantee

React performs additional JavaScript work to calculate and compare UI descriptions. The benefit is a consistent declarative update model and targeted commits, not a universal claim that every React update is faster than every direct DOM operation.

A React application can still be slow because components perform expensive calculations, state is owned too high in the tree, large lists render unnecessarily, effects create repeated requests, or the browser performs costly layout and painting.

15.8 React owns its root DOM

After React mounts a root, application components should not independently rewrite descendants with `innerHTML`, `appendChild`, or manual class changes. Direct manipulation can create DOM that does not correspond to React's latest description. A later commit may overwrite it or preserve incorrect assumptions about identity.

Browser APIs remain appropriate for focus, media playback, measurements, third-party widgets, and other systems that are inherently outside the component description or require DOM nodes. Such integration is deliberate and usually mediated through refs and effects rather than scattered selectors.

One owner per DOM region

A migration can temporarily keep vanilla and React regions side by side, but they should not both modify the same descendants. Define separate mount points or a clear cut-over boundary.

15.9 Choosing React deliberately

React is a strong fit when the interface benefits from reusable components, declarative state-driven rendering, a large ecosystem, and integration with React-based frameworks. It also adds runtime code, a build requirement for JSX and TypeScript, framework conventions, and a new rendering model that the team must understand.

A small mostly static site may not need React. A lightweight library, server-rendered templates with progressive enhancement, or web components may be simpler. Conversely, a highly interactive application with many shared states and reusable views may benefit substantially from React's component model.

React also does not decide the rendering architecture. It can be used for a pure client SPA, hydrated server output, statically generated pages, selective interactive regions, or Server Components through a framework. The architecture decision from the previous chapter therefore remains separate from the library decision.

React is a rendering model, not a complete architecture

Choosing React answers how components describe and update UI. It does not by itself answer where rendering occurs, how routes load data, where server state is cached, how authentication works, or how the application is deployed.

15.10 Chapter summary

What to remember about React foundations

- React components describe the desired UI from data instead of issuing a sequence of DOM update commands.
- JSX is transformed JavaScript syntax that creates React elements. It is neither HTML text nor a complete DOM tree.
- Function components should remain pure. The same inputs should produce the same output without side effects during rendering.
- Props create explicit component contracts, while `children` supports composition through nested content.
- Components form a render tree whose boundaries should express meaningful responsibilities, ownership, or reuse.
- React separates rendering the next UI description from committing required changes to the host DOM.
- The virtual DOM is an informal model for in-memory UI descriptions and reconciliation, not an automatic performance guarantee.
- React owns the descendants of its root. Manual DOM rewrites inside that region conflict with the rendering model.
- Choosing React does not decide routing, data access, rendering location, persistence, or deployment architecture.

16 React Component Design and Routing

A React application becomes maintainable when component boundaries, data contracts, list identity, file organisation, and URLs are designed deliberately. This chapter develops the presentational structure of an application before adding substantial local and shared state.

16.1 Conditional rendering

React uses ordinary JavaScript control flow to select JSX. The appropriate form depends on whether the alternatives are whole component states, small inline differences, or optional fragments.[63]

16.1.1 Early return for distinct page states

An early return isolates an empty state

```tsx
function RecordList({ records }: { records: RecordSummary[] }) {
  if (records.length === 0) {
    return <p className="empty-state">No records match the current
     view.</p>;
  }

  return (
    <ul>
      {records.map((record) => (
        <li key={record.id}>{record.title}</li>
      ))}
    </ul>
  );
}
```

16.1.2 Ternary expression for one of two values

A ternary chooses one visible label

```tsx
<StatusBadge
  label={reviewed ? "Reviewed" : "Pending"}
  tone={reviewed ? "positive" : "neutral"}
/>
```

16.1.3 Logical AND for optional content

Render a note only when it exists

```tsx
{record.warning && (
  <p className="warning">{record.warning}</p>
)}
```

The left side should be a boolean. A numeric value such as `0` is rendered as text rather than disappearing, so write `count > 0 \&\& ...` when zero is possible.

A component may return `null` when it should render nothing. Returning an empty `div` adds an unnecessary DOM node and may affect layout or semantics.

16.2 Rendering collections and stable keys

Arrays are commonly transformed into JSX with `map`. Each sibling needs a stable `key` so React can match items across renders when entries are inserted, removed, or reordered.[64]

Render a collection with domain IDs as keys

```tsx
type RecordListProps = {
  records: RecordSummary[];
};

function RecordList({ records }: RecordListProps) {
  return (
    <ul className="record-list">
      {records.map((record) => (
        <li key={record.id}>
          <RecordCard {...record} />
        </li>
      ))}
    </ul>
  );
}
```

A key is interpreted relative to sibling elements. It is not automatically passed as a prop, and it does not need to be globally unique. It must be stable for the conceptual item among its siblings.

Array indexes are safe only when the collection is genuinely static. Items are never inserted, deleted, reordered, filtered, or associated with component state. Domain IDs are preferable whenever available.

Position is not identity

Using `key={index}` tells React that "the item at this position" is the same conceptual item after an update. If the list changes order, React may preserve local state or DOM identity for the wrong record.

16.3 Organising components by feature

A flat directory works for a small prototype but becomes difficult when many files have generic names such as `List.tsx`, `Card.tsx`, and `utils.ts`. A feature-oriented structure keeps code near the domain capability it supports:

Feature-oriented structure with component responsibilities

```
src/
  app/
    App.tsx                    # application composition
    router.tsx                 # routing configuration
  features/
    dashboard/
      DashboardPage.tsx        # feature component
      SummaryCards.tsx         # presentational, feature-local
    records/
```

```
      RecordListPage.tsx       # feature component
      components/
        RecordList.tsx         # presentational, feature-local
        RecordCard.tsx         # presentational, feature-local
      recordTypes.ts
    timeline/
      TimelinePage.tsx         # feature component
      components/
        TimelineList.tsx       # presentational, feature-local
        TimelineEvent.tsx      # presentational, feature-local
  shared/
    components/
      Panel.tsx                # presentational, reused across features
      StatusBadge.tsx          # presentational, reused across features
    format/
      dates.ts
```

Listing 16.3 illustrates two separate dimensions. Feature-oriented organisation describes where a component belongs, whereas **feature component** and **presentational component** describe the responsibility it has. A presentational component may remain inside a feature when it represents feature-specific concepts. Being presentational does not automatically make it shared.

A component remains feature-local when its meaning and change reasons belong to one feature. It moves to `shared` only after it has a genuinely reusable contract. Prematurely globalising components often produces a complicated prop API full of feature-specific exceptions. The broader idea of structuring interactive software around components with explicit interfaces and responsibilities is well established in software-architecture research.[7]

The distinction between **presentational** and **feature** components can help:

Presentational component Receives data and callbacks through props and focuses on rendering a reusable view.

Feature component Knows domain use cases, chooses data, owns state, coordinates routing, or translates application actions into presentational props.

16.4 Routing in a React SPA

React does not include a browser router. A routing library maps URLs to component trees and integrates links, nested layouts, route parameters, query parameters, and missing-route behaviour. React Router is a common choice and documents declarative route definitions, dynamic segments, and URL values.[65, 66] Alternatives include TanStack Router, which emphasises inferred TypeScript safety and typed search parameters, and Wouter, which provides a deliberately minimal routing API for React and Preact.[67, 68] In web-engineering literature, navigation is likewise treated as a first-class design concern alongside content, presentation, and application behaviour rather than as an incidental collection of links.[69]

Declarative routes for several views

```tsx
import {
  BrowserRouter,
  Navigate,
  Route,
  Routes
} from "react-router-dom";

export function AppRouter() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<AppShell />}>
          <Route index element={<DashboardPage />} />
          <Route path="records" element={<RecordListPage />} />
          <Route path="records/:recordId" element={<
    RecordDetailPage />} />
          <Route path="timeline" element={<TimelinePage />} />
          <Route path="*" element={<Navigate to="/" replace />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

The outer route can render a shared shell with an `Outlet`. Child routes replace only the nested view. A wildcard route makes the missing-route policy explicit.

16.4.1 Route parameters identify resources

Read and validate a dynamic route parameter

```tsx
import { useParams } from "react-router-dom";

function RecordDetailPage() {
  const { recordId } = useParams<{ recordId: string }>();
  const record = records.find((item) => item.id === recordId);

  if (!record) {
    return <p>Record not found.</p>;
  }

  return <RecordCard {...record} />;
}
```

A route parameter is text from the URL and may be absent or invalid. The component must handle that case rather than asserting that every URL names a real record.

16.4.2 Query parameters represent optional view state

Read filter state from the URL

```tsx
import { useSearchParams } from "react-router-dom";

function TimelinePage() {
  const [searchParams] = useSearchParams();
  const personId = searchParams.get("person");

  const visibleEvents = personId
```

```
    ? events.filter((event) => event.personIds.includes(personId))
    : events;

  return <TimelineList events={visibleEvents} />;
}
```

Use a path segment when the value identifies the resource represented by the route. Use a query parameter when it modifies how that resource collection is viewed, such as filtering, sorting, paging, or selecting an optional tab.

16.5 Chapter summary

What to remember about component design and routing

- Conditional rendering uses ordinary JavaScript control flow. Choose early returns, ternaries, and optional fragments according to the shape of the alternatives.
- Collections require stable domain keys so React can preserve the identity of conceptual items across insertion, deletion, filtering, and reordering.
- Component boundaries should separate meaningful responsibilities rather than merely make files smaller.
- Feature-oriented organisation keeps domain-specific components, types, and utilities close to the capability they support.
- Shared components should move to a common area only after they have a genuinely reusable contract.
- React does not include routing. A router maps URLs to component trees and integrates history, links, parameters, query values, and missing routes.
- Route parameters normally identify resources, while query parameters commonly represent optional view state such as filters and sorting.
- Client-side routes require server or static-host fallback configuration so refreshes and deep links still deliver the application entry document.

17 React State, Interactivity, and Reactive Data Flow

Static components become applications when users can trigger transitions and the rendered UI responds to changing data. React organises this through event handlers, component state, immutable updates, and one-way data flow. The central design problem is not where to place a Hook mechanically, but which component owns each fact and which values should be stored at all. The broader idea of declaratively deriving changing outputs from time-varying values has a research lineage in functional reactive programming, although React's programming model is not identical to it.[8]

17.1 Responding to events

React event handlers are functions passed to JSX props such as `onClick`, `onChange`, and `onSubmit`. Pass the function, but do not call it during render.[70]

A typed React click handler

```tsx
function ReviewButton() {
  function handleClick() {
    console.log("review requested");
  }

  return <button onClick={handleClick}>Mark reviewed</button>;
}
```

Console output after one click

```
review requested
```

React delegates and normalises much of its event handling internally. Application components normally attach handlers declaratively to the element that owns the interaction rather than manually registering and removing DOM listeners after each render.

Event delegation remains a valid browser technique, but it is no longer required merely because a list is re-rendered. React associates handlers with the component description and updates them as part of the commit.

17.2 State as component memory

A normal local variable is recreated on every render and changing it does not request another render. `useState` provides a value associated with the component's position in the render tree and a setter that requests an update.[71]

Local state changes the rendered result

```tsx
import { useState } from "react";

function ReviewToggle() {
  const [reviewed, setReviewed] = useState(false);

  return (
    <button onClick={() => setReviewed((current) => !current)}>
      {reviewed ? "Reviewed" : "Mark reviewed"}
```

```
    </button>
  );
}
```

A setter does not change the current snapshot

State behaves as a snapshot for a particular render. Calling the setter schedules a future render. It does not rewrite the current variable in place. Functional updates such as `setReviewed(current => !current)` are useful when the next value depends on the previous one.

State is local to each mounted component instance. Two occurrences of `ReviewToggle` at different tree positions receive independent state. React preserves state while the same component identity remains at the same position, and resets it when type or key changes.[62]

17.3 Controlled form inputs

An input is **controlled** when React state supplies its current value and an event handler updates that state. The rendered value therefore remains the source of truth.

A controlled search field

```
import { useState } from "react";

function RecordSearch() {
  const [term, setTerm] = useState("");

  return (
    <label>
      Search
      <input
        value={term}
        onChange={(event) => setTerm(event.target.value)}
      />
    </label>
  );
}
```

If `value` is supplied without an `onChange` that changes it, typing appears ineffective. The browser briefly receives input, but the next React render restores the state value.

A form with several related fields can use separate state variables or one object. Separate values are clear when fields change independently. A single object can be useful when the values form one domain record and are submitted or reset together. The state shape should follow update relationships rather than a rule that every input needs its own Hook.

17.4 Immutable state updates

Objects and arrays stored in state are ordinary mutable JavaScript values, but React code should treat each render's state as read-only. Create a new object or array for the next state rather than modifying the current one.[72, 73]

Toggle one item without mutating the state array

```ts
type RecordState = {
  id: string;
  title: string;
  bookmarked: boolean;
};

function toggleBookmark(items: RecordState[], id: string):
    RecordState[] {
  return items.map((item) =>
    item.id === id
      ? { ...item, bookmarked: !item.bookmarked }
      : item
  );
}
```

The outer array is new, and the changed item is new. Unchanged objects can be safely reused because their values did not change.

Sorting state requires a non-mutating result

```ts
const visibleRecords = records
  .filter(matchesCurrentFilters)
  .toSorted(compareRecords);
```

Where `toSorted` is not available for the selected target, copy first:

Copy before using mutating sort

```ts
const visibleRecords = [...records]
  .filter(matchesCurrentFilters)
  .sort(compareRecords);
```

New identity communicates an update

A state update gives React a new value to use in the next render. Do not change the value from the current render directly. Instead, pass a replacement value to the state setter. For objects and arrays, this means creating a new object or array rather than modifying the existing one. This keeps earlier state values unchanged, allows old and new values to be compared reliably, and prevents a change from unexpectedly affecting other code that uses the same object.

17.5 Stored state versus derived values

The previous sections showed how `useState` stores information between renders and how its setter replaces that information for a future render. A separate design question is which values should be stored as state at all. State should contain the minimal source of truth, which means information that can change independently and cannot be calculated from current props or other state. Values such as filtered lists, totals, percentages, and selected objects can often be derived during rendering instead.[74]

Derive a list and count from one source of truth

```ts
function RecordOverview({ records, term }: Props) {
  const visibleRecords = records.filter((record) =>
    record.title.toLowerCase().includes(term.toLowerCase())
  );

  const bookmarkedCount = records.filter(
```

```
    (record) => record.bookmarked
  ).length;

  return (
    <>
      <p>{bookmarkedCount} bookmarked</p>
      <RecordList records={visibleRecords} />
    </>
  );
}
```

In this example, neither `visibleRecords` nor `bookmarkedCount` needs its own `useState` call. Both values are fully determined by `records` and `term`. Whenever those inputs change and the component renders again, the derived values are calculated from the new inputs.

Storing `visibleRecords` separately would create two representations of the same facts, namely the source records and the supposedly corresponding filtered list. Every records or filter update would then have to keep both representations synchronised.

Do not store what you can calculate

Redundant state can become stale. Deriving values during render makes the relationship explicit and guarantees that the result corresponds to the inputs of that render.

17.5.1 Memoising an expensive derived value

Derived values are normally calculated directly during rendering, as in the previous example. If a calculation is measurably expensive, `useMemo` can reuse its previous result while its dependencies remain unchanged. React recalculates the value when one of those dependencies changes.[75]

Memoise an expensive derived list

```
import { useMemo } from "react";

function RecordOverview({ records, term }: Props) {
  const visibleRecords = useMemo(
    () =>
      records.filter((record) =>
        record.title.toLowerCase().includes(term.toLowerCase())
      ),
    [records, term]
  );

  const bookmarkedCount = records.filter(
    (record) => record.bookmarked
  ).length;

  return (
    <>
      <p>{bookmarkedCount} bookmarked</p>
      <RecordList records={visibleRecords} />
    </>
  );
}
```

The function passed to `useMemo` runs during rendering and must remain pure. The dependency array lists the reactive values used by the calculation, so that when either `records` or `term` changes, React calculates `visibleRecords` again. The inexpensive count remains a direct calculation because memoising every derived value would add complexity without necessarily improving performance.

Unlike `useState`, `useMemo` has no setter and does not represent independently changeable application data. Updating state requests a render, but `useMemo` only affects how a value is calculated during that render. Its result must therefore be derivable entirely from the listed dependencies.

Memoisation is not state

Do not use `useMemo` to preserve authoritative application data. React treats memoisation as a performance optimisation and may discard a cached value. The component must remain correct if the calculation runs again. Begin with a direct calculation and add memoisation only when measurement shows a meaningful cost or when stable identity benefits another measured optimisation.

17.6 Lifting state and passing callbacks

When several components need the same changing value, move its state to their nearest common ancestor. The owner passes values downward through props and passes callbacks that children can invoke to request changes. React documentation calls this **lifting state up**.[76]

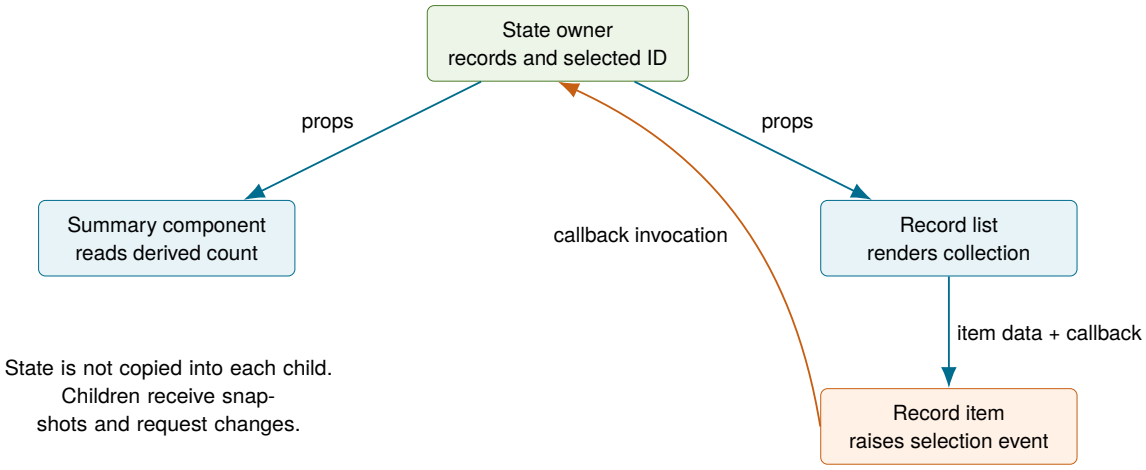


Figure 17.1: Unidirectional data flow: the owner passes data downward, while child interactions invoke callbacks that request updates at the owner.

One owner supplies data and an update callback.

```
type RecordsPageProps = {
  initialRecords: RecordState[];
};

function RecordsPage({ initialRecords }: RecordsPageProps) {
  const [records, setRecords] = useState(initialRecords);

  function handleToggleBookmark(id: string) {
    setRecords((current) => toggleBookmark(current, id));
  }

```

```
  return (
    <>
      <BookmarkSummary records={records} />
      <RecordList
        records={records} // pass data down
        onToggleBookmark={handleToggleBookmark} // pass callback
      />
    </>
  );
}
```

The child receives both the current records and a callback through its props. It reads the data to render the list and invokes the callback to request a change:

A child reads data and requests an update through props

```
type RecordListProps = {
  records: RecordState[];
  onToggleBookmark: (id: string) => void;
};

function RecordList({
  records,  // data received
  onToggleBookmark // callback received
}: RecordListProps) {
  return (
    <ul>
      {records.map((record) => (
        <li key={record.id}>
          <span>{record.title}</span>
          <button
            onClick={() => onToggleBookmark(record.id)} // execute
    callback
          >
            {record.bookmarked ? "Remove bookmark" : "Bookmark"}
          </button>
        </li>
      ))}
    </ul>
  );
}
```

The child does not reach into the parent or mutate its state. The callback is a capability with a clear name and parameter type. The child can request that one record be toggled, while the owner remains the single place that decides how the update is applied.

17.6.1 Prop drilling

Passing props through one or two intermediate components is ordinary explicit data flow. It becomes **prop drilling** when several components repeatedly forward values they do not otherwise use merely so that a distant descendant can receive them. For example, this layout uses `title` itself but only passes the record-related props onward:

An intermediate component forwards props it does not use

```
type RecordsLayoutProps = RecordListProps & {
  title: string;
};

function RecordsLayout({
```

```
  title,
  records,
  onToggleBookmark
}: RecordsLayoutProps) {
  return (
    <main>
      <h1>{title}</h1>
      <RecordList
        records={records}
        onToggleBookmark={onToggleBookmark}
      />
    </main>
  );
}
```

Long forwarding chains obscure dependencies

Prop drilling couples intermediate component APIs to data and actions they do not use. Adding, renaming, or removing a deeply consumed prop may then require changes at every level of the chain, making the actual owner and consumer harder to see. A short, explicit prop path is usually clearer.

17.7 Reducers for related state transitions

Several state variables may form one state domain whose transitions need names, validation, and coordinated resets. `useReducer` moves update logic into a reducer, which is a pure function from current state and action to next state.[77] Research on functional reactive programming provides a useful theoretical contrast by making time-varying values and their dependencies explicit and compositional.[78]

A typed reducer for search and filters

```
type FilterState = {
  term: string;
  status: "all" | "reviewed" | "pending";
  sort: "title" | "date";
};

type FilterAction =
  | { type: "termChanged"; term: string }
  | { type: "statusChanged"; status: FilterState["status"] }
  | { type: "sortChanged"; sort: FilterState["sort"] }
  | { type: "cleared" };

const initialFilters: FilterState = {
  term: "",
  status: "all",
  sort: "date"
};

function filterReducer(
  state: FilterState,
  action: FilterAction
): FilterState {
  switch (action.type) {
    case "termChanged":
      return { ...state, term: action.term };
    case "statusChanged":
```

```
      return { ...state, status: action.status };
    case "sortChanged":
      return { ...state, sort: action.sort };
    case "cleared":
      return initialFilters;
  }
}
```

Dispatch actions from a component

```
const [filters, dispatch] = useReducer(filterReducer,
    initialFilters);

<input
  value={filters.term}
  onChange={(event) =>
    dispatch({ type: "termChanged", term: event.target.value })
  }
/>

<button onClick={() => dispatch({ type: "cleared" })}>
  Clear filters
</button>
```

A reducer is valuable when transitions are easier to understand as domain events than as scattered setters. It does not create global state and does not inherently improve performance. It centralises state-transition logic.

17.8 Context and its limits

The prop-drilling example showed a distribution problem, where a value has a clear owner and consumer, but several components in between must forward it. Context is React's built-in mechanism for making a value available to a descendant subtree without passing it explicitly through every intermediate component. A context defines the channel, a provider supplies its value, and `useContext` reads the nearest matching provider above the consuming component. Consumers render again when that provided value changes.[79]

Context does not store state by itself. A provider may distribute a value from `useState`, `useReducer`, props, or another source. It is appropriate for data that is conceptually shared by a subtree, such as theme, locale, authenticated identity, or a domain state owner used by several descendants.

A typed context with a safe access Hook

```
import {
  createContext,
  useContext,
  useState,
  type ReactNode
} from "react";

type BookmarkContextValue = {
  bookmarkedIds: Set<string>;
  toggleBookmark: (id: string) => void;
};

const BookmarkContext = createContext<BookmarkContextValue | null
    >(null);

```

```tsx
export function BookmarkProvider({ children }: { children:
    ReactNode }) {
  const [bookmarkedIds, setBookmarkedIds] = useState<Set<string>>(
    () => new Set()
  );

  function toggleBookmark(id: string) {
    setBookmarkedIds((current) => {
      const next = new Set(current);
      next.has(id) ? next.delete(id) : next.add(id);
      return next;
    });
  }

  return (
    <BookmarkContext.Provider value={{ bookmarkedIds,
    toggleBookmark }}>
      {children}
    </BookmarkContext.Provider>
  );
}

export function useBookmarks() {
  const value = useContext(BookmarkContext);
  if (!value) throw new Error("useBookmarks requires
    BookmarkProvider");
  return value;
}
```

Context solves distribution, not state modelling. It does not decide which values should be stored, prevent mutation, handle server caching, or make every value globally appropriate. A frequently changing context value can also cause many consumers to re-render.

For shared client state that benefits from a store outside the component tree, external alternatives include Zustand[80], which exposes a small hook-based store, and Redux Toolkit[81], the Redux project's recommended toolset for structured Redux state logic. Such libraries introduce their own abstractions and should address a demonstrated need rather than replace local state and props by default.

Context is not a default state location

First identify the state owner. Use Context when distribution through the subtree is the problem. If two nearby components share a value, ordinary lifted state and props are usually clearer.

17.9 Common state-design problems

The most persistent React defects usually come from representation and ownership rather than Hook syntax.

A reliable review asks:

- What is the smallest source of truth?
- Which component owns it?
- Which values are derived rather than stored?
- Which events can update it?
- Are updates immutable?

Problem	Symptom	Preferred response
Duplicated state	two views disagree about the same fact	keep one owner and derive or pass the value
Redundant state	filtered list or count becomes stale	calculate it from current inputs during render
Direct mutation	update appears late or changes another consumer	create new objects and arrays for next state
State owned too low	sibling components cannot coordinate	lift state to the nearest common owner
State owned too high	unrelated updates re-render broad subtrees	move state closer to the components that use it
Excessive prop drilling	intermediary components only forward props	consider composition or a focused Context
Unnecessary global state	distant features become coupled	keep state local until sharing is demonstrated
Index-based identity	state follows list positions after reordering	use stable domain keys

Table 17.1: Common state problems and structural remedies

- Does the URL need to represent any of this state?
- Will state be intentionally preserved or reset when navigation changes the tree?

17.10 From React interaction to progressive web applications

This chapter completes the manuscript's focused treatment of React by adding local interaction, forms, derived values, shared ownership, reducers, and Context to the component and routing foundations. These mechanisms organise client-side UI state. They do not by themselves persist data, synchronise with a backend, or make an application available offline.

The following chapters introduces progressive web application capabilities. Installability, service workers, caching, offline behaviour, and application updates. Those browser-platform concerns can be combined with a React interface, but they are distinct from deciding how components represent, own, and distribute state.

17.11 Chapter summary

What to remember about React state and interactivity

- Event handlers are passed to JSX and request changes in response to user interaction. They should not be called during rendering.
- `useState` associates component memory with a position in the render tree and schedules a new render through its setter.
- Controlled form elements receive their current value from React state and report edits through event handlers.
- Objects and arrays in state should be treated as read-only snapshots. Create new values for updates instead of mutating previous state.
- Store the minimal source of truth and derive filtered lists, counts, selections, and percentages from current inputs during rendering.
- Shared state moves to the nearest common owner, which passes values and update callbacks down the tree.
- Reducers are useful when one state domain has several named, related transitions and coordinated reset rules.
- Context solves repeated deep prop transport, but it does not automatically provide ownership, persistence, reducers, or good global-state design.
- Duplicated, stale, overly global, and incorrectly owned state are representation problems rather than merely Hook-syntax problems.
- React state mechanisms organise client-side UI data. Persistence, backend synchronisation, and offline behaviour are separate concerns.

Progressive Web Applications

Progressive Web Application Foundations

18

A browser application normally begins as a collection of documents, styles, scripts, and network requests. Progressive Web Applications (PWAs) keep that foundation but add selected web-platform capabilities that can make the application behave more like software installed on the device. A PWA may be launched from an operating-system application surface, continue to provide useful functionality when the network is unreliable, receive selected background events, and integrate with device capabilities that are not available to a plain document.

The important word is *progressive*. A PWA is not a separate programming model and it is not a framework. It is still a web application. Additional behaviour is layered on top of the ordinary web experience and should degrade gracefully when a capability is unavailable. [82, 83] Web-engineering research has examined PWAs specifically as a web-native approach to cross-platform mobile application development and identified both their unification potential and platform-dependent trade-offs.[84]

18.1 From website to application experience

The boundary between a website and an installed application is less rigid than it first appears. A web application already has many properties associated with application software. It can maintain local state, communicate with remote services, render complex interfaces, access selected device APIs, and execute application logic in JavaScript. PWAs extend this model in three broad directions:

- **installation and integration**. The application can expose metadata that lets supporting browsers integrate it with operating-system launch surfaces and standalone windows.
- **network resilience**. A service worker can mediate requests and combine the network with explicitly managed cached responses.
- **background and re-engagement capabilities**. Selected browser APIs can deliver events to a service worker even when no application page is currently active.

These are capabilities rather than a universal checklist. Browser vendors differ in which features they support and in how they expose installation to users. A robust application should therefore not assume that every PWA capability is present simply because the application has a manifest or a service worker.[82, 85] Empirical analysis of PWA features in embedded web views further illustrates that available capabilities depend on the concrete web execution environment, not only on application code.[86]

A PWA is still a web application

A PWA does not replace HTML, CSS, JavaScript, HTTP, React, or the rendering architectures introduced earlier in this manuscript. It adds browser and operating-system integration around an ordinary web application. The same application can therefore remain usable in a normal browser tab even when installation, background execution, or another enhancement is unavailable.

18.2 Progressive enhancement as the design principle

Progressive enhancement means designing a reliable baseline first and adding optional capabilities on top of it. Feature detection is therefore preferable to assuming a particular browser environment.

Detecting service-worker support before registration

```
if ("serviceWorker" in navigator) {
  navigator.serviceWorker.register("/sw.js");
}
```

The same principle applies to notification, push, background synchronisation, and installation APIs. A missing capability should reduce functionality rather than break the application completely.

This also separates two concerns that are easy to conflate:

- **application functionality** describes what the product must allow the user to accomplish.
- **platform enhancement** describes additional ways in which that functionality can be delivered or integrated.

For example, an investigation portal might require users to read previously loaded case information while temporarily disconnected. That is an application requirement. Whether the application is also installable on the home screen is a separate enhancement. Likewise, a notification can be useful without being essential to the correctness of the underlying application.

18.3 The main PWA building blocks

Three platform concepts form the core of most PWAs:

1. the **Web App Manifest**, which describes how the application should appear and launch when integrated with the device.
2. the **Service Worker**, an event-driven worker that can intercept requests and respond to background events.
3. browser storage APIs such as **Cache Storage** and **IndexedDB**, which persist resources and application data across page loads.

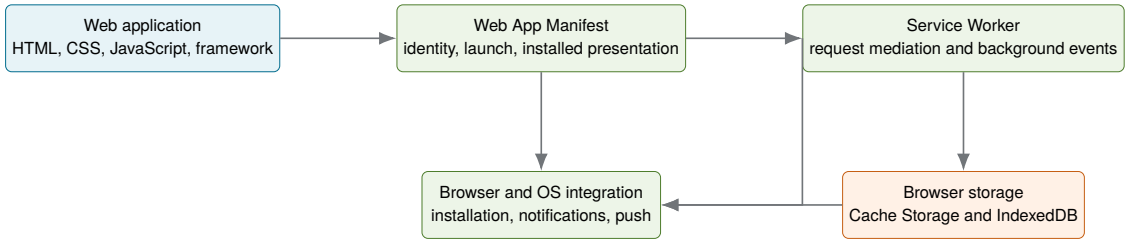


Figure 18.1: PWAs combine an ordinary web application with independent platform mechanisms. The manifest describes the installed experience, the service worker handles browser events, and persistent storage supports offline resources and application data.

The boundaries between these components matter. The manifest does not implement offline behaviour. Cache Storage does not intercept requests by itself. A service worker does not automatically make an application installable. Each mechanism solves a different part of the application problem.

Do not use “PWA” as shorthand for “service worker”

A service worker is one powerful browser mechanism commonly used by PWAs, but the concepts are not identical. An application can use a service worker without presenting itself as an installable PWA, and current installability requirements do not universally require a service worker. Treat the manifest, installation model, service worker, and offline design as related but separate concerns.[83, 87]

18.4 The Web App Manifest

A **Web App Manifest** is a JSON-based document containing metadata about a web application. It gives the browser a central description of information such as the application’s name, icons, starting URL, display preference, colours, and identity. The manifest specification defines the format, while browsers decide how much of that metadata they use for installation and operating-system integration.[88, 89]

The document is linked from HTML using a `link` element:

Linking a web app manifest

```html
<link rel="manifest" href="/app.webmanifest">
```

A small manifest can look as follows:

A minimal application-oriented manifest

```json
{
  "name": "Case Review Portal",
  "short_name": "Case Review",
  "id": "/",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "theme_color": "#1f4b66",
  "background_color": "#ffffff",
  "icons": [
    {
      "src": "/icons/app-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/app-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

The individual members answer different questions:

`name` and `short_name` provide human-readable labels. The shorter form can be used where display space is limited.

`id` identifies the installed application independently of a launch URL that may later change.

`start_url` specifies where the application should start when launched from the installed experience.

`scope` describes the navigation scope associated with the application.

`display` expresses a preferred display mode such as `standalone`, while the user agent remains responsible for the actual presentation.

`icons` provides application images in sizes and formats suitable for different device surfaces.

`theme_color` and `background_color` provide colour hints for browser and launch UI.

A manifest can contain additional metadata, including shortcuts and screenshots. It is usually better to begin with the properties that directly support identity, launch behaviour, and installation, and add richer metadata only when the target platforms make use of it.[89, 90]

18.5 Installation

Installation changes how the user reaches and experiences the web application. A supporting browser may expose the application through an install action. After installation, the application can appear alongside other applications in operating-system surfaces such as a launcher, start menu, dock, or home screen. Depending on the platform and manifest, it may open in a standalone window without the browser's usual navigation chrome.[87]

Installation does *not* mean that the application has been converted into a native executable. The installed experience still uses the browser engine and web security model. Its code and resources are still web resources, and updates are still delivered through the web deployment model.

Installability criteria are browser dependent. A manifest is central to the cross-browser installation model, but the exact metadata requirements, prompting behaviour, and available user interface differ between platforms. Consequently, application logic should not depend on one browser-specific installation prompt being present.

Installation is a user choice, not a deployment step

Deploying a PWA does not "install" it on user devices. The application becomes eligible for installation, and the browser or operating system decides which installation surfaces to expose. Users can continue to use the same application in an ordinary browser tab.

18.6 Secure contexts

Service workers and many other powerful web APIs are restricted to secure contexts. In normal deployment this means HTTPS. Browsers generally treat `http://localhost` as a secure context for local development, which allows service workers to be tested without obtaining a development certificate.[91]

The restriction is important because a service worker can intercept requests for pages within its scope and persist beyond an individual page load. Allowing an attacker on the network to inject or replace such code over an unencrypted connection would create a persistent compromise. HTTPS is therefore not merely a deployment convention around PWAs. It is part of the platform security boundary.

18.7 Manifest, service worker, and application shell

It is useful to distinguish three concepts that often occur together:

Mechanism	Main responsibility	Typical examples
Web App Manifest	Describe application identity and preferred installed presentation.	Name, icons, launch URL, display mode, colours.
Service Worker	Handle browser events independently from a particular document.	Request interception, cache strategies, push events, background sync.
Application shell	Provide the minimum interface structure needed to start the client application.	HTML entry point, core CSS, JavaScript bundle, basic offline UI.

Table 18.1: Different PWA mechanisms solve different problems

The **application shell** is an architectural pattern rather than a PWA API. In a client-rendered application, the shell may consist of the HTML entry point, core CSS, JavaScript bundles, icons, and a small set of static UI resources. These are attractive candidates for precaching because they are known at build time and needed before dynamic data can be displayed.

This distinction will become important in the next chapter. Caching the shell can make the interface start without the network, but it does not automatically make all application data available offline. Offline capability must be designed around the application's resource and data requirements.

18.8 Choosing what the PWA should improve

Before writing a service worker, define the desired user experience. Different applications need different levels of resilience and integration. Useful design questions include:

- Should the application start when completely offline?
- Which previously viewed information should remain available?
- Which actions may be performed offline and synchronised later?
- Which data must always be fresh before the user acts on it?
- Is installation useful on desktop, mobile, or both?
- Are notifications genuinely valuable, and when would users expect them?
- What should happen when a new application version becomes available while the user is working?

These questions turn PWA development from a collection of APIs into an engineering problem. A cache-first strategy, background sync, or push notification is useful only when it implements a deliberate product requirement.

Offline is a product state

"Works offline" is too vague to be a requirement. Decide which screens, resources, data, and operations remain meaningful with-

out connectivity. A good offline experience communicates what is available, what is stale, and which actions must wait for the network.

18.9 Chapter summary

What to remember about Progressive Web Applications

- A Progressive Web Application is an enhanced web application rather than a separate application platform or framework.
- Progressive enhancement adds capabilities such as installation and offline operation on top of a reliable web application baseline.
- The Web App Manifest describes application identity, appearance, launch behaviour, and installed presentation.
- Installation allows a web application to integrate more closely with the operating system, but the exact installation experience depends on the browser and platform.
- Service workers execute separately from the page and provide an event-driven environment for request handling and selected background capabilities.
- Browser storage can preserve resources and application data across page loads and periods without network connectivity.
- The manifest, service worker, and browser storage solve different problems and should not be treated as one PWA mechanism.
- PWA capabilities should be detected and used progressively because support and behaviour can differ between browsers and platforms.

Service Workers, Offline Operation, and Caching

19

Offline behaviour in a PWA is not produced by a special “offline mode” switch. It emerges from a programmable request path. A service worker can observe requests made by controlled pages and decide whether a response should come from the network, Cache Storage, a generated fallback, or another source. This makes the service worker a form of application-controlled proxy inside the browser.[92, 93]

The flexibility is powerful, but it introduces responsibility. Once application code begins deciding which response to return, freshness, invalidation, privacy, storage growth, update compatibility, and failure handling become part of the client architecture.

19.1 The service-worker execution model

A service worker is a worker context associated with an origin and registration scope. It runs separately from application documents and does not have direct DOM access. Communication with pages therefore occurs through mechanisms such as messages and the `clients` API rather than through direct element manipulation.[91]

Service workers are also event driven. The browser can start a worker to handle an event and terminate it again when there is no work to perform. Long-lived in-memory variables are therefore not a safe place for durable state. Persistent state belongs in browser storage such as Cache Storage or IndexedDB.[93, 94] Empirical research on real PWAs has additionally studied the resource and energy implications of service-worker use, showing that this architectural mechanism has effects that deserve measurement rather than being treated as cost-free infrastructure.[95]

A service worker is not a background server process

Do not assume that a service worker remains alive for the lifetime of the installed application. The browser controls its execution lifetime. Code must be written so that an event can be handled correctly even if the worker was stopped after the previous event and restarted later.

19.2 Registration and scope

A page registers a service worker through `navigator.serviceWorker`. A registration associates a service-worker script with a scope.

Registering a service worker

```
if ("serviceWorker" in navigator) {
  try {
    const registration = await navigator.serviceWorker.register("/
     sw.js");
    console.log("scope", registration.scope);
  } catch (error) {
    console.error("service worker registration failed", error);
  }
}
```

By default, the location of the service-worker script constrains the maximum scope it can control. A worker registered from `/app/sw.js`, for example, normally controls URLs beneath `/app/`. Applications commonly place the worker near the origin root when it is intended to control the entire application.[92, 94]

Registration does not mean that the current page is immediately controlled. The browser first runs the service-worker lifecycle, and an already-open page may continue to be controlled by an older worker or by no worker until navigation. Understanding this lifecycle is essential for both offline behaviour and updates.

19.3 Lifecycle events and functional events

The Service Workers specification distinguishes **lifecycle events** from **functional events**. The principal lifecycle events are `install` and `activate`. Events such as `fetch` perform application functionality after activation and should not be described as lifecycle phases.[93]

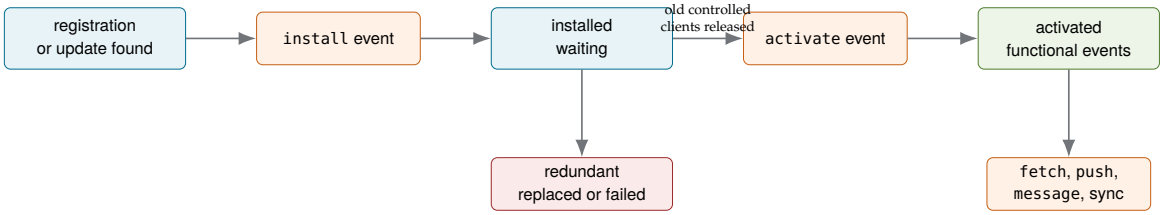


Figure 19.1: Simplified service-worker lifecycle. `install` and `activate` are lifecycle events. Events such as `fetch` are functional events handled after activation. A newly installed worker can wait while an older worker still controls open clients.

A newly discovered service worker progresses conceptually through the following states:

1. **installing**. The browser starts the new worker and dispatches `install`.
2. **installed / waiting**. Installation succeeded, but an older worker may still control open clients.
3. **activating**. The worker is becoming active and receives `activate`.
4. **activated**. The worker can handle functional events for clients in its scope.
5. **redundant**. The worker has been replaced or failed and is no longer used.

19.3.1 The install event

The `install` event is commonly used to prepare a minimum set of resources needed for offline startup. `event.waitUntil()` extends the event until the supplied promise settles. If required installation work fails, the browser can treat the installation as unsuccessful rather than activating a worker with an incomplete cache.[92]

Precaching a small application shell during installation

```
const SHELL_CACHE = "shell-v1";
const SHELL_RESOURCES = [
  "/",
  "/index.html",
  "/offline.html",
  "/assets/app.css",
```

```
  "/assets/app.js"
];

self.addEventListener("install", (event) => {
  event.waitUntil(
    caches.open(SHELL_CACHE).then((cache) => {
      return cache.addAll(SHELL_RESOURCES);
    })
  );
});
```

Precaching should remain deliberate. Large optional resources, user-specific data, and assets that may never be requested are often better cached at runtime rather than delaying installation with an unnecessarily large download.[96]

19.3.2 The activate event

The `activate` event is commonly used to migrate worker-managed state and delete caches that belong to obsolete application versions.

Removing obsolete application caches during activation

```
const CURRENT_CACHES = new Set(["shell-v2", "images-v1"]);

self.addEventListener("activate", (event) => {
  event.waitUntil(
    caches.keys().then((names) =>
      Promise.all(
        names
          .filter((name) => !CURRENT_CACHES.has(name))
          .map((name) => caches.delete(name))
      )
    )
  );
});
```

Cache deletion should target application-owned cache names rather than indiscriminately removing every cache on the origin. Several application features or independently deployed front ends may share the same origin.

19.4 Fetch interception

Once activated, a service worker can receive `fetch` events for requests in its scope. Calling `event.respondWith()` supplies a promise for the response that the controlled page will receive.[92]

A service worker that currently forwards every request to the network

```
self.addEventListener("fetch", (event) => {
  event.respondWith(fetch(event.request));
});
```

This simple example changes nothing from the user's perspective, but it establishes the important architectural position:

```
page request → service worker → network or local response
```

The worker can now select a strategy based on the request URL, method, destination, expected freshness, and failure mode.

19.5 Cache Storage

The **Cache Storage API** manages named `Cache` objects. Each cache stores mappings from requests to responses. It is available from service workers and window contexts in secure contexts.[97, 98]

Writing and reading one response with Cache Storage

```
const cache = await caches.open("images-v1");
const request = new Request("/images/map-preview.webp");

const response = await fetch(request);
if (response.ok) {
  await cache.put(request, response.clone());
}

const cached = await cache.match(request);
```

The response is cloned because a response body is a stream and cannot be consumed repeatedly without cloning or recreating it.

Cache Storage should not be confused with the browser's normal HTTP cache. The HTTP cache is influenced by HTTP caching semantics and is largely managed by the user agent. Cache Storage is application-managed storage. Entries do not automatically expire or refresh merely because an HTTP freshness lifetime has passed. The application or a library must implement its own retention and update policy.[98]

Table 19.1: HTTP caching and Cache Storage have different control models

Property	HTTP cache	Cache Storage
Population	Normally managed by browser networking according to HTTP behaviour.	Explicitly populated by application or service-worker code.
Lookup	Performed as part of browser network fetching.	Explicit through `cache.match()` or service-worker strategy code.
Expiration	Influenced by HTTP freshness and validation headers.	No automatic application-level expiry policy; code must replace or delete entries.
Typical role	General transfer optimisation.	Offline resources and explicitly controlled runtime caching.

19.6 Precaching and runtime caching

Two complementary approaches are common.

Precaching stores resources whose URLs are known during build or service-worker installation. Versioned JavaScript and CSS bundles, icons, and an offline fallback page are typical examples. A build tool can generate the exact list from the production output.

Runtime caching stores resources when requests occur. This is appropriate for resources whose exact URLs are not all known during the build,

such as responsive images, selected API responses, or content reached through user navigation.[96, 99]

The distinction is architectural rather than merely temporal. Precaching says, "this resource is part of the versioned application required for the intended baseline experience." Runtime caching says, "this response became useful while the application was running and may be worth retaining."

19.7 Caching strategies

A **caching strategy** defines the order in which the service worker consults the cache and network, when it updates the cache, and what happens on failure. Different resources should use different strategies because freshness, latency, and offline value differ.[100, 101] Controlled experiments on PWA caching have measured effects on both performance and energy consumption, reinforcing that cache policy is an engineering trade-off rather than a universally beneficial switch.[11]

19.7.1 Cache first

Cache first prefers a previously cached response and uses the network only on a miss. A successful network response can then be cached for future requests.

A simplified cache-first strategy

```
async function cacheFirst(request) {
  const cached = await caches.match(request);
  if (cached) {
    return cached;
  }

  const response = await fetch(request);
  const cache = await caches.open("static-v1");
  await cache.put(request, response.clone());
  return response;
}
```

Cache first (Fig. 19.2) is a strong choice for immutable or versioned resources such as hashed JavaScript bundles, fonts, icons, and images that change rarely. It reduces latency and works well offline. Its main risk is staleness when the application has no reliable invalidation policy.

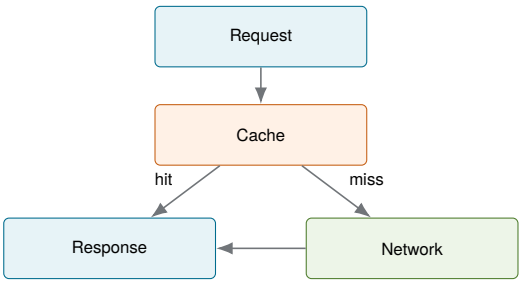


Figure 19.2: Cache-first strategy. The cache is checked first. The network is used only when no cached response is available.

19.7.2 Network first

Network first requests the freshest response and falls back to a cached value if the network fails.

A simplified network-first strategy

```
async function networkFirst(request) {
  const cache = await caches.open("data-v1");

  try {
    const response = await fetch(request);
    if (response.ok) {
      await cache.put(request, response.clone());
    }
    return response;
  } catch {
    const cached = await cache.match(request);
    if (cached) {
      return cached;
    }
    throw new Error("no network or cached response");
  }
}
```

This strategy, illustrated in Fig. 19.3, is useful when current data matters but an older response is still more useful than complete failure. It is less attractive when a poor connection causes a long delay before the network attempt fails. Production implementations often add a timeout or use a library strategy that already handles these details.

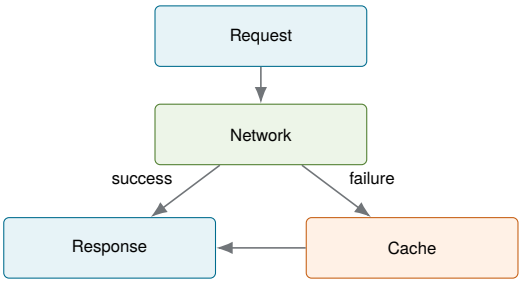


Figure 19.3: Network-first strategy. A fresh network response is preferred. The cache provides a fallback when the network request fails.

19.7.3 Stale while revalidate

Stale while revalidate prioritises fast display (Fig. 19.4). If a cached response exists, it is returned immediately while a network request refreshes the cache in parallel. The next request then receives the newer value.

A simplified stale-while-revalidate strategy

```
async function staleWhileRevalidate(request) {
  const cache = await caches.open("content-v1");
  const cached = await cache.match(request);

  const networkResponse = fetch(request).then(async (response) =>
    {
    if (response.ok) {
      await cache.put(request, response.clone());
    }
    return response;
  });

```

```
  return cached ?? networkResponse;
}
```

The strategy is suitable when slightly stale content is acceptable and perceived speed matters. It should not be used for data where showing an old value could cause an incorrect or unsafe decision.

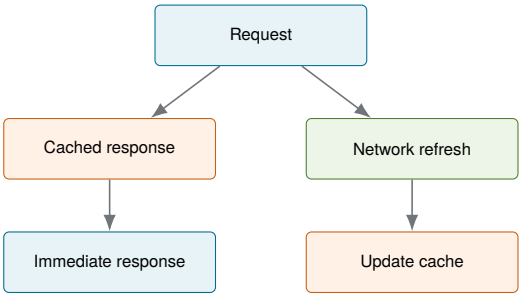


Figure 19.4: Stale-while-revalidate strategy. A cached response is returned immediately while the resource is refreshed from the network in parallel.

19.7.4 Offline fallback

A fallback strategy handles the case where neither the desired network response nor an appropriate cached response is available. Navigation requests can return a precached offline page, and image requests can return a placeholder.

Returning an offline page when navigation fails

```
self.addEventListener("fetch", (event) => {
  if (event.request.mode !== "navigate") {
    return;
  }

  event.respondWith(
    fetch(event.request).catch(() => caches.match("/offline.html")
    )
  );
});
```

A fallback should communicate the actual limitation. Showing an old page while silently implying that its data is current can be worse than showing a clear offline state.

19.8 Selecting strategies by resource semantics

A useful starting point is to classify requests according to how stale a response may become and whether a local copy is valuable without the network.

This table is not a rulebook. A map tile can be an excellent cache-first resource in one application and a poor candidate in another because of licensing, storage volume, or freshness requirements.

Resource	Typical starting strategy	Reasoning
Versioned JS and CSS	Precache or cache first	Build output is immutable for that version and required for startup.
Fonts and stable icons	Cache first	Rarely changing and useful across sessions.
Navigation documents	Network first or app-shell fallback	Freshness may matter, but offline navigation should degrade deliberately.
Frequently changing API GET	Network first	Prefer current data while retaining a deliberate fallback.
Non-critical content feed	Stale while revalidate	Fast display can be more valuable than perfect freshness.
Authentication and sensitive account operations	Network only	Stale or persisted responses can be misleading or inappropriate.

Table 19.2: Example strategy choices, with application semantics remaining decisive

19.9 What should not be cached blindly

The danger of PWA caching is not that caching is inherently unsafe, but that a generic "cache every successful response" policy ignores application semantics.

Particular care is required for:

- **authentication and authorisation responses**, where stale state may incorrectly suggest that a user is still authorised.
- **personal or sensitive data**, especially on shared devices, where persistent client storage creates privacy and logout concerns.
- **rapidly changing correctness-critical data**, where an old value can lead to an incorrect action.
- **unbounded collections**, such as arbitrary images or API responses, which can consume storage indefinitely without an eviction policy.
- **responses the server explicitly intends not to persist**, where application-level caching would contradict the system's data-handling policy.

Mutation requests such as `POST`, `PUT`, `PATCH`, and `DELETE` should be modelled as operations rather than ordinary cached resources. If the product must support offline writes, a common architecture stores pending operations in structured local storage such as IndexedDB and submits them later when connectivity returns. That introduces additional problems such as idempotency, ordering, conflicts, retries, and user feedback. It is not equivalent to caching a response.

Cache policy is part of the data model

For each cached category, define why it is cached, how long it may remain useful, how it is updated, how much storage it may consume, and when it must be removed. "Put it in Cache Storage" is an implementation step, not a complete caching policy.

19.10 Cache invalidation and versioning

Cached resources eventually become obsolete. Common invalidation techniques include the following:

- content-hashed filenames for build assets, where a changed file naturally receives a new URL.
- versioned cache names, with old versions deleted during activation.
- explicit maximum age or maximum entry counts for runtime caches.
- network revalidation strategies for data that should periodically refresh.

A build-generated precache is particularly effective for hashed assets because the build tool knows exactly which files belong to one application version. Libraries such as Workbox can maintain revision information and remove obsolete precache entries automatically.[99, 102]

19.11 Offline does not mean disconnected from state

An application can be offline while still containing several forms of state:

- static resources in Cache Storage.
- structured records in IndexedDB.
- UI state in memory.
- pending operations that have not yet reached the server.
- metadata describing when cached data was last refreshed.

The UI should distinguish these states explicitly. A cached record may be available but stale. A user action may be accepted locally but still pending synchronisation. A failed network request may have no meaningful fallback at all.

This is why offline-first architecture becomes more complex as soon as the application permits data modification. Reading a cached document is straightforward. Editing shared or server-authoritative data offline requires a synchronisation protocol and a conflict model.

19.12 Chapter summary

What to remember about service workers and caching

- A service worker is an event-driven browser worker that can mediate requests between an application, the network, and locally stored responses.
- The service-worker lifecycle separates installation and activation from functional events such as `fetch`.
- The `install` event is commonly used to prepare resources, while `activate` is commonly used to clean up outdated application state and caches.
- Cache Storage provides application-managed persistence for request–response pairs and is different from the browser's

automatically managed HTTP cache.

- Precaching stores resources known at build or installation time, while runtime caching stores resources encountered while the application is running.
- Cache first favours speed and offline availability, network first favours freshness, and stale while revalidate combines an immediate cached response with asynchronous refresh.
- Offline fallbacks make network failure an explicit application state instead of exposing only the browser's generic connection error.
- Caching strategy must follow resource semantics. Sensitive, correctness-critical, mutation-related, or unbounded data should not be cached indiscriminately.
- Cache entries require explicit lifecycle and invalidation decisions because application-managed caches do not automatically remain correct or bounded.

Background Capabilities, Updates, and PWA Tooling

20

Caching is only one reason to use a service worker. Because the browser can start a worker in response to selected events, PWAs can support tasks that are not tied to the lifetime of an open document. Push messages, notifications, and background synchronisation extend the application beyond a conventional page session. At the same time, the service-worker update model introduces a new deployment concern. *Two application versions can temporarily coexist in the same browser.*

Background APIs are intentionally restricted for privacy, battery, and abuse prevention, and browser support is not uniform. The engineering goal is therefore not to reproduce an unrestricted native background process. It is to use the smallest browser capability that supports a clear user need.[94, 103] These trade-offs also appear in the research literature on PWAs as a cross-platform application approach.[84]

20.1 Event-driven background operation

A service worker can be started when an event needs to be delivered even if no controlled page is currently running. Examples include a push event or, in supporting browsers, a background synchronisation event. Once the event has been handled, the browser may stop the worker again.[93, 94]

This model has two consequences:

1. background work must be finite and event oriented.
2. durable state must not depend on variables remaining in memory between events.

A worker can use `event.waitUntil()` to keep an event alive while an asynchronous operation completes, but that is not permission to run indefinitely. Long-running computation, continuous polling, and permanent socket ownership do not fit the service-worker lifecycle.

Do not model a service worker as a hidden server

A service worker may be restarted with an empty JavaScript heap. Persist information that must survive between events, and design each event handler so that it can reconstruct the state it needs.

20.2 Background synchronisation

The Background Synchronization API can defer selected work until the browser determines that connectivity is suitable. A common use case is retrying an operation that the user initiated while temporarily offline. Support is not uniform across browsers, so applications need a fallback path such as retrying when the page is next opened.[103] Background synchronisation has also been described and evaluated as an explicit architectural pattern for progressive web applications rather than merely as an API call.[104]

A robust design separates the operation from the transport attempt:

1. the user performs an action.
2. the application records the intended operation durably, for example in IndexedDB.
3. it attempts the network request.
4. if connectivity prevents completion, the operation remains pending.
5. background sync or a later foreground session retries it.
6. successful acknowledgement removes the pending operation.

This architecture raises questions that simple response caching does not:

- Can an operation be safely retried without creating duplicates?
- Does order matter between pending operations?
- What happens if server state changed while the client was offline?
- How does the UI distinguish *saved locally* from *accepted by the server*?

Consequently, offline mutation support should be designed together with server semantics such as idempotency keys, version checks, or domain-specific conflict handling.

Periodic Background Sync exists as a separate capability for occasional background refresh, but support and browser policy are substantially more restricted. It should be presented as an optional platform enhancement rather than a dependable cross-browser timer.[105]

20.3 Notifications and push are different APIs

The terms *notification* and *push notification* are often used interchangeably, but two distinct mechanisms are involved.

The **Notifications API** displays a system-level notification after the user has granted permission. A page or service worker can request that a notification be shown.

The **Push API** establishes a subscription that allows an application server to send a message through the browser's push infrastructure. A push event can wake the service worker. The worker may then update local state, communicate with open clients, or display a notification.[106, 107]

The conceptual flow proceeds as follows:

```
application server → push service → browser → service
    worker → notification or application logic
```

Push therefore concerns *delivery*, while notifications concern *presentation*. A local application can show a notification without receiving a push message, and a push event does not conceptually have to be described as the notification itself.

20.4 Permission handling

Powerful capabilities require user permission because they can interrupt, track, or otherwise affect the user beyond the current page. The Permissions API provides a common way to query the state of supported permissions, while individual APIs define how permission is requested.[108]

For notifications, the browser exposes three conceptual states:

- `default`. The user has not made a decision.
- `granted`. Notifications are allowed.
- `denied`. Notifications are blocked.

Requesting notification permission from a user-initiated action

```
async function enableNotifications() {
  const result = await Notification.requestPermission();

  if (result === "granted") {
    console.log("notifications enabled");
  }
}
```

Permission should be requested in context, normally after a user action that makes the benefit clear. Asking immediately on first page load gives the user little information about why the capability is useful and can permanently reduce the application's ability to use it if the request is denied.[106]

Permission denial is a normal application state rather than an exceptional error. The application should continue without the enhancement and provide settings or explanatory UI where appropriate.

Permission is part of UX design

The technical question is whether the API can be called. The product question is whether the user has enough context to make an informed choice. Request capabilities when their purpose is visible, explain their value, and make refusal non-destructive.

20.5 The service-worker update model

A service worker can outlive the page that originally registered it, so application updates cannot simply replace running code in place. When the browser detects a new service-worker script, it installs the new worker separately. If an older worker still controls open pages, the new worker normally waits. Once the old worker no longer controls clients, the new worker can activate.[92, 109]

This behaviour prevents a page from unexpectedly switching network semantics halfway through its lifetime. It also means that two versions may coexist temporarily:

- an old service worker controls already-open clients.
- a new service worker is installed and waiting.
- new application assets may already exist in a new cache.
- activation occurs later unless application code deliberately accelerates it.

A worker can call `self.skipWaiting()` to request immediate progression from waiting to activation, and an active worker can use `clients.claim()` to begin controlling eligible clients without waiting for their next navigation. These mechanisms are useful but should not be applied mechanically because changing the worker underneath an old page can create version skew between HTML, JavaScript, cached resources, and request behaviour.[110, 111]

20.6 Application update UX

The update strategy should follow the consequences of reloading the application.

20.6.1 Automatic update

An application can activate new worker code and refresh pages automatically. This is convenient when sessions are short, local unsaved state is negligible, and running the newest version is more important than preserving the current view.

20.6.2 Prompted update

Applications with long-lived sessions or unsaved state often show a message such as “A new version is available” and let the user choose when to reload. This is especially valuable for forms, editors, or multi-step workflows where an automatic refresh could discard work. The Vite PWA plugin explicitly supports both prompt-based and automatic update behaviour.[112, 113]

An application update is a state transition

A new deployment can change JavaScript, HTML, cached assets, API expectations, and the service worker itself. Decide when those changes may become visible to an already-running client. Treat update UX as part of application state management rather than as a hidden cache detail.

20.7 Testing offline behaviour

PWA bugs often appear only after the application has been visited before, a worker has changed versions, a cache contains old resources, or the network disappears at a particular moment. Testing only a clean online page load misses these states.

Browser developer tools provide dedicated views for service workers and origin storage. In Chromium-based developer tools, the Application panel can inspect the manifest, service-worker registrations, Cache Storage, and other persisted data. Network tools can simulate offline operation and throttled connections.[114]

A useful manual test matrix includes the following cases:

- first visit with an empty origin storage.
- second visit after the service worker controls the page.
- reload while completely offline.
- navigation to a route that was not previously visited.
- slow or intermittently failing network rather than only perfect online/offline states.
- application update while an old tab remains open.
- clearing site data and registering from scratch.
- denied notification permission.
- storage containing an older cache version.

Production builds should also be tested. Build-generated asset names, precache manifests, and plugin behaviour differ from an ordinary development server. A development environment that bypasses the real service worker can hide deployment-specific errors.

20.8 Workbox

Writing a small service worker by hand is valuable for understanding the platform. Production service workers, however, repeatedly need routing, versioned precaching, cache strategies, cache expiry, navigation fallbacks, and update handling. **Workbox** is a set of modules that implements common service-worker patterns while still exposing explicit configuration.[115]

Important Workbox modules include the following:

- `workbox-precaching` for build-time resources and revision-aware precaching.
- `workbox-routing` for matching requests and assigning handlers.
- `workbox-strategies` for cache-first, network-first, stale-while-revalidate, and related policies.
- `workbox-expiration` for limiting runtime caches by age or number of entries.
- `workbox-window` for service-worker registration and page-to-worker interaction.

The conceptual advantage of learning the native APIs first is that Workbox names correspond directly to mechanisms already understood.

Expressing a runtime strategy with Workbox

```
import { registerRoute } from "workbox-routing";
import { StaleWhileRevalidate } from "workbox-strategies";
import { ExpirationPlugin } from "workbox-expiration";

registerRoute(
  ({ request }) => request.destination === "image",
  new StaleWhileRevalidate({
    cacheName: "runtime-images",
    plugins: [
      new ExpirationPlugin({
        maxEntries: 60,
        maxAgeSeconds: 7 * 24 * 60 * 60
      })
    ]
  })
);
```

The library removes boilerplate, not architectural responsibility. Developers still decide which routes match, which strategy is correct, how much staleness is acceptable, and which data must not be stored.

20.9 PWA integration with Vite

The Vite ecosystem provides `vite-plugin-pwa`, which connects application builds with manifest generation, service-worker generation, Workbox, and registration. The plugin can generate a complete service

worker for common cases or compile a custom service worker and inject a build-generated precache manifest.[112, 116]

A minimal configuration can generate the manifest and a service worker from the Vite build:

Adding PWA generation to a Vite project

```
import { defineConfig } from "vite";
import { VitePWA } from "vite-plugin-pwa";

export default defineConfig({
  plugins: [
    VitePWA({
      registerType: "prompt",
      manifest: {
        name: "Case Review Portal",
        short_name: "Case Review",
        start_url: "/",
        display: "standalone",
        theme_color: "#1f4b66"
      }
    })
  ]
});
```

The plugin offers two major service-worker generation approaches:[112, 117]

`generateSW` Workbox generates the service worker from configuration. This is the default and is appropriate when standard routing and caching strategies are sufficient.

`injectManifest` the application provides a custom service-worker source file. The plugin builds it and injects the build's precache manifest. This is appropriate when application-specific service-worker logic is required.

The choice mirrors a general tooling principle from earlier chapters. Begin with generated conventions when they express the required behaviour clearly, and move to custom code when the application actually needs control that configuration cannot express.

Do not begin PWA learning with the plugin

A one-line plugin can hide the distinction between manifest metadata, service-worker registration, lifecycle events, precaching, runtime caching, and update behaviour. Implement or at least inspect a small native service worker first. Then use Workbox and Vite to automate patterns whose semantics are already understood.

20.10 Framework integration and React

PWAs are orthogonal to React. React controls component rendering and client-side state. The service worker sits outside the React tree and participates in browser-level request and background-event handling. A React application can therefore use the same manifest, service-worker, caching, and push concepts as a non-React application.

Framework integration mainly improves the bridge between worker state and application UI. For example, the React integration of `vite-plugin-pwa`

can expose state indicating that offline resources are ready or that a new version is waiting. A React component can render an update banner, but the underlying lifecycle is still the browser service-worker lifecycle.[118]

This separation is useful architecturally:

- React owns UI state and rendering.
- the service worker owns request interception and service-worker events.
- persistent browser storage owns durable offline data.
- the application defines the protocol between these layers.

20.11 A production decision checklist

Before shipping PWA behaviour, verify the decisions behind the implementation:

- Which resources are precached, and why are all of them required?
- Which runtime requests are cached, and what freshness policy applies?
- What is the cache size or expiry policy?
- Which personal or sensitive data can persist on the device?
- What does each screen show when offline?
- Can any user action be queued offline, and how are retries made safe?
- How does the application behave when permissions are denied?
- What happens when a new worker is waiting while the user has unsaved state?
- Has the production build been tested with old caches, a slow network, and no network?

20.12 Chapter summary

What to remember about PWA background operation and tooling

- Service workers enable event-driven background capabilities rather than unrestricted or permanently running background execution.
- Background synchronisation can retry durable pending operations when connectivity returns, but availability depends on browser support.
- Push and notifications are separate mechanisms. Push delivers information to a service worker, while notifications present information to the user.
- Permission requests should be contextual, user initiated, and optional. Denying a permission must remain a supported application state.
- A newly installed service worker can remain waiting while an older worker still controls open clients, so deployment does not imply immediate application replacement.
- Applications need an explicit update strategy and update UX, particularly when automatically reloading could discard unsaved user work.
- Browser developer tools can inspect manifests, service workers,

Cache Storage, worker updates, and offline behaviour and should be part of PWA testing.

- Workbox provides reusable implementations of common service-worker routing, caching, expiration, and precaching patterns.
- `vite-plugin-pwa` integrates PWA generation with Vite and can either generate a service worker or incorporate a more customised Workbox-based implementation.
- Tooling reduces implementation boilerplate but does not decide application semantics such as freshness, privacy, offline behaviour, cache policy, or update UX.

Web Accessibility and Performance

21 Web Performance

Web performance is both a technical property and a user-experience property. A page can be functionally correct and still feel broken when its content appears too late, interaction is delayed, or the layout moves while the user is trying to act. Performance engineering therefore asks what work the browser and network must perform and when the user perceives useful progress.

The browser-loading process from the architecture chapters is important here because performance problems rarely come from one isolated number. Network latency, resource size, HTML parsing, JavaScript execution, style calculation, layout, painting, and later user interactions all contribute to the experience. The goal of this chapter is to build a model that makes those costs observable and gives developers concrete levers for reducing them. Large-scale measurement research likewise finds that website complexity is multidimensional and that page composition, object counts, and dependency structure influence performance.[119]

21.1 Performance as latency, work, and perception

A useful first distinction is between **latency** and **work**. Latency is time spent waiting for something that is not yet available, such as a DNS result or a network response. Work is computation that must actually be performed, such as parsing JavaScript, calculating layout, decoding an image, or running an event handler. Both can delay the user, but they require different remedies.

Performance also has a perceptual dimension. Users do not experience a waterfall chart or a CPU profile directly. They experience questions such as:

- Is there useful content yet?
- Can I interact with it now?
- Does the interface respond promptly when I click or type?
- Does content unexpectedly move while I am reading or interacting?

This is why a single “page load time” is rarely enough. Modern performance measurement separates loading, responsiveness, and visual stability rather than compressing the entire experience into one timestamp.[120]

Performance is a path, not one number

A fast server does not guarantee a fast page, and a small JavaScript bundle does not guarantee responsive interaction. Network transfer, parsing, execution, rendering, and user interaction form a chain. Improving the slowest relevant parts of that chain is more useful than optimising an isolated metric without understanding what it represents.

21.2 From navigation to pixels

When a user navigates to a URL, the browser must first obtain the resources that describe the page and then transform those resources into pixels. The exact network protocol can change the details, but the conceptual flow remains useful:[121, 122]

1. resolve the destination and establish the required network connection.
2. send the request for the document and begin receiving HTML.
3. parse the HTML into the Document Object Model (DOM).
4. discover and request dependent resources such as stylesheets, scripts, fonts, and images.
5. parse CSS into the CSS Object Model (CSSOM) and execute JavaScript where required.
6. combine document structure and styles into the structures needed for rendering.
7. calculate layout, determining the size and position of visible elements.
8. paint pixels and composite the resulting layers.
9. continue executing application code and responding to later user input.

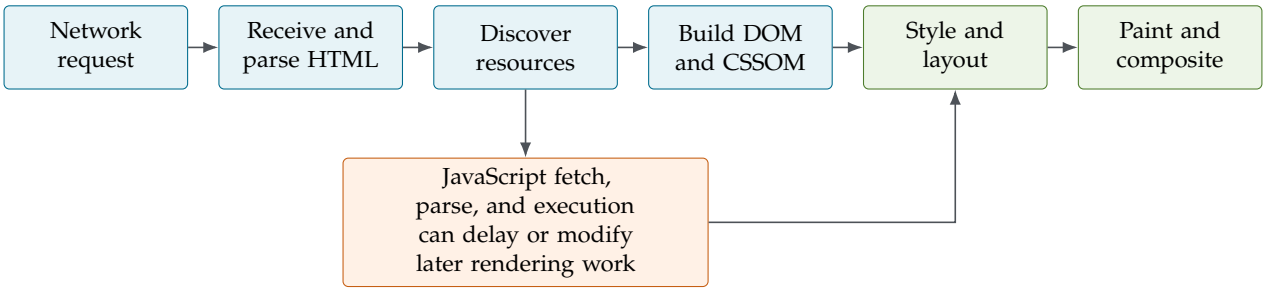


Figure 21.1: A simplified browser loading and rendering pipeline. The stages overlap in real browsers; the diagram emphasises dependencies rather than a strictly sequential implementation.

These stages overlap. A browser does not necessarily wait until the complete HTML document has arrived before doing anything else. It can parse progressively, discover subresources, and start useful work while later bytes are still arriving. This makes resource ordering important. A resource discovered early can influence when later work becomes possible. WProf's dependency analysis of page loads provides an empirical model of these relationships and shows how networking, computation, and browser dependencies jointly determine load time.[9]

21.3 The critical rendering path

The **critical rendering path** describes the browser work required to turn HTML, CSS, and JavaScript into rendered content. The DOM represents document structure, the CSSOM represents styling rules, and the browser combines the relevant information to determine what must be rendered. Layout calculates geometry. Paint turns that geometry into pixels.[122]

A simplified dependency is.

HTML → DOM CSS → CSSOM DOM + CSSOM → rendering → layout → paint

JavaScript can intervene in this path because it can inspect or modify the DOM and styles. A classic script encountered during HTML parsing is therefore potentially parser-blocking. The browser may need to stop parsing, obtain the script, and execute it before it can safely continue.

21.3.1 Classic, deferred, asynchronous, and module scripts

The loading mode of a script determines how its network fetch and execution interact with HTML parsing.

Form	Fetching	Execution relative to HTML parsing
`<script src="...">`	requested when discovered	parsing normally pauses until the script is available and executed
`<script defer src="...">`	fetched in parallel	executes after HTML parsing; document order is preserved among deferred scripts
`<script async src="...">`	fetched in parallel	executes when available; execution order is not tied to document order
`<script type="module" src="...">`	module graph fetched in parallel	deferred by default for ordinary module scripts

Table 21.1: Simplified script-loading behaviour.

The correct choice depends on dependencies. An analytics script that does not depend on surrounding markup may be suitable for asynchronous execution. Application modules commonly benefit from module semantics and deferred execution. A script that must run before later markup is interpreted is a different case.

Blocking is a dependency decision

Removing all blocking work is not a meaningful goal. Some work is genuinely required before useful content can appear. Performance engineering is about identifying what is critical, loading that work early, and keeping unrelated work off the critical path.

21.4 Main-thread work and responsiveness

The runtime model from Chapter 1 becomes a performance concern when JavaScript occupies the browser's main thread for too long. While a long task is executing, the browser cannot promptly run another JavaScript callback on that thread. Depending on the work involved, rendering and input handling can also be delayed.

Consider an input handler that performs an expensive calculation synchronously:

A long handler delays visible feedback

```
1 button.addEventListener("click", () => {
2   status.textContent = "Working...";
3
4   let result = 0;
```

```
  for (let i = 0; i < 300_000_000; i += 1) {
    result = (result + i) % 97;
  }

  output.textContent = String(result);
});
```

Although the first line changes the DOM, the browser may not paint the "Working..." state immediately because JavaScript continues occupying the main thread. The user experiences the delay as an unresponsive control.

Reducing main-thread work can involve doing less work, splitting work into smaller pieces, postponing non-critical work, moving suitable computation to a worker, or avoiding repeated calculations altogether. The correct intervention depends on profiling rather than guesswork.

21.5 Measuring what users experience

Performance metrics are useful only when their meaning is clear. Several measurements describe different points in loading and interaction.

Table 21.2: Common web performance measurements

Metric	What it approximates	Typical interpretation
TTFB	time until the first response byte becomes available	server/network responsiveness before document transfer progresses
FCP	first contentful paint	when the browser first paints content from the document
LCP	largest contentful paint	when the main visible content has likely become available
INP	interaction to next paint	responsiveness across user interactions
CLS	cumulative layout shift	unexpected visual movement during the page lifetime

Older material often uses **Time to Interactive** (TTI) as a single point at which a page is considered reliably interactive. It can still be useful when reading historical performance material, but current Core Web Vitals focus on LCP, INP, and CLS because they capture user-visible loading, interaction responsiveness, and visual stability more directly.[120]

At the time of writing, the recommended "good" Core Web Vitals thresholds are LCP at or below 2.5 seconds, INP at or below 200 milliseconds, and CLS at or below 0.1, evaluated at the 75th percentile of page visits.[120]

Lab measurements and real-user measurements answer different questions

A repeatable local or synthetic test is excellent for comparing changes under controlled conditions. Real-user monitoring reveals what actual devices, networks, locations, and usage patterns experience. Mature performance work uses lab data to diagnose and reproduce issues and field data to understand the population.

21.6 Optimising resource loading

A performance optimisation should remove unnecessary work, shorten a dependency, or perform unavoidable work at a better time. Common strategies include the following:

21.6.1 Reduce bytes and requests where they matter

Compression, minification, efficient formats, and removing unused code reduce transfer and processing cost. Bundling can reduce request overhead, but modern HTTP makes "fewer files at any cost" an oversimplification. Large bundles can delay code that the current page does not need. The build pipeline in Part 6.10 therefore needs to balance caching, chunking, and dependency boundaries rather than merely producing one file.

21.6.2 Load non-critical code later

Lazy loading delays work until it is likely to be needed. Dynamic imports are a natural mechanism for splitting JavaScript by feature or route:

Loading a feature only when it is needed

```
async function openAnalytics() {
  const { showAnalytics } = await import("./analytics.js");
  showAnalytics();
}
```

The feature is absent from the initial execution path and can become a separate build chunk. This improves startup only if the deferred feature is genuinely non-critical. Lazy-loading something the user needs immediately merely moves the delay to a later moment.[123]

21.6.3 Give images explicit geometry and appropriate sources

Images frequently dominate transferred bytes and can also create layout shifts. Responsive image metadata lets the browser select an appropriate resource, while explicit dimensions reserve layout space before the image arrives.

Responsive image with reserved layout space

```
<img
  src="case-640.jpg"
  srcset="case-640.jpg 640w, case-1280.jpg 1280w"
  sizes="(min-width: 900px) 720px, 100vw"
  width="1280"
  height="720"
  alt="Investigation workspace showing three evidence cards"
  loading="lazy"
>
```

Native `loading="lazy"` is appropriate for many images outside the initial viewport. It should not be applied mechanically to the primary above-the-fold image because delaying a critical image can make the initial experience worse.

21.6.4 Cache and move content closer

HTTP caching avoids repeatedly transferring resources that have not changed. Content delivery networks can serve static assets from infrastructure closer to users and reduce latency. Long-lived caching works especially well with content-hashed build artefacts because a changed resource receives a new URL while unchanged resources remain reusable.

21.7 Preventing unnecessary layout and paint work

Performance does not stop after first render. JavaScript and CSS can repeatedly trigger style calculation, layout, paint, and compositing.

A visible layout shift occurs when content changes position without the user expecting it. Typical causes include images without reserved dimensions, advertisements or embeds inserted above existing content, and late font or component changes that alter geometry. Reserving space before asynchronous content arrives prevents many of these shifts.

Repeatedly reading layout information and then changing layout-affecting styles inside a loop can also force the browser to alternate between measurement and recalculation. A better pattern is to collect information, calculate the desired result, and batch DOM changes where practical.

Optimise only after measuring

A micro-optimisation can make code harder to understand without changing anything users can perceive. Use the browser's Network and Performance tools, Lighthouse or comparable audits, and application-specific measurements to identify the actual bottleneck before changing architecture.

21.8 Performance budgets and regression prevention

Performance is easy to lose gradually. A new analytics package adds JavaScript. A new image is larger than expected. Another font weight is loaded. A feature adds work to every render. A **performance budget** turns important limits into engineering constraints, for example.

- maximum initial JavaScript transferred.
- maximum image weight for a page type.
- a target LCP or INP under a defined test profile.
- no unexpected layout shift above an agreed CLS threshold.

Budgets are most useful when they are measurable in CI or monitoring rather than existing only as documentation. They make performance a property that can regress and be detected, similar to tests, lint rules, and type checks.[124]

21.9 Chapter summary

What to remember about web performance

- Web performance includes network latency, resource transfer, parsing, JavaScript execution, rendering work, and later interaction responsiveness.
- The critical rendering path transforms HTML and CSS into the structures needed for layout and paint. JavaScript can block or modify that path.
- Script loading mode determines whether fetching and execution block HTML parsing or can overlap with it.
- Long-running JavaScript can delay rendering and interaction because important browser work shares the main thread.
- TTFB, FCP, LCP, INP, and CLS describe different parts of the experience. One metric cannot represent the entire page.
- Current Core Web Vitals focus on loading performance, responsiveness, and visual stability through LCP, INP, and CLS.
- Effective optimisation removes unnecessary work, reduces critical bytes and dependencies, defers non-critical resources, reserves layout space, and uses caching appropriately.
- Measure first, optimise the observed bottleneck, and use performance budgets to prevent regressions.

22 Web Accessibility

Web accessibility means designing and implementing web content so that people can perceive, understand, navigate, and interact with it using different abilities, devices, and assistive technologies. It is not a specialised layer added after an interface is finished. Decisions about HTML semantics, keyboard interaction, focus, colour, forms, images, and dynamic updates determine whether the same application can be used through different modes of interaction.[125, 126]

Accessibility benefits users with permanent disabilities, but the underlying design principles are broader. A keyboard-operable control is also useful when a pointing device is unavailable. Clear labels help screen-reader users and reduce ambiguity for everyone. Captions help people who are deaf or hard of hearing and also users in environments where audio cannot be played. The central engineering lesson is therefore to avoid assuming that every user perceives and operates the interface in the same way.

22.1 WCAG and the POUR principles

The Web Content Accessibility Guidelines (WCAG) organise accessibility requirements around four principles, commonly abbreviated as **POUR**. Content should have the following properties.[126, 127]

Perceivable Information and interface components must be available in forms users can perceive, for example through text alternatives, captions, sufficient contrast, and adaptable structure.

Operable Users must be able to operate the interface, including through keyboard interaction, usable focus behaviour, sufficient time, and navigation mechanisms.

Understandable Content and interactions should be readable, predictable, and supportive when users make mistakes.

Robust Markup and interface semantics should be interpretable by browsers and assistive technologies across different environments.

WCAG 2.2 defines testable success criteria at conformance levels A, AA, and AAA. The levels should not be interpreted as three independent versions of accessibility. Higher conformance builds on the lower levels. Legal requirements vary by jurisdiction, but WCAG provides the central technical reference used by many accessibility policies and evaluation practices.[127] Empirical work with blind web users has also shown that conformance guidance does not capture every usability barrier encountered in practice, so guideline compliance should be treated as necessary evidence rather than a complete substitute for user-centred evaluation.[12]

Accessibility is an interface contract

A visual design is only one representation of an interface. The document must also expose enough structure, names, roles, states, relationships, and interaction behaviour for keyboard users and assistive technologies to operate it reliably.

22.2 From DOM to accessibility tree

Browsers maintain more than the visual DOM representation introduced earlier. Semantic information from HTML, ARIA, control states, and other browser knowledge is exposed through platform accessibility APIs. Assistive technologies such as screen readers use this information to understand the interface.

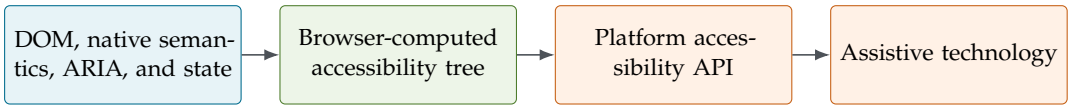


Figure 22.1: A simplified accessibility information flow. The browser derives semantic information from document structure and state and exposes it through platform accessibility APIs to assistive technologies.

The resulting **accessibility tree** is related to the DOM but is not a copy of every DOM node. Decorative or hidden content may be omitted, while semantic information such as a control's role, accessible name, checked state, or expanded state becomes important. The consequence is practical. Two interfaces that look identical can expose very different information to assistive technology.

Browser developer tools can inspect this semantic representation. Doing so is often more informative than looking only at source markup because it reveals what role and accessible name the browser actually computed.

22.3 Start with semantic HTML

The strongest default accessibility strategy is to use native HTML elements for their intended purpose. A native `button` already participates in keyboard navigation, exposes button semantics, supports disabled state, and has well-defined activation behaviour. A generic `div` does not gain those behaviours merely because CSS makes it look like a button.[128]

Native semantics provide behaviour as well as meaning

```
<!-- Prefer this -->
<button type="button">Save changes</button>

<!-- Avoid reimplementing the same control from a generic element
     -->
<div class="button-like">Save changes</div>
```

Similarly, headings should reflect document hierarchy, links should navigate, labels should identify form controls, and landmarks such as `header`, `nav`, `main`, `aside`, and `footer` should describe page regions.

Semantic HTML reduces code

Native controls are not only more accessible by default. They also remove custom JavaScript that would otherwise be required to reproduce focusability, keyboard activation, states, and browser behaviour. Accessibility and maintainability often point toward the same implementation.

22.4 Accessible names and text alternatives

Interactive elements need a meaningful **accessible name**. For a normal button, visible text often provides that name automatically. Icon-only controls require an explicit textual alternative.

An icon-only button still needs a name

```html
<button type="button" aria-label="Close dialog">
  <svg aria-hidden="true" viewBox="0 0 24 24">
    <!-- decorative icon paths -->
  </svg>
</button>
```

Images require the same kind of purpose-based reasoning. The question is not simply "what objects are visible?" but "what information or function would be lost if the image could not be seen?"

- **Informative images** need alternative text that communicates the relevant information or purpose.
- **Functional images** need an alternative that describes the action or destination rather than merely the visual appearance.
- **Decorative images** should normally be ignored by assistive technology, commonly through an empty `alt` attribute when using an `img` element.

Alternative text follows the image purpose

```html
<img
  src="trend-up.svg"
  alt="Request volume increased by 18 percent this week"
>

<img
  src="decorative-divider.svg"
  alt=""
>
```

Avoid redundant phrases such as "image of" when the assistive technology already announces that the item is an image. The alternative should communicate the information needed in context rather than mechanically describing every visible detail.

22.5 Keyboard interaction and focus

An interface is not operable if important functionality requires a mouse or precise pointer movement. Native links, buttons, and form controls already participate in the browser's keyboard model. Custom interactive widgets must reproduce the expected keyboard behaviour deliberately.[128, 129]

The `tabindex` attribute is useful but easy to misuse:

- `tabindex="0"` places an otherwise non-tabbable element in the normal sequential focus order.
- `tabindex="-1"` allows an element to receive focus programmatically without adding it to ordinary Tab navigation.
- positive values such as `tabindex="3"` create a custom focus order and should generally be avoided because they easily diverge from visual and DOM order.

Focus should also remain visible. Removing the browser's focus outline without providing an equivalent replacement makes keyboard navigation difficult or impossible to follow.

Dynamic applications create another problem. Rendering new content does not automatically tell the user where attention should move. Opening a modal, changing routes in a client-rendered application, reporting validation errors, or restoring content after an asynchronous operation may require deliberate focus management.

Focus is state

Focus has a current owner, can move, and affects what keyboard input means. Treating it as accidental browser behaviour leads to interfaces that look correct but become disorienting when operated without a pointer.

22.6 ARIA supplements semantics

WAI-ARIA provides roles, states, and properties that expose additional semantics to assistive technologies. It is especially important for dynamic widgets and states that HTML alone cannot express clearly. ARIA does not change visual appearance and does not automatically implement keyboard interaction.[129]

The first rule is therefore simple. Prefer a native semantic HTML element when one exists. Use ARIA to supplement semantics, not to replace native behaviour unnecessarily.

ARIA describing state on a disclosure control

```
<button
  type="button"
  aria-expanded="false"
  aria-controls="advanced-options"
>
  Advanced options
</button>

<section id="advanced-options" hidden>
  <!-- additional controls -->
</section>
```

If JavaScript expands the section, it must keep `aria-expanded` consistent with the visible state. An accessibility attribute that becomes stale is duplicated state in another form. The visual interface says one thing while the semantic interface says another.

22.6.1 Live regions for dynamic updates

Single-page applications frequently change content without a page navigation. A sighted user may see a status message appear, but a screen-reader user may receive no indication unless the update is exposed appropriately. ARIA live regions provide a mechanism for announcing relevant dynamic changes.[130]

A status message that can be announced

```
<p role="status" aria-live="polite">
  Three results found.
</p>
```

Live regions should be used selectively. Announcing every small state change can overwhelm the user and make the interface harder to follow.

22.7 Colour, contrast, and non-colour cues

Colour is useful for grouping and emphasis, but it should not be the only carrier of meaning. A validation state that changes only from a green border to a red border is ambiguous for users with some forms of colour-vision deficiency and may be unclear even to users with typical vision. Add text, an icon with an accessible label, or another structural cue.

WCAG 2.2 Level AA requires a contrast ratio of at least 4.5:1 for normal text and 3:1 for qualifying large text. The ratio is calculated from the relative luminance of the foreground and background colours.[131]

A useful engineering consequence follows. Contrast is measurable. Do not estimate compliance by eye. Browser developer tools and accessibility audit tools can calculate contrast for the actual rendered colours.

Do not encode meaning in colour alone

Colour can reinforce a state, category, or warning, but the same information should remain available through text, shape, position, or another programmatically available cue.

22.8 Accessible forms

Forms combine labels, instructions, validation, error recovery, focus, and dynamic updates, which makes them one of the most important places to apply accessibility systematically.[132]

Every form control needs a clear purpose and an associated label. When possible, use the native `label` element with matching `for` and `id` attributes.[133]

Explicitly associated form labels

```
<label for="email">Email address</label>
<input
  id="email"
  name="email"
  type="email"
  autocomplete="email"
  required
>
```

Placeholder text is not a reliable substitute for a label because it disappears as the user types and may not be exposed consistently as the control's name. Related controls can be grouped with `fieldset` and `legend`.

Validation should explain both *that* something is wrong and *how to correct it*. Client-side validation improves feedback speed but is not a security boundary. Server-side validation remains necessary.[134]

Connecting a field to its error message

```
<label for="project-name">Project name</label>
<input
  id="project-name"
```

```
  name="project-name"
  aria-describedby="project-name-error"
  aria-invalid="true"
>
<p id="project-name-error" role="alert">
  Enter a project name with at least three characters.
</p>
```

In a React application, the same principle applies. State controls what is rendered, but the rendered result must include the semantic relationship between the field and its feedback.

22.9 Accessibility in dynamic React interfaces

React does not make an interface accessible or inaccessible by itself. The resulting DOM, semantics, keyboard behaviour, focus transitions, and announcements determine accessibility.

Several React patterns deserve special attention:

- **Conditional rendering** can add or remove the currently focused element. Decide where focus should move next.
- **Client-side routing** changes substantial page content without a traditional browser navigation. Update the document title and consider moving focus to the new main heading or content region when appropriate.
- **Asynchronous loading** needs meaningful loading, empty, success, and error states. Important status changes may require a live region.
- **Custom components** should preserve native semantics. A reusable `Button` component should normally render an actual `button`, not a clickable `div`.
- **Visual state and ARIA state** must be derived from the same source of truth so that they cannot drift apart.

A React component keeping visual and semantic state aligned

```tsx
type DisclosureProps = {
  open: boolean;
  onToggle: () => void;
};

function Disclosure({ open, onToggle }: DisclosureProps) {
  return (
    <>
      <button
        type="button"
        aria-expanded={open}
        aria-controls="details"
        onClick={onToggle}
      >
        Details
      </button>

      {open && <section id="details">Additional information</
    section>}
    </>
  );
}
```

Because `aria-expanded` and the conditional content are both calculated from `open`, the semantic and visual representations share one source of truth.

22.10 Testing accessibility

Automated checks are valuable for detecting missing labels, some invalid ARIA relationships, contrast problems, and other machine-detectable defects. They cannot determine whether alternative text communicates the right information, whether a workflow is understandable, or whether keyboard focus moves logically through a complex interaction. Comparative research on accessibility evaluation tools has likewise documented the harm of relying on automated testing alone, including limitations in coverage and correctness.[135]

A practical review combines several techniques:

- operate the complete workflow with the keyboard only.
- inspect focus visibility and order.
- inspect the accessibility tree and computed accessible names in browser developer tools.
- run automated accessibility audits as a regression check.
- use at least one screen reader to inspect important workflows.
- test zoom, text scaling, narrow viewports, and different contrast conditions.
- review forms, errors, modal interactions, route changes, and dynamic updates explicitly.

The purpose is not to chase an automated score. It is to verify that the application exposes a coherent interface through multiple modes of perception and interaction.

22.11 Chapter summary

What to remember about web accessibility

- Accessibility is a property of the complete interface, not an optional visual enhancement added at the end.
- WCAG organises accessibility around the principles Perceivable, Operable, Understandable, and Robust.
- Browsers expose a semantic accessibility representation to assistive technologies. Visually identical DOMs can therefore differ significantly in accessibility.
- Prefer native semantic HTML because it provides meaning, keyboard behaviour, focusability, and states with less custom code.
- Accessible names and text alternatives should describe purpose and information in context.
- Keyboard operation, visible focus, and deliberate focus management are essential for dynamic applications.
- ARIA supplements native semantics. It does not create missing interaction behaviour automatically.
- Dynamic status changes may need live regions so that assistive technologies can announce them.

- Use sufficient contrast and never make colour the only carrier of meaning.
- Forms need labels, instructions, accessible validation feedback, and server-side validation for security.
- Automated audits are useful but incomplete. Accessibility testing also requires keyboard, assistive-technology, and human workflow evaluation.

WebSockets and Multiuser Applications

23 WebSocket Foundations

Many web applications need to show changes soon after they occur. A new chat message, another user's cursor, a changed status, a newly assigned task, or a live measurement. Ordinary HTTP remains an excellent protocol for loading and changing resources, but its request–response interaction model does not by itself give the server an open channel through which it can send an event at an arbitrary later time. Real-time applications therefore add a communication mechanism that can remain active beyond one request.

The WebSocket protocol provides a persistent, bidirectional communication channel between a client and a server. After an opening handshake, either peer can send messages while the connection remains open. This makes WebSockets a natural transport for chat, games, dashboards, notifications, and multiuser applications. It does not, however, define the application's messages, state model, conflict policy, authentication rules, or recovery behaviour. Those are architectural responsibilities built on top of the transport.[136, 137]

23.1 HTTP request–response and persistent connections

A typical HTTP interaction begins when a client sends a request. The server produces a response, and that application-level exchange is complete. Modern HTTP implementations may reuse an underlying network connection for several requests, but the application interaction still remains request initiated. The client asks, and the server responds.

Loading a resource with HTTP

```
const response = await fetch("/api/evidence");

if (!response.ok) {
  throw new Error(‘Request failed: ${response.status}‘);
}

const evidence = await response.json();
renderEvidence(evidence);
```

The server cannot later reuse this completed `fetch` operation to announce that another user changed an item. The browser must initiate another request, or the application must establish a separate mechanism for server-originated updates.

A WebSocket interaction has a different lifetime. The client opens one logical connection and both peers may exchange messages until one side closes it or the network path fails.

Opening a persistent WebSocket connection

```
const socket = new WebSocket("wss://example.test/realtime");

socket.addEventListener("open", () => {
  socket.send(JSON.stringify({ type: "session.join" }));
});

socket.addEventListener("message", (event) => {
```

```
  const message = JSON.parse(event.data);
  handleServerMessage(message);
});
```

Property	HTTP request–response	WebSocket connection
Initiation	The client initiates each application request.	The client establishes the connection; afterwards either peer may send.
Lifetime	One request followed by one response.	Persistent until closed or interrupted.
Server-originated updates	Require another technique, such as polling, long polling, or server-sent events.	Can be sent immediately over the open connection.
Message semantics	Commonly expressed as methods and resources.	Defined by an application-specific message protocol.
Typical strengths	Resource loading, caching, commands, stateless APIs.	Low-latency bidirectional events and multiuser coordination.

Table 23.1: HTTP requests and WebSocket connections serve different interaction patterns

Real-time does not mean replacing HTTP

A real-time application commonly uses both protocols. HTTP loads durable resources, submits conventional commands, and benefits from established caching semantics. WebSockets carry events whose value depends on an already-open, low-latency channel. Choosing WebSockets does not make REST endpoints obsolete.

23.2 Polling, long polling, server-sent events, and WebSockets

WebSockets are not the only way to learn about server-side changes. The correct choice depends on the direction, frequency, and latency requirements of the communication.

23.2.1 Polling

With polling, the client sends a request repeatedly, regardless of whether anything changed:

A simple polling loop

```
async function pollWorkspace(): Promise<void> {
  const response = await fetch("/api/workspace");
  const snapshot = await response.json();
  applySnapshot(snapshot);
}

const pollingId = window.setInterval(pollWorkspace, 2_000);
```

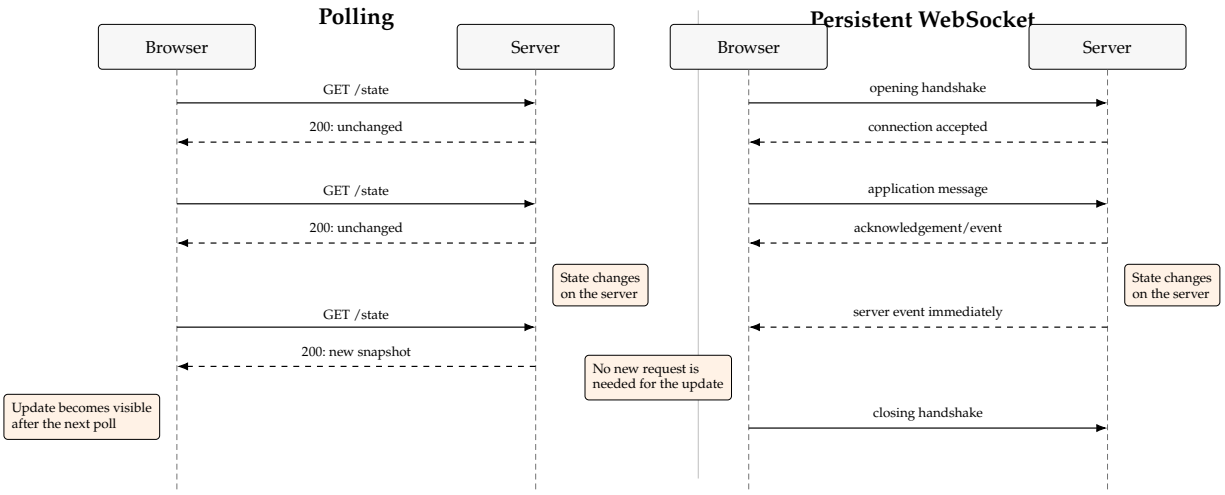


Figure 23.1: Polling repeatedly creates request–response exchanges, including exchanges in which no state changed. A WebSocket establishes one persistent channel over which either peer can send an application event.

If the interval is two seconds, a change may remain invisible for almost two seconds. Shorter intervals reduce that delay but increase the number of empty requests. With many connected users, repeated headers, authentication checks, routing, and response processing create work even when the application state is unchanged.

23.2.2 Long polling

Long polling keeps one HTTP request open until the server has data or a timeout is reached. After the response arrives, the client immediately starts another request. It reduces empty responses compared with fixed-interval polling, but still repeatedly creates request–response cycles and requires careful timeout and reconnection handling.

23.2.3 Server-sent events

Server-sent events provide a persistent server-to-client stream through the `EventSource` API. They are well suited to feeds and notifications where only the server needs to push updates. Client-to-server commands still use ordinary HTTP requests.

23.2.4 WebSockets

WebSockets provide bidirectional messages over one persistent connection. They are especially useful when both client and server frequently initiate events, such as a collaborative workspace in which user actions are propagated to the server and then broadcast to other participants. Early engineering literature on WebSocket highlighted this persistent bidirectional model as a way to communicate and display real-time data without repeated HTTP request–response cycles.[138]

Select the simplest transport that fits

Do not choose WebSockets only because an application contains changing data. Polling may be sufficient for an infrequently re-

freshed status page, and server-sent events may be simpler for a one-way event stream. WebSockets become valuable when bidirectional, low-latency interaction is a central requirement. Large-scale measurement of the real-time web has found that WebSocket adoption coexists with polling and other mechanisms, which supports treating transport selection as an architectural choice rather than an automatic default.[13]

23.3 The opening handshake

A WebSocket connection begins with a handshake that is integrated with the web platform. In the common HTTP/1.1 form, the browser sends an HTTP request asking the server to upgrade the connection to the WebSocket protocol. The request contains WebSocket-specific headers, including a random key. A compatible server validates the request and responds with status `101 Switching Protocols` and a derived acceptance value. After this exchange, the peers send WebSocket frames rather than ordinary HTTP messages over that connection.[136]

The simplified request looks like this:

Conceptual HTTP/1.1 WebSocket upgrade request

```
GET /realtime HTTP/1.1
Host: example.test
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: https://example.test
```

A successful response has the following shape:

Conceptual upgrade response

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

The handshake allows existing web infrastructure to participate in connection establishment and gives the server an opportunity to reject unsupported protocols, untrusted origins, invalid credentials, or excessive connection attempts before the application channel opens.

The upgrade diagram is the HTTP/1.1 teaching model

The familiar `Upgrade: websocket` and `101` exchange describes the HTTP/1.1 opening handshake. WebSockets can also be bootstrapped over newer HTTP versions through extended connection mechanisms. The application-level result is still an open WebSocket channel. The exact transport negotiation is normally handled by browsers, servers, proxies, and deployment infrastructure.[139]

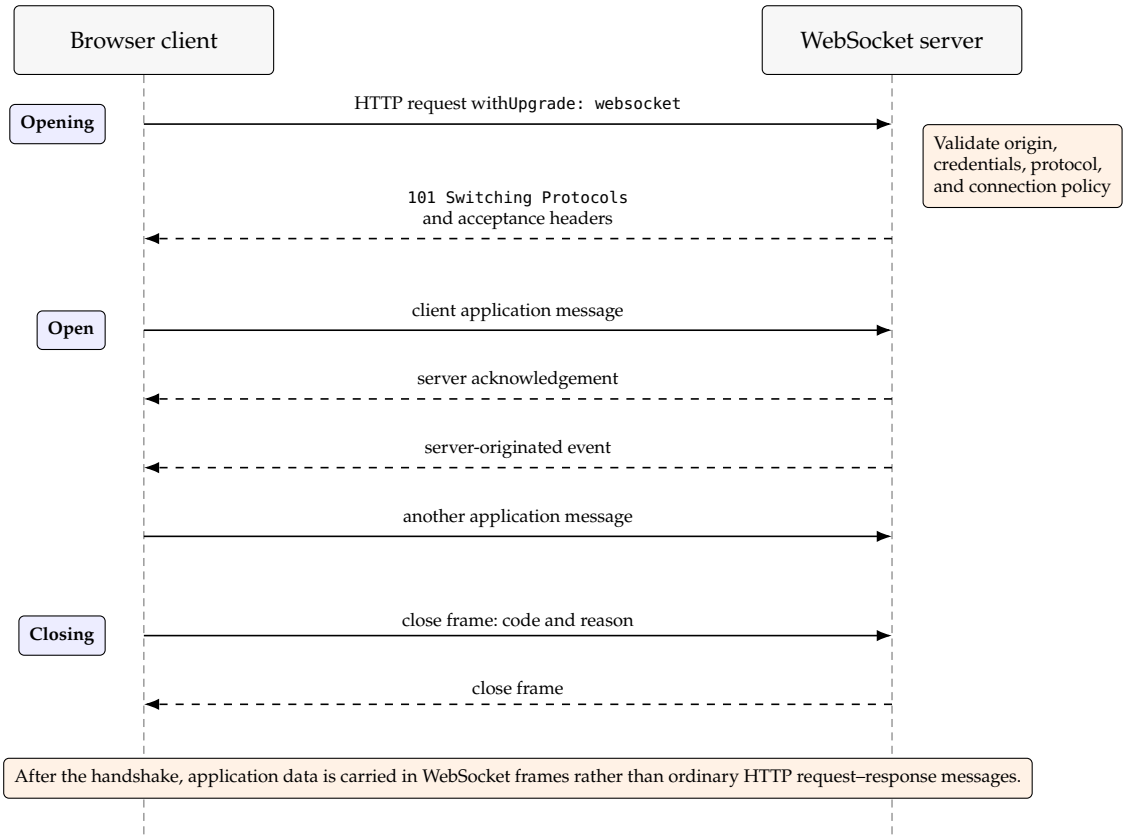


Figure 23.2: Simplified WebSocket lifecycle using the familiar HTTP/1.1 upgrade handshake. The opening request is still an opportunity to accept or reject the connection before bidirectional application messaging begins.

23.4 The browser WebSocket lifecycle

The browser exposes the connection through the `WebSocket` interface. Its `readyState` property moves through four states:

Client code normally observes four events. `open`, `message`, `error`, and `close`.[140]

Observing the complete client lifecycle

```
const socket = new WebSocket("wss://example.test/realtime");

socket.addEventListener("open", () => {
  console.log("WebSocket open");
});

socket.addEventListener("message", (event) => {
  console.log("Received", event.data);
});

socket.addEventListener("error", () => {
  console.error("WebSocket error");
});

socket.addEventListener("close", (event) => {
  console.log("Closed", {
    code: event.code,
    reason: event.reason,
    clean: event.wasClean
  });
});
```

State	Value	Meaning	Permitted application behaviour
`CONNECTING`	0	The opening handshake is in progress.	Wait; application messages cannot yet be sent safely.
`OPEN`	1	The connection is established.	Send and receive application messages.
`CLOSING`	2	A closing handshake has started.	Stop creating new messages and finish cleanup.
`CLOSED`	3	The connection is closed or could not be opened.	Reconnect when appropriate or show an offline state.

Table 23.2: States of a browser WebSocket object

The `error` event indicates failure but intentionally exposes little diagnostic detail to browser code. Application recovery should therefore be centred on connection state and the later `close` event rather than on trying to infer every network cause from `error`.

23.5 Opening, messaging, and closing

A complete connection has three conceptual phases.

23.5.1 Opening

The client creates the `WebSocket`. The browser performs the handshake. the application waits for `open`. Messages produced before the connection is open must either be rejected, buffered by application code, or represented as local pending operations.

23.5.2 Messaging

WebSocket frames can carry text or binary data. Many web applications define a JSON message protocol because JSON is easy to inspect and maps naturally to JavaScript values:

Sending one application message

```
const message = {
  type: "bookmark.set",
  evidenceId: "e-17",
  bookmarked: true
};

if (socket.readyState === WebSocket.OPEN) {
  socket.send(JSON.stringify(message));
}
```

The transport only carries bytes. It does not know whether this message is a bookmark change, a chat message, a document update, or an invalid object. The application protocol introduced in Chapter 24 defines that meaning.

23.5.3 Closing

Either side may start a closing handshake. A normal client closure can include a close code and a concise reason:

Closing a connection deliberately

```
socket.close(1000, "Page is leaving the live workspace");
```

A graceful close differs from an abrupt loss of connectivity. In both cases the browser eventually reports `close`, but the close code and `wasClean` flag can help the application distinguish a normal shutdown from a connection that disappeared unexpectedly. Previously queued outgoing data is not automatically discarded when `close()` begins the closing handshake.[141]

A connected socket is not a complete feature

A successful `open` event proves only that a transport channel exists. It does not prove that the user is authorised for every message, that incoming JSON has the expected shape, that state is current, or that recovery after a disconnection will be correct.

23.6 Client–server event flow

A useful real-time architecture separates four steps:

1. a user action creates an application event on one client.
2. the client serialises and sends a protocol message.
3. the server validates the message and applies an authoritative state transition.
4. the server acknowledges or broadcasts the accepted result to relevant clients.

For a bookmark toggle, the sender should not directly send rendered HTML or instructions such as “turn the third card blue.” It should send domain intent or a state transition identified by stable data:

Domain-level client intent

```
{
  "type": "bookmark.set",
  "sessionId": "workspace-main",
  "evidenceId": "e-17",
  "bookmarked": true,
  "operationId": "op-42"
}
```

The server verifies the session, evidence identifier, message fields, and permission to perform the action. It then changes the shared state and emits an accepted event or updated snapshot. Every client renders from that state using its own component tree.

Send domain events, not DOM commands

A real-time protocol should describe what happened in the application’s domain. “bookmark e-17”, “participant joined”, or “note version 8 was accepted.” It should not encode browser-specific rendering operations. This keeps the server independent of one UI implementation and gives clients a clear state transition to apply.

23.7 Chapter summary

What to remember about WebSocket foundations

- HTTP and WebSockets solve different interaction problems and are commonly used together.
- Polling repeatedly asks for changes. A WebSocket keeps a bidirectional channel open.
- A WebSocket begins with a web-compatible opening handshake and then carries framed messages.
- Browser clients observe `open`, `message`, `error`, and `close` across four connection states.
- The WebSocket transport carries data but does not define the application's protocol, state ownership, validation, or conflict policy.

24 Designing a Multiuser Message Architecture

A WebSocket connection gives two peers a channel, but a multiuser application needs more than a channel. It needs a vocabulary of messages, rules for who receives them, a model of shared state, and a decision about which participant is authoritative when concurrent actions disagree. These choices form an application-level protocol.

This chapter develops a small but extensible architecture for a shared workspace. The examples use bookmarks, notes, and a draft document because they illustrate three increasingly difficult forms of shared state. A set-like choice, independently versioned text, and a multi-field object. The same principles apply to games, dashboards, collaborative forms, and live operations tools.

24.1 Protocol before implementation

It is tempting to begin by calling `socket.send()` from event handlers and adding cases to a server callback whenever a new feature appears. That approach quickly creates an undocumented protocol in which message fields are optional, client and server assumptions drift apart, and errors are discovered only at runtime.

A better starting point is to define the protocol as a set of message types. Each message should answer at least four questions:

1. What kind of event or command is this?
2. Which session or document does it concern?
3. Which data is required to process it?
4. How can the sender and receiver identify its version or operation?

24.2 Typed messages with discriminated unions

TypeScript discriminated unions are well suited to JSON protocols. Every variant has a stable discriminator, usually called `type`, and that value determines the remaining fields.

A typed client-to-server protocol

```ts
type SessionId = string;
type DocumentId = string;
type OperationId = string;

type ClientMessage =
  | {
      type: "session.join";
      sessionId: SessionId;
    }
  | {
      type: "bookmark.set";
      sessionId: SessionId;
      evidenceId: string;
      bookmarked: boolean;
      operationId: OperationId;
    }
  | {
```

```
      type: "note.editing";
      sessionId: SessionId;
      documentId: DocumentId;
      editing: boolean;
    }
  | {
      type: "note.save";
      sessionId: SessionId;
      documentId: DocumentId;
      baseVersion: number;
      text: string;
      operationId: OperationId;
    }
  | {
      type: "hypothesis.update";
      sessionId: SessionId;
      documentId: DocumentId;
      baseVersion: number;
      patch: Partial<HypothesisDraft>;
      operationId: OperationId;
    };
```

The server-to-client protocol is related but not necessarily identical. The server may assign identity, send initial snapshots, acknowledge operations, or report conflicts:

A typed server-to-client protocol

```
type ServerMessage =
  | {
      type: "session.welcome";
      clientId: string;
      displayName: string;
      snapshot: WorkspaceSnapshot;
    }
  | {
      type: "presence.changed";
      participants: Participant[];
    }
  | {
      type: "bookmark.changed";
      evidenceId: string;
      bookmarked: boolean;
      operationId: OperationId;
      sequence: number;
    }
  | {
      type: "document.accepted";
      documentId: DocumentId;
      version: number;
      value: unknown;
      operationId: OperationId;
    }
  | {
      type: "document.conflict";
      documentId: DocumentId;
      currentVersion: number;
      currentValue: unknown;
      rejectedOperationId: OperationId;
    }
  | {
      type: "protocol.error";
      code: string;
      message: string;
```

```
    };
```

A switch statement can then be checked exhaustively:

Exhaustive message handling

```
function handleServerMessage(message: ServerMessage): void {
  switch (message.type) {
    case "session.welcome":
      initialiseSession(message);
      return;

    case "presence.changed":
      updatePresence(message.participants);
      return;

    case "bookmark.changed":
      applyBookmarkEvent(message);
      return;

    case "document.accepted":
      acceptDocumentVersion(message);
      return;

    case "document.conflict":
      showConflict(message);
      return;

    case "protocol.error":
      showProtocolError(message);
      return;

    default: {
      const unreachable: never = message;
      return unreachable;
    }
  }
}
```

Types document trusted code, not network data

A TypeScript type can ensure that code constructs and handles known message variants consistently. It cannot prove that bytes received from the network match that type. Incoming JSON must still be parsed and validated at runtime before it is treated as a `ClientMessage` or `ServerMessage`.

24.3 Sharing protocol definitions

When browser and server are built from the same TypeScript repository, protocol types should live in a shared module rather than being copied into two entry points:

One possible source layout

```
src/
  client/
    realtime/
      connection.ts
  server/
    realtime/
      server.ts
```

```
  shared/
    protocol.ts
    domain.ts
```

Both sides import from `shared/protocol.ts`. This provides one compile-time source of truth for names and shapes. The runtime validator should ideally be shared as well, so that the accepted wire format cannot drift away from the TypeScript declarations.

24.4 Runtime validation at the boundary

Every incoming message crosses a trust boundary. `JSON.parse()` may return any JavaScript value, including `null`, arrays, deeply nested objects, or an object with misleading fields. A safe server first parses, then validates, then authorises, and only then mutates shared state.

A narrow manual validator for one message type

```ts
type BookmarkSetMessage = Extract<
  ClientMessage,
  { type: "bookmark.set" }
>;

function isBookmarkSetMessage(
  value: unknown
): value is BookmarkSetMessage {
  if (typeof value !== "object" || value === null) {
    return false;
  }

  const candidate = value as Record<string, unknown>;

  return (
    candidate.type === "bookmark.set" &&
    typeof candidate.sessionId === "string" &&
    typeof candidate.evidenceId === "string" &&
    typeof candidate.bookmarked === "boolean" &&
    typeof candidate.operationId === "string"
  );
}
```

For a larger protocol, a schema-validation library can reduce repetitive code and derive TypeScript types from runtime schemas or vice versa. The architectural rule remains the same. A cast such as `JSON.parse(data) as ClientMessage` does not validate anything.

One object with many optional fields is not a protocol

A message type such as `\{ type. String. Text?. String. Id?. String. version?. Number. ... \}` allows combinations that are meaningless. A `type` discriminator plus variant-specific required fields makes valid states easier to express and invalid states easier to reject.

24.5 Sessions, rooms, and documents

A WebSocket server commonly manages several scopes.

Connection One network channel to one client instance.
Participant An application identity associated with a connection.
Session or room A group of participants who should receive the same events.
Document One independently synchronised unit of state inside a session.

Even an application with one visible shared workspace benefits from explicit session scoping. Broadcasting through a session abstraction documents the intended boundary and prevents unrelated future connections from accidentally receiving every message.

Server-side session and document structures

```
type VersionedDocument<T> = {
  version: number;
  value: T;
};

type Session = {
  id: SessionId;
  clients: Set<ConnectedClient>;
  sequence: number;
  bookmarks: VersionedDocument<Set<string>>;
  hypothesis: VersionedDocument<HypothesisDraft>;
  notes: Map<DocumentId, VersionedDocument<string>>;
};

const sessions = new Map<SessionId, Session>();
```

A session-scoped broadcast iterates over clients in one session rather than all open sockets:

Broadcasting within one session

```
function broadcast(
  session: Session,
  message: ServerMessage
): void {
  const encoded = JSON.stringify(message);

  for (const client of session.clients) {
    if (client.socket.readyState === client.socket.OPEN) {
      client.socket.send(encoded);
    }
  }
}
```

The bookmark set, hypothesis draft, and each note are separate documents because they can change independently. Editing one note should not invalidate a user who is saving a different note. Smaller version domains reduce false conflicts and make snapshots, permissions, and recovery more precise.

Choose the conflict domain deliberately

A version number protects one unit of concurrency. If an entire workspace has one version, every change conflicts with every other change. If each field has its own version, conflicts are precise but the protocol becomes more complex. A document boundary is therefore an architectural trade-off, not merely a data structure choice.

24.6 Presence is derived, temporary state

Presence answers questions such as “who is online?” or “who is currently editing this document?” It is not the same as durable user data. Presence is usually derived from live connections and expires when the server decides a participant is no longer active.

A server may assign a temporary identity when a connection joins:

Assigning connection-scoped identity

```ts
function createParticipant(): Participant {
  return {
    clientId: crypto.randomUUID(),
    displayName: randomDisplayName(),
    connectedAt: Date.now()
  };
}
```

The server stores the participant with the connection and includes the current participant list in a welcome or presence message. Clients mirror that list in UI state, but the authoritative presence set remains on the server.

A soft editing indicator is another form of presence:

An ephemeral editing signal

```ts
type EditingMessage = {
  type: "note.editing";
  sessionId: SessionId;
  documentId: DocumentId;
  editing: boolean;
};
```

The indicator should expire after a timeout or when the connection disappears. Otherwise a crashed tab could leave a permanent “editing” label. This is why soft presence is generally safer than an enforced lock for short-form collaboration.

Disconnected and absent are related but not identical

A close event is evidence that one connection ended. Presence is an application judgement. A user may have several tabs, a mobile and desktop connection, or a connection that is temporarily unreachable. Applications must decide whether presence is per connection, per account, or per active session, and how quickly uncertain connections expire.

24.7 Authoritative state and event flow

A simple multiuser system usually treats the server as authoritative. Clients send intent. The server decides whether it is valid, applies it to the current state, increments versions or sequence numbers, and broadcasts the accepted result.

For set-like bookmark state, the flow can be:

1. the user toggles an item.
2. the client sends `bookmark.set`.
3. the server validates the item and session.
4. the server adds or removes the stable identifier from the set.

5. the server increments the session sequence.
6. the server broadcasts `bookmark.changed` to the room.

Sending the desired final state, `bookmarked: true`, is often safer than sending only `toggle`. If a toggle message is retried accidentally, the same operation could undo itself. Setting an explicit value is naturally idempotent. Applying it twice has the same result as applying it once.

24.8 Optimistic user interfaces

Waiting for a complete network round trip before changing the screen can make a responsive interface feel slow. An optimistic UI applies the expected result locally as soon as the user acts, sends the operation, and later reconciles the local state with the server response.

Conceptual optimistic bookmark update

```
function setBookmarkOptimistically(
  evidenceId: string,
  bookmarked: boolean
): void {
  const operationId = crypto.randomUUID();

  applyLocalBookmark(evidenceId, bookmarked);
  markPending(operationId);

  send({
    type: "bookmark.set",
    sessionId: currentSessionId,
    evidenceId,
    bookmarked,
    operationId
  });
}
```

The operation identifier lets the client correlate an acknowledgement or broadcast with its pending change. If the server rejects the operation, the client must either roll back, replace its state with the authoritative value, or show an explicit conflict. Optimism changes perceived latency, but it does not remove the need for server authority.

Optimistic does not mean unverified

An optimistic UI predicts success locally. The server still validates and accepts or rejects the operation. A robust client can explain what is pending, what was confirmed, and what changed during reconciliation.

24.9 Version-based conflict detection

Text and multi-field drafts are harder than set membership because two users can start from the same value and produce incompatible replacements. A practical introductory strategy is optimistic concurrency control with document versions. Concurrency control has been a central problem in groupware research since early shared-editing systems, where concurrent operations must preserve a coherent shared state without destroying users' work.[142]

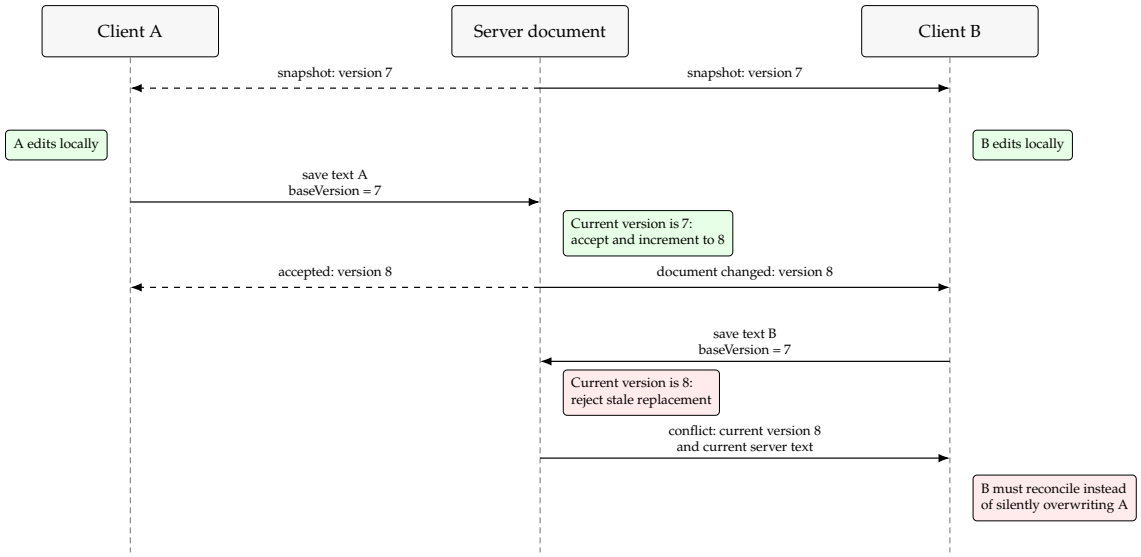


Figure 24.1: Version-based optimistic concurrency control. Both clients start from version 7. The first accepted save advances the server to version 8; the second save is detected as stale and receives the current server state.

The server stores:

A versioned note document

```ts
type NoteDocument = {
  version: number;
  text: string;
};
```

A client that began editing version 7 sends its replacement together with `baseVersion: 7`. The server accepts the save only if the current version is still 7:

Detecting a stale save on the server

```ts
function saveNote(
  document: NoteDocument,
  message: NoteSaveMessage
): SaveResult {
  if (message.baseVersion !== document.version) {
    return {
      status: "conflict",
      currentVersion: document.version,
      currentText: document.text
    };
  }

  document.text = message.text;
  document.version += 1;

  return {
    status: "accepted",
    version: document.version,
    text: document.text
  };
}
```

A conflict response should include enough current state for the user to make an informed decision. For a short note, options may include discarding the local edit, copying it elsewhere, or deliberately replacing the newer version. A silent last-write-wins rule is simple but can destroy another user's work without warning.

24.9.1 Whole-document and field-level versions

A whole-form version treats any concurrent change as a conflict. This is easy to explain and implement, but two users changing unrelated fields still interfere. Field-level versions reduce those false conflicts but require more protocol variants, more metadata, and clearer UI reconciliation.

There is no universally correct granularity. Short forms often tolerate a whole-document version. Long or highly concurrent documents eventually require operation-based merging approaches such as operational transformation or conflict-free replicated data types. A classic account of operational transformation describes the issues and algorithms involved in preserving consistency in real-time group editors.[14] Those approaches are outside the basic version-checking model presented here.

24.10 Basic conflict-handling strategies

Several simple policies recur in multiuser systems:

Reject stale writes Compare a base version and require the user or client to reconcile.

Last write wins Accept the latest timestamp or server order. Simple, but may silently lose intent.

Field-level merge Combine changes to independent fields while flagging overlapping edits.

Soft coordination Show who is editing and reduce the probability of a conflict without enforcing a lock.

Hard locking Allow one editor at a time. Requires leases, expiry, crash recovery, and permission rules.

For small shared notes, a useful compromise is soft editing presence plus version-based save rejection and an explicit warning. It reveals conflicts without pretending to merge arbitrary text.

24.11 Chapter summary

What to remember about multiuser architecture

- A typed, discriminated protocol makes message variants and responsibilities explicit.
- Network data remains untrusted even when client and server share TypeScript types.
- Sessions scope recipients. Documents scope versions and conflicts.
- Presence is ephemeral state derived from connections and should expire safely.
- Server-authoritative event flow validates intent before broadcasting accepted state.
- Optimistic UI improves responsiveness but must reconcile with server acknowledgements or conflicts.
- Version checks detect stale writes without claiming to solve general collaborative text merging.

25 Reliability and Security in Real-Time Applications

A real-time feature may work perfectly while two browser tabs and a development server are running on one machine, yet fail under ordinary production conditions. Networks pause without closing cleanly, laptops sleep, proxies time out idle connections, servers restart, clients reconnect with stale state, and malformed messages arrive outside the expected UI. Reliability and security must therefore be part of the protocol design rather than added after the happy path works. Dependability research provides a broader vocabulary for these concerns, including faults, errors, failures, availability, reliability, safety, integrity, and maintainability.[15]

25.1 Connection status is application state

A user should not have to infer connection health from whether a recent click appeared elsewhere. Represent connection status explicitly:

A small connection-state model

```ts
type ConnectionStatus =
  | { state: "connecting"; attempt: number }
  | { state: "online"; connectedAt: number }
  | { state: "reconnecting"; attempt: number }
  | { state: "offline"; reason?: string };
```

This state drives a visible indicator and determines whether new operations are sent immediately, queued locally, or disabled. It also prevents components from inventing inconsistent meanings for `open`, `error`, and `close`.

Transport state and application state differ

A socket can be open while the client has not yet joined a session or received an initial snapshot. Conversely, the UI may still display cached application state after the transport has closed. Model both concerns explicitly.

25.2 Reconnection with backoff

Immediately reconnecting in a tight loop can overload a server that is already recovering. A common policy uses exponential backoff. Each failed attempt waits longer, up to a maximum. Random jitter prevents thousands of clients from retrying at exactly the same moments.

Calculating a reconnect delay

```ts
function reconnectDelay(attempt: number): number {
  const base = 500;
  const maximum = 30_000;
  const exponential = Math.min(maximum, base * 2 ** attempt);
  const jitter = Math.random() * 0.3 * exponential;

  return exponential + jitter;
}
```

The attempt counter should reset after a connection remains healthy or after a successful session synchronisation, not merely when the TCP/WebSocket channel briefly opens.

A simplified reconnect loop

```
function scheduleReconnect(): void {
  const delay = reconnectDelay(reconnectAttempt);
  connectionStatus = {
    state: "reconnecting",
    attempt: reconnectAttempt
  };

  window.setTimeout(() => {
    reconnectAttempt += 1;
    connect();
  }, delay);
}
```

Applications should stop retrying when the user deliberately signed out, left the session, or received a non-retryable policy rejection.

25.3 Reconnect means resynchronise

A new connection is not a continuation of the old one. While the client was offline, other participants may have changed every shared document. The safest basic recovery strategy proceeds as follows:

1. establish a new WebSocket connection.
2. authenticate or establish temporary identity.
3. join the intended session.
4. receive an authoritative snapshot and current sequence/version values.
5. reconcile or discard local pending operations according to policy.
6. resume live event processing.

A client must not assume that its last rendered state remains current simply because the connection was restored quickly.

A welcome message that establishes a new baseline

```
type SessionWelcome = {
  type: "session.welcome";
  clientId: string;
  displayName: string;
  sessionSequence: number;
  snapshot: WorkspaceSnapshot;
};
```

The snapshot establishes a baseline. Events received afterwards must have a sequence greater than that baseline. More advanced systems may request only missed events, but that requires a durable event log and retention policy.

25.4 Heartbeats and half-open connections

A network path can fail without immediately producing a useful browser `close` event. For example, a device may lose connectivity, a NAT mapping may expire, or an intermediary may silently discard an idle connection.

A heartbeat creates periodic traffic and requires evidence that the peer is still reachable.

The WebSocket protocol defines ping and pong control frames. Node server libraries such as `ws` can send pings and observe pongs. Browser JavaScript does not expose protocol ping/pong frames through the standard `WebSocket` interface. The browser handles protocol-level pongs internally. Applications that need heartbeat events visible to browser code can additionally define application-level `heartbeat.ping` and `heartbeat.pong` messages.[137, 143]

Server-side heartbeat using the ws library

```
function markAlive(this: TrackedSocket): void {
  this.isAlive = true;
}

server.on("connection", (socket: TrackedSocket) => {
  socket.isAlive = true;
  socket.on("pong", markAlive);
});

const heartbeatId = setInterval(() => {
  for (const socket of server.clients as Set<TrackedSocket>) {
    if (!socket.isAlive) {
      socket.terminate();
      continue;
    }

    socket.isAlive = false;
    socket.ping();
  }
}, 30_000);
```

Heartbeat intervals must be chosen with infrastructure timeouts and network cost in mind. Too frequent wastes power and bandwidth. Too slow delays failure detection. A missed heartbeat should normally trigger connection cleanup and the existing reconnection path rather than inventing a second recovery system.

An application message is not a protocol ping frame

A JSON message such as `\{ type: "heartbeat.ping" \}` can support application-visible liveness and latency measurements, but it is ordinary data. Do not confuse it with the WebSocket protocol's control frames, which browsers handle below the JavaScript API.

25.5 Ordering within and across connections

WebSocket messages on one healthy connection are carried over an ordered transport. If one endpoint sends message A and then message B on that connection, the receiver does not normally observe B before A. This guarantee does not eliminate application-level ordering problems.

Problems arise when:

- a client reconnects and retries an operation that the server already accepted.
- an old snapshot is applied after a newer event because asynchronous handlers complete in a different order.

- separate server tasks produce results concurrently.
- several server instances broadcast through an external broker.
- pending offline operations are replayed against a newer baseline.

A session sequence gives accepted server events a total order:

Applying only the next session event

```
function applySequencedEvent(event: SequencedEvent): void {
  if (event.sequence <= lastAppliedSequence) {
    return; // duplicate or older event
  }

  if (event.sequence !== lastAppliedSequence + 1) {
    requestResynchronisation();
    return;
  }

  applyEvent(event);
  lastAppliedSequence = event.sequence;
}
```

Per-document versions answer whether a write was based on the current value of one document. Session sequences order accepted events. document versions detect stale edits. Many applications need both.

25.6 Duplicate messages and idempotency

TCP does not ordinarily duplicate bytes within one connection, but application operations may be resent after uncertainty. Suppose a client sends an update, loses the connection before seeing the acknowledgement, and retries after reconnecting. The server may receive the logical operation twice.

Every client operation can include a unique `operationId`. The server keeps a bounded record of recently processed identifiers per session or participant:

Deduplicating retried operations

```
function processOperation(
  session: Session,
  message: OperationMessage
): ServerMessage {
  const previous = session.processed.get(message.operationId);

  if (previous) {
    return previous;
  }

  const result = applyValidatedOperation(session, message);
  session.processed.set(message.operationId, result);
  return result;
}
```

The record must eventually expire or be bounded. Operations should also be idempotent where possible. “Set bookmark to true” is idempotent. “toggle bookmark” is not. Idempotence has also been studied as a systematic fault-tolerance technique because repeated execution can then be made observationally equivalent to a single successful operation.[144]

Sequence, version, and operation ID solve different problems

A sequence orders accepted events. A document version checks whether an edit was based on current state. An operation ID recognises a retry of the same logical command. Treating one of these values as a substitute for all three creates subtle recovery bugs.

25.7 Backpressure and overloaded clients

A persistent connection can produce data faster than a client or network can consume it. The browser's `bufferedAmount` reports bytes queued by the client for transmission. Server libraries offer their own buffering and stream controls. A system should define limits and decide whether to slow producers, coalesce replaceable events, drop low-value presence updates, or close an unhealthy connection.

For example, cursor positions may be safely replaced by the newest position, whereas document saves cannot be discarded. Real-time does not imply that every intermediate event has equal value.

25.8 Authentication begins during connection establishment

WebSockets do not define an application identity system. A production system must decide how the connection is associated with a user or service. Common patterns include an existing same-origin session cookie, a short-lived connection token, or an authentication message immediately after opening.

The opening handshake should validate the expected `Origin`. Browsers can include ambient cookies in a WebSocket handshake, so accepting any origin may allow a malicious site to open an authenticated connection on behalf of a logged-in user. An explicit origin allowlist is therefore an important browser security boundary. Production deployments should use `wss://` so that TLS protects confidentiality and integrity in transit.[145] More generally, empirical analysis of CSRF defences in web frameworks shows why application security cannot be inferred from framework defaults alone. protections vary and must be matched to the actual request and authentication model.[16]

Authentication and authorisation are separate

Authentication answers "who is this connection?" Authorisation answers "may this participant perform this action on this session or document?" A valid connection must not imply permission for every message type.

25.9 Validate and authorise every message

A server should not trust identifiers, display names, versions, or roles provided by the client. Connection-associated values such as the participant identity should come from server state, not from each message.

A secure processing pipeline proceeds as follows:

1. reject messages that exceed a size limit.
2. parse JSON safely.
3. validate the discriminated message shape and field constraints.
4. obtain identity and session membership from the connection context.
5. authorise the requested action for the target document.
6. apply domain validation and version checks.
7. mutate authoritative state.
8. emit an acknowledgement, event, or structured error.
9. log security-relevant metadata without exposing secrets.

Do not accept identity from the message body

```ts
function handleNoteSave(
  client: ConnectedClient,
  message: NoteSaveMessage
): void {
  const session = requireMembership(client, message.sessionId);
  const document = requireWritableNote(
    session,
    client.participantId,
    message.documentId
  );

  const result = saveNote(document, message);
  sendSaveResult(session, client, result);
}
```

Even an anonymous classroom application needs validation. A random display name is not strong authentication, but the server can still prevent spoofing by assigning it itself, restricting message sizes, checking known document IDs, and refusing malformed versions or fields.

The client UI is not a security control

A hidden button, a TypeScript type, or a disabled form field does not prevent a client from constructing arbitrary WebSocket data in the browser console or a custom program. Security decisions belong on the server-side boundary where messages are accepted.

25.10 Connection and message limits

Persistent connections create resource risks that ordinary short requests do not expose in the same way. Servers should consider:

- maximum connections per account, address, or session.
- maximum message size and nesting depth.
- rate limits per message type.
- timeouts for unauthenticated or idle connections.
- bounded presence, deduplication, and pending-operation records.
- limits on rooms and documents one connection may join.
- graceful rejection and close codes for policy violations.

These limits protect availability and make failure behaviour predictable. OWASP additionally recommends validating origin, using secure transport, authorising actions at message level, constraining payloads, and monitoring abnormal connection and validation events.[145]

25.11 Observability and testing

HTTP access logs record the opening handshake but do not automatically describe every later WebSocket message. Real-time systems need application-level observability:

- connection opened, authenticated, joined, closed, and expired.
- session and participant identifiers suitable for correlation.
- message type, size, validation result, and processing latency.
- sequence gaps, duplicate operation IDs, and conflicts.
- reconnect attempts and snapshot resynchronisations.
- heartbeat failures and forced terminations.

Do not log complete sensitive document contents or authentication tokens merely because message-level logging is useful.

Testing should include more than two healthy tabs. Deliberately stop and restart the server, delay handlers, send malformed JSON, duplicate operations, submit a stale version, open a connection from an unexpected origin in a controlled test, and verify that the client recovers from a fresh snapshot.

25.12 Chapter summary

What to remember about reliable and secure real-time systems

- Connection status must be visible application state rather than an implicit assumption.
- Reconnection should use backoff and must establish a new authoritative baseline.
- Heartbeats detect silent failures. Browser code may need application-level heartbeat messages in addition to protocol ping/pong.
- Ordering within one connection does not solve retries, stale snapshots, or reconnect races.
- Sequences, document versions, and operation IDs address different consistency problems.
- WebSocket connections require explicit origin checks, secure transport, authentication, message-level authorisation, validation, and resource limits.
- Real-time behaviour must be tested under disconnection, duplication, delay, malformed input, and server restart.

References

[1] Kyle Simpson. *You Don't Know JS Yet: Get Started*. 2nd ed. GetiPub and Leanpub, 2020 (cited on pages iii, 2, 10, 19, 22).

[2] John Resig, Bear Bibeault, and Josip Maras. *Secrets of the JavaScript Ninja*. 2nd ed. Manning, 2016 (cited on pages iii, 4, 7, 9, 13, 14, 16, 26, 37, 38).

[3] Nicolas Bevacqua. *JavaScript Application Design: A Build First Approach*. Manning, 2015 (cited on pages iii, 31, 40, 45–47).

[4] Tim Berners-Lee et al. 'The World-Wide Web'. In: *Communications of the ACM* 37.8 (1994), pp. 76–82. DOI: 10.1145/179606.179671 (cited on pages iii, 75).

[5] Ali Mesbah and Arie van Deursen. 'A Component- and Push-Based Architectural Style for AJAX Applications'. In: *Journal of Systems and Software* 81.12 (2008), pp. 2194–2209. DOI: 10.1016/j.jss.2008.04.005 (cited on pages iii, 77).

[6] Piero Fraternali et al. 'Engineering Rich Internet Applications with a Model-Driven Approach'. In: *ACM Transactions on the Web* 4.2 (2010), pp. 1–47. DOI: 10.1145/1734200.1734204 (cited on pages iii, 83).

[7] Richard N. Taylor et al. 'A Component- and Message-Based Architectural Style for GUI Software'. In: *IEEE Transactions on Software Engineering* 22.6 (1996), pp. 390–406. DOI: 10.1109/32.508313 (cited on pages iii, 96, 102).

[8] Conal Elliott and Paul Hudak. 'Functional Reactive Animation'. In: *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming*. Association for Computing Machinery, 1997, pp. 263–273. DOI: 10.1145/258948.258973 (cited on pages iii, 105).

[9] Xiao Sophia Wang et al. 'Demystifying Page Load Performance with WProf'. In: *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*. USENIX Association, 2013, pp. 473–485 (cited on pages iii, 86, 143).

[10] Aiden Bai. 'Million.js: A Fast Compiler-Augmented Virtual DOM for the Web'. In: *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC '23)*. Association for Computing Machinery, 2023, pp. 1813–1820. DOI: 10.1145/3555776.3577683 (cited on pages iii, 97).

[11] Ivano Malavolta et al. 'Evaluating the Impact of Caching on the Energy Consumption and Performance of Progressive Web Apps'. In: *Proceedings of the IEEE/ACM 7th International Conference on Mobile Software Engineering and Systems (MOBILESoft '20)*. Association for Computing Machinery, 2020, pp. 109–119. DOI: 10.1145/3387905.3388593 (cited on pages iii, 127).

[12] Christopher Power et al. 'Guidelines are Only Half of the Story: Accessibility Problems Encountered by Blind Users on the Web'. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2012, pp. 433–442. DOI: 10.1145/2207676.2207736 (cited on pages iii, 149).

[13] Paul Murley et al. 'WebSocket Adoption and the Landscape of the Real-Time Web'. In: *Proceedings of The Web Conference 2021*. Association for Computing Machinery, 2021, pp. 1192–1203. DOI: 10.1145/3442381.3450063 (cited on pages iii, 161).

[14] Chengzheng Sun and Clarence A. Ellis. 'Operational Transformation in Real-Time Group Editors: Issues, Algorithms, and Achievements'. In: *Proceedings of the 1998 ACM Conference on Computer Supported Cooperative Work*. Association for Computing Machinery, 1998, pp. 59–68. DOI: 10.1145/289444.289469 (cited on pages iii, 174).

[15] Algirdas Avizienis et al. 'Basic Concepts and Taxonomy of Dependable and Secure Computing'. In: *IEEE Transactions on Dependable and Secure Computing* 1.1 (2004), pp. 11–33. DOI: 10.1109/TDSC.2004.2 (cited on pages iii, 175).

[16] Xhelal Likaj, Soheil Khodayari, and Giancarlo Pellegrino. 'Where We Stand (or Fall): An Analysis of CSRF Defenses in Web Frameworks'. In: *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses*. Association for Computing Machinery, 2021, pp. 370–385. DOI: 10.1145/3471621.3471846 (cited on pages iii, 179).

[17] Mozilla Developer Network (MDN). *JavaScript Execution Model*. Mozilla Developer Network. Apr. 6, 2026. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model (visited on 08/06/2026) (cited on pages 4, 6, 7).

[18] Philip Roberts. *Loupe*. URL: https://latentflip.com/loupe/ (visited on 08/06/2026) (cited on page 5).

[19] OWASP Foundation. *Cross Site Scripting Prevention Cheat Sheet*. URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html (visited on 08/06/2026) (cited on page 27).

[20] Mozilla Developer Network. *JavaScript Modules*. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules (visited on 08/06/2026) (cited on page 31).

[21] Mozilla Developer Network. *Strict Mode*. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Strict_mode (visited on 08/06/2026) (cited on page 31).

[22] Mozilla Developer Network. *await*. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await (visited on 08/06/2026) (cited on page 39).

[23] Mozilla Developer Network. *Using Promises*. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises (visited on 08/06/2026) (cited on page 39).

[24] npm, Inc. *package.json Documentation*. URL: https://docs.npmjs.com/cli/configuring-npm/package-json/ (visited on 08/06/2026) (cited on pages 46, 48, 49).

[25] npm, Inc. *package-lock.json Documentation*. URL: https://docs.npmjs.com/cli/v11/configuring-npm/package-lock-json/ (visited on 08/06/2026) (cited on pages 49, 50).

[26] pnpm contributors. *pnpm: Fast, Disk Space Efficient Package Manager*. URL: https://pnpm.io/ (visited on 09/09/2026) (cited on page 50).

[27] Vite Team. *Vite: Getting Started*. URL: https://vite.dev/guide/ (visited on 08/06/2026) (cited on page 50).

[28] Vite Team. *Why Vite*. URL: https://vite.dev/guide/why.html (visited on 08/06/2026) (cited on page 50).

[29] Vite Team. *Building for Production*. URL: https://vite.dev/guide/build (visited on 08/06/2026) (cited on page 51).

[30] Prettier Contributors. *Prettier Documentation*. URL: https://prettier.io/docs/ (visited on 08/06/2026) (cited on page 53).

[31] OpenJS Foundation. *ESLint Documentation*. URL: https://eslint.org/docs/latest/ (visited on 08/06/2026) (cited on page 53).

[32] Microsoft. *The TypeScript Handbook*. URL: https://www.typescriptlang.org/docs/handbook/intro.html (visited on 08/06/2026) (cited on page 57).

[33] Microsoft. *TypeScript Handbook: Narrowing*. URL: https://www.typescriptlang.org/docs/handbook/2/narrowing.html (visited on 08/06/2026) (cited on page 60).

[34] Microsoft. *TypeScript Declaration Files: Do's and Don'ts*. URL: https://www.typescriptlang.org/docs/handbook/declaration-files/do-s-and-don-ts.html (visited on 08/06/2026) (cited on page 61).

[35] Microsoft. *TypeScript Handbook: Object Types*. URL: https://www.typescriptlang.org/docs/handbook/2/objects.html (visited on 08/06/2026) (cited on page 62).

[36] Microsoft. *TypeScript Handbook: Generics*. URL: https://www.typescriptlang.org/docs/handbook/2/generics.html (visited on 08/06/2026) (cited on page 63).

[37] Microsoft. *TypeScript Utility Types*. URL: https://www.typescriptlang.org/docs/handbook/utility-types.html (visited on 08/06/2026) (cited on page 64).

[38] Microsoft. *TypeScript Handbook: Type Declarations*. URL: https://www.typescriptlang.org/docs/handbook/2/type-declarations.html (visited on 08/06/2026) (cited on page 66).

[39] Microsoft. *TSConfig Reference*. URL: https://www.typescriptlang.org/tsconfig/ (visited on 08/06/2026) (cited on page 67).

[40] Microsoft. *TSConfig Option: strict*. URL: https://www.typescriptlang.org/tsconfig/strict (visited on 08/06/2026) (cited on page 67).

[41] GitHub. *Understanding GitHub Actions*. URL: https://docs.github.com/en/actions/get-started/understand-github-actions (visited on 08/06/2026) (cited on page 70).

[42] GitHub. *Dependency Caching*. URL: https://docs.github.com/en/actions/concepts/workflows-and-actions/dependency-caching (visited on 08/06/2026) (cited on page 71).

[43] GitHub. *Using Custom Workflows with GitHub Pages*. URL: https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages (visited on 08/06/2026) (cited on page 72).

[44] GitHub. *Workflow Syntax for GitHub Actions*. URL: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax (visited on 08/06/2026) (cited on page 72).

[45] Leon Freudenthaler. *Web Application Architectures*. Lecture slides, Hochschule Campus Wien. Advanced Web Engineering course material. 2026 (cited on page 75).

[46] Jesse James Garrett et al. 'Ajax: A new approach to web applications'. In: (2005) (cited on page 77).

[47] Ali Mesbah and Arie van Deursen. 'Migrating Multi-page Web Applications to Single-page AJAX Interfaces'. In: *11th European Conference on Software Maintenance and Reengineering (CSMR 2007)*. IEEE, 2007, pp. 181–190. DOI: 10.1109/CSMR.2007.33 (cited on page 81).

[48] Mozilla Developer Network. *History API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/History_API (visited on 08/06/2026) (cited on page 82).

[49] React Team. *hydrateRoot*. URL: https://react.dev/reference/react-dom/client/hydrateRoot (visited on 08/06/2026) (cited on page 86).

[50] Shaghayegh Mardani, Mayank Singh, and Ravi Netravali. 'Fawkes: Faster Mobile Page Loads via App-Inspired Static Templating'. In: *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, 2020, pp. 879–894 (cited on page 87).

[51] React Team. *prerender*. URL: https://react.dev/reference/react-dom/static/prerender (visited on 08/06/2026) (cited on page 88).

[52] Astro Technology Company. *Islands Architecture*. URL: https://docs.astro.build/en/concepts/islands/ (visited on 08/06/2026) (cited on page 88).

[53] Vercel. *Server and Client Components*. URL: https://nextjs.org/docs/app/getting-started/server-and-client-components (visited on 08/06/2026) (cited on page 89).

[54] Qwik Team. *Resumable Applications*. URL: https://qwik.dev/docs/concepts/resumable/ (visited on 08/06/2026) (cited on page 89).

[55] React Team. *Describing the UI*. URL: https://react.dev/learn/describing-the-ui (visited on 08/06/2026) (cited on pages 92, 94).

[56] React Team. *createRoot*. URL: https://react.dev/reference/react-dom/client/createRoot (visited on 08/06/2026) (cited on page 94).

[57] React Team. *Keeping Components Pure*. URL: https://react.dev/learn/keeping-components-pure (visited on 08/06/2026) (cited on page 95).

[58] React Team. *Passing Props to a Component*. URL: https://react.dev/learn/passing-props-to-a-component (visited on 08/06/2026) (cited on page 95).

[59] React Team. *Using TypeScript*. URL: https://react.dev/learn/typescript (visited on 08/06/2026) (cited on page 95).

[60] React Team. *Understanding Your UI as a Tree*. URL: https://react.dev/learn/understanding-your-ui-as-a-tree (visited on 08/06/2026) (cited on page 96).

[61] React Team. *Render and Commit*. URL: https://react.dev/learn/render-and-commit (visited on 08/06/2026) (cited on page 97).

[62] React Team. *Preserving and Resetting State*. URL: https://react.dev/learn/preserving-and-resetting-state (visited on 08/06/2026) (cited on pages 97, 106).

[63] React Team. *Conditional Rendering*. URL: https://react.dev/learn/conditional-rendering (visited on 08/06/2026) (cited on page 100).

[64] React Team. *Rendering Lists*. URL: https://react.dev/learn/rendering-lists (visited on 08/06/2026) (cited on page 101).

[65] React Router Team. *Routing*. URL: https://reactrouter.com/start/declarative/routing (visited on 08/06/2026) (cited on page 102).

[66] React Router Team. *URL Values*. URL: https://reactrouter.com/start/declarative/url-values (visited on 08/06/2026) (cited on page 102).

[67] TanStack Team. *TanStack Router: Overview*. URL: https://tanstack.com/router/latest/docs/overview (visited on 09/09/2026) (cited on page 102).

[68] Alexey Avdeev and Wouter contributors. *Wouter: A Minimalist-Friendly Router for React and Preact*. URL: https://github.com/molefrog/wouter (visited on 09/09/2026) (cited on page 102).

[69] Gerti Kappel et al., eds. *Web Engineering: The Discipline of Systematic Development of Web Applications*. John Wiley & Sons, 2006 (cited on page 102).

[70] React Team. *Responding to Events*. URL: https://react.dev/learn/responding-to-events (visited on 08/06/2026) (cited on page 105).

[71] React Team. *State: A Component's Memory*. URL: https://react.dev/learn/state-a-components-memory (visited on 08/06/2026) (cited on page 105).

[72] React Team. *Updating Objects in State*. URL: https://react.dev/learn/updating-objects-in-state (visited on 08/06/2026) (cited on page 106).

[73] React Team. *Updating Arrays in State*. URL: https://react.dev/learn/updating-arrays-in-state (visited on 08/06/2026) (cited on page 106).

[74] React Team. *Choosing the State Structure*. URL: https://react.dev/learn/choosing-the-state-structure (visited on 08/06/2026) (cited on page 107).

[75] React Team. *Describing the UI*. URL: https://react.dev/reference/react/useMemo (visited on 08/06/2026) (cited on page 108).

[76] React Team. *Sharing State Between Components*. URL: https://react.dev/learn/sharing-state-between-components (visited on 08/06/2026) (cited on page 109).

[77] React Team. *Extracting State Logic into a Reducer*. URL: https://react.dev/learn/extracting-state-logic-into-a-reducer (visited on 08/06/2026) (cited on page 111).

[78] Zhanyong Wan and Paul Hudak. 'Functional Reactive Programming from First Principles'. In: *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '00)*. Association for Computing Machinery, 2000, pp. 242–252. DOI: 10.1145/349299.349331 (cited on page 111).

[79] React Team. *Passing Data Deeply with Context*. URL: https://react.dev/learn/passing-data-deeply-with-context (visited on 08/06/2026) (cited on page 112).

[80] Zustand contributors. *Zustand: Introduction*. URL: https://zustand.docs.pmnd.rs/learn/getting-started/introduction (visited on 09/09/2026) (cited on page 113).

[81] Redux Team. *Redux Toolkit: Overview*. URL: https://redux.js.org/redux-toolkit/overview (visited on 09/09/2026) (cited on page 113).

[82] Mozilla Developer Network (MDN). *Progressive Web Apps*. URL: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps (visited on 08/19/2026) (cited on page 117).

[83] Mozilla Developer Network (MDN). *What is a Progressive Web App?* URL: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/What_is_a_progressive_web_app (visited on 08/19/2026) (cited on pages 117, 119).

[84] Andreas Biørn-Hansen, Tim A. Majchrzak, and Tor-Morten Grønli. 'Progressive Web Apps for the Unified Development of Mobile Applications'. In: *Web Information Systems and Technologies*. Ed. by Tim A. Majchrzak et al. Vol. 322. Lecture Notes in Business Information Processing. Cham: Springer, 2018, pp. 64–86. DOI: 10.1007/978-3-319-93527-0_4 (cited on pages 117, 133).

[85] Google. *Learn Progressive Web Apps*. URL: https://web.dev/learn/pwa/welcome (visited on 08/19/2026) (cited on page 117).

[86] Thomas Steiner. 'What is in a Web View? An Analysis of Progressive Web App Features When the Means of Web Access is not a Web Browser'. In: *Companion Proceedings of The Web Conference 2018*. International World Wide Web Conferences Steering Committee, 2018, pp. 789–796. DOI: 10.1145/3184558.3188742 (cited on page 117).

[87] Mozilla Developer Network (MDN). *Making PWAs Installable*. URL: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Making_PWAs_installable (visited on 08/19/2026) (cited on pages 119, 120).

[88] World Wide Web Consortium. *Web Application Manifest*. URL: https://w3c.github.io/manifest/ (visited on 08/19/2026) (cited on page 119).

[89] Mozilla Developer Network (MDN). *Web App Manifest*. URL: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Manifest (visited on 08/19/2026) (cited on pages 119, 120).

[90] Google. *Web App Manifest*. URL: https://web.dev/learn/pwa/web-app-manifest (visited on 08/19/2026) (cited on page 120).

[91] Mozilla Developer Network (MDN). *Service Worker API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API (visited on 08/19/2026) (cited on pages 120, 123).

[92] Mozilla Developer Network (MDN). *Using Service Workers*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API/Using_Service_Workers (visited on 08/19/2026) (cited on pages 123–125, 135).

[93] World Wide Web Consortium. *Service Workers*. URL: https://w3c.github.io/ServiceWorker/ (visited on 08/19/2026) (cited on pages 123, 124, 133).

[94] Google. *Service Workers*. URL: https://web.dev/learn/pwa/service-workers (visited on 08/19/2026) (cited on pages 123, 124, 133).

[95] Ivano Malavolta et al. 'Assessing the Impact of Service Workers on the Energy Efficiency of Progressive Web Apps'. In: *2017 IEEE/ACM 4th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. IEEE/ACM, 2017, pp. 35–45. DOI: 10.1109/MOBILESoft.2017.7 (cited on page 123).

[96] Google. *Caching Resources During Runtime*. URL: https://developer.chrome.com/docs/workbox/caching-resources-during-runtime (visited on 08/19/2026) (cited on pages 125, 127).

[97] Mozilla Developer Network (MDN). *CacheStorage*. URL: https://developer.mozilla.org/en-US/docs/Web/API/CacheStorage (visited on 08/19/2026) (cited on page 126).

[98] Mozilla Developer Network (MDN). *Cache*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Cache (visited on 08/19/2026) (cited on page 126).

[99] Google. *workbox-precaching*. URL: https://developer.chrome.com/docs/workbox/modules/workbox-precaching (visited on 08/19/2026) (cited on pages 127, 131).

[100] Mozilla Developer Network (MDN). *Caching*. URL: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Caching (visited on 08/19/2026) (cited on page 127).

[101] Google. *Strategies for Service Worker Caching*. URL: https://developer.chrome.com/docs/workbox/caching-strategies-overview (visited on 08/19/2026) (cited on page 127).

[102] Google. *workbox-expiration*. URL: https://developer.chrome.com/docs/workbox/modules/workbox-expiration (visited on 08/19/2026) (cited on page 131).

[103] Mozilla Developer Network (MDN). *Offline and Background Operation*. URL: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Offline_and_background_operation (visited on 08/19/2026) (cited on page 133).

[104] Kashish Behl and Gaurav Raj. 'Architectural Pattern of Progressive Web and Background Synchronization'. In: *2018 International Conference on Advances in Computing and Communication Engineering (ICACCE)*. IEEE, 2018, pp. 366–371. DOI: 10.1109/ICACCE.2018.8441701 (cited on page 133).

[105] Mozilla Developer Network (MDN). *Web Periodic Background Synchronization API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Web_Periodic_Background_Synchronization_API (visited on 08/19/2026) (cited on page 134).

[106] Mozilla Developer Network (MDN). *Make PWAs Re-engageable Using Notifications and Push APIs*. URL: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Tutorials/js13kGames/Re-engageable_Notifications_Push (visited on 08/19/2026) (cited on pages 134, 135).

[107] Mozilla Developer Network (MDN). *ServiceWorkerRegistration: showNotification() Method*. URL: https://developer.mozilla.org/en-US/docs/Web/API/ServiceWorkerRegistration/showNotification (visited on 08/19/2026) (cited on page 134).

[108] Mozilla Developer Network (MDN). *Permissions API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Permissions_API (visited on 08/19/2026) (cited on page 134).

[109] Google. *Update*. URL: https://web.dev/learn/pwa/update (visited on 08/19/2026) (cited on page 135).

[110] Mozilla Developer Network (MDN). *ServiceWorkerGlobalScope: skipWaiting() Method*. URL: https://developer.mozilla.org/en-US/docs/Web/API/ServiceWorkerGlobalScope/skipWaiting (visited on 08/19/2026) (cited on page 135).

[111] Mozilla Developer Network (MDN). *Clients: claim() Method*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Clients/claim (visited on 08/19/2026) (cited on page 135).

[112] Vite PWA contributors. *Service Worker Strategies and Behaviors*. URL: https://vite-pwa-org.netlify.app/guide/service-worker-strategies-and-behaviors (visited on 08/19/2026) (cited on pages 136, 138).

[113] Vite PWA contributors. *Prompt for New Content Refreshing*. URL: https://vite-pwa-org.netlify.app/guide/prompt-for-update (visited on 08/19/2026) (cited on page 136).

[114] Google. *Debug Progressive Web Apps*. URL: https://developer.chrome.com/docs/devtools/progressive-web-apps (visited on 08/19/2026) (cited on page 136).

[115] Google. *Workbox*. URL: https://developer.chrome.com/docs/workbox (visited on 08/19/2026) (cited on page 137).

[116] Vite PWA contributors. *Vite PWA: Getting Started*. URL: https://vite-pwa-org.netlify.app/guide/ (visited on 08/19/2026) (cited on page 138).

[117] Vite PWA contributors. *Advanced: injectManifest*. URL: https://vite-pwa-org.netlify.app/guide/inject-manifest (visited on 08/19/2026) (cited on page 138).

[118] Vite PWA contributors. *React Integration*. URL: https://vite-pwa-org.netlify.app/frameworks/react (visited on 08/19/2026) (cited on page 139).

[119] Michael Butkiewicz, Harsha V. Madhyastha, and Vyas Sekar. 'Understanding Website Complexity: Measurements, Metrics, and Implications'. In: *Proceedings of the 2011 ACM SIGCOMM Conference on Internet Measurement Conference*. Association for Computing Machinery, 2011, pp. 313–328. DOI: 10.1145/2068816.2068846 (cited on page 142).

[120] Google. *Web Vitals*. URL: https://web.dev/articles/vitals (visited on 08/18/2026) (cited on pages 142, 145).

[121] Mozilla Developer Network (MDN). *Populating the Page: How Browsers Work*. URL: https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/How_browsers_work (visited on 08/18/2026) (cited on page 143).

[122] Mozilla Developer Network (MDN). *Critical Rendering Path*. URL: https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/Critical_rendering_path (visited on 08/18/2026) (cited on page 143).

[123] Mozilla Developer Network (MDN). *Web Performance Guides*. URL: https://developer.mozilla.org/en-US/docs/Web/Performance/Guides (visited on 08/18/2026) (cited on page 146).

[124] Mozilla Developer Network (MDN). *Web Performance*. URL: https://developer.mozilla.org/en-US/docs/Web/Performance (visited on 08/18/2026) (cited on page 147).

[125] Mozilla Developer Network (MDN). *Accessibility*. URL: https://developer.mozilla.org/en-US/docs/Web/Accessibility (visited on 08/18/2026) (cited on page 149).

[126] W3C Web Accessibility Initiative. *Accessibility Principles*. URL: https://www.w3.org/WAI/fundamentals/accessibility-principles/ (visited on 08/18/2026) (cited on page 149).

[127] World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*. Dec. 12, 2024. URL: https://www.w3.org/TR/WCAG22/ (visited on 08/18/2026) (cited on page 149).

[128] Mozilla Developer Network (MDN). *HTML: A Good Basis for Accessibility*. URL: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Accessibility/HTML (visited on 08/18/2026) (cited on pages 150, 151).

[129] Mozilla Developer Network (MDN). *ARIA*. URL: https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA (visited on 08/18/2026) (cited on pages 151, 152).

[130] Mozilla Developer Network (MDN). *ARIA Guides*. URL: https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Guides (visited on 08/18/2026) (cited on page 152).

[131] W3C Web Accessibility Initiative. *Understanding Success Criterion 1.4.3: Contrast (Minimum)*. URL: https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum (visited on 08/18/2026) (cited on page 153).

[132] W3C Web Accessibility Initiative. *Forms Tutorial*. URL: https://www.w3.org/WAI/tutorials/forms/ (visited on 08/18/2026) (cited on page 153).

[133] W3C Web Accessibility Initiative. *Labeling Controls*. URL: https://www.w3.org/WAI/tutorials/forms/labels/ (visited on 08/18/2026) (cited on page 153).

[134] W3C Web Accessibility Initiative. *Validating Input*. URL: https://www.w3.org/WAI/tutorials/forms/validation/ (visited on 08/18/2026) (cited on page 153).

[135] Markel Vigo, Justin Brown, and Vivienne Conway. 'Benchmarking Web Accessibility Evaluation Tools: Measuring the Harm of Sole Reliance on Automated Tests'. In: *Proceedings of the 10th International Cross-Disciplinary Conference on Web Accessibility (W4A '13)*. Association for Computing Machinery, 2013, pp. 1–10. DOI: 10.1145/2461121.2461124 (cited on page 155).

[136] Ian Fette and Alexey Melnikov. *The WebSocket Protocol*. RFC 6455. Internet Engineering Task Force, 2011. DOI: 10.17487/RFC6455. (Visited on 08/06/2026) (cited on pages 158, 161).

[137] WHATWG. *WebSockets Living Standard*. URL: https://websockets.spec.whatwg.org/ (visited on 08/06/2026) (cited on pages 158, 177).

[138] Victoria Pimentel and Bradford G. Nickerson. 'Communicating and Displaying Real-Time Data with WebSocket'. In: *IEEE Internet Computing* 16.4 (2012), pp. 45–53. DOI: 10.1109/MIC.2012.64 (cited on page 160).

[139] Patrick McManus. *Bootstrapping WebSockets with HTTP/2*. RFC 8441. Internet Engineering Task Force, 2018. DOI: 10.17487/RFC8441. (Visited on 08/06/2026) (cited on page 161).

[140] Mozilla Developer Network (MDN). *WebSocket. Web APIs*. URL: https://developer.mozilla.org/en-US/docs/Web/API/WebSocket (visited on 08/06/2026) (cited on page 162).

[141] Mozilla Developer Network (MDN). *WebSocket: close() method. Web APIs*. URL: https://developer.mozilla.org/en-US/docs/Web/API/WebSocket/close (visited on 08/06/2026) (cited on page 164).

[142] C. A. Ellis and S. J. Gibbs. 'Concurrency Control in Groupware Systems'. In: *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, 1989, pp. 399–407. DOI: 10.1145/67544.66963 (cited on page 172).

[143] websockets/ws contributors. *ws: a WebSocket client and server implementation for Node.js*. URL: https://github.com/websockets/ws (visited on 08/06/2026) (cited on page 177).

[144] Ganesan Ramalingam and Kapil Vaswani. 'Fault Tolerance via Idempotence'. In: *Proceedings of the 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. Association for Computing Machinery, 2013, pp. 249–262. DOI: 10.1145/2429069.2429100 (cited on page 178).

[145] OWASP Foundation. *WebSocket Security Cheat Sheet*. URL: https://cheatsheetseries.owasp.org/cheatsheets/WebSocket_Security_Cheat_Sheet.html (visited on 08/06/2026) (cited on pages 179, 180).